

About This Phase

Phases 1 and 2 taught you to write correct Python and to organize it into well-designed classes. Phase 3 is where that becomes software you could genuinely rely on: programs that take command-line arguments, talk to the operating system and to other services, log what they do, run on a schedule, and do several things at once. By the end of these seventeen lessons, you will have built a complete monitoring tool that uses every one of them. Nothing here is filler; every lesson is something the capstone, or a later lesson, genuinely depends on.

Every lesson follows the same structure as before: Why This Matters, Learning Objectives, the lesson itself, Common Mistakes to Avoid, and a Lesson Project.

Contents

Lesson 3.1: Advanced CLI Tools	2
Lesson 3.2: subprocess & Sys Interaction	8
Lesson 3.3: Automation & Batch Processing	13
Lesson 3.4: requests and APIs	19
Lesson 3.5: Env Variables & Config	25
Lesson 3.6: Logging and Error Tracking	31
Lesson 3.7: Schedule & Cron	37
Lesson 3.8: Iterators & itertools	43
Lesson 3.9: Generators & yield	48
Lesson 3.10: functools (partial, lru_cache)	54
Lesson 3.11: Context Managers	60
Lesson 3.12: threading & multiprocessing	66
Lesson 3.13: async/await (asyncio)	73
Lesson 3.14: Error Handling in Concurrency	79
Lesson 3.15: Debugging with pdb	85
Lesson 3.16: Performance Profiling	90
Lesson 3.17: Design Patterns in Python	96
Phase 3 Capstone Project	103

Lesson 3.1: Advanced CLI Tools

Why This Matters

Every script you have written so far has had its inputs written directly into the code: a filename typed into a variable, a number hardcoded into a loop. That works for a program only you will ever run, once. A real tool takes its inputs from whoever runs it, at the moment they run it, from the command line: `python organize.py photos --dry-run`. This lesson teaches you to build programs that behave the way every professional command-line tool you have ever used behaves, with arguments, options, built-in help, and clear errors, using nothing but the standard library.

Learning Objectives

By the end of this lesson, you will be able to:

- Read raw command-line arguments from `sys.argv` and explain why that is not enough for a real tool
- Build a command-line interface with `argparse`: positional arguments, options, and flags
- Validate input automatically with `type`, `choices`, and `default`
- Organize a larger tool into subcommands, and report success or failure with an exit code

The Raw Material: `sys.argv`

When you run a Python file from a terminal, everything typed after the word `python` is handed to your program as a list of strings, available as `sys.argv` from the `sys` module you first met in Lesson 1.15. The first item is always the name of the script itself; every item after it is one argument, exactly as typed.

```
1 import sys
2
3 print(sys.argv)
```

Saved as `demo.py` and run as `python demo.py hello 42 --loud`, this prints `['demo.py', 'hello', '42', '--loud']`. Notice that 42 arrives as the string "42", not the integer, exactly as `input()` did in Lesson 1.3; the terminal has no idea what types you meant, so everything is text. Using `sys.argv` directly works for the tiniest scripts, but the moment a tool needs optional flags, default values, type conversion, or a `--help` screen, you would be writing dozens of lines of fragile string-checking by hand. The standard library already solved that problem, and that solution is what the rest of this lesson is about.

`argparse`: Positional Arguments

The `argparse` module turns a description of your tool's arguments into a complete, working parser. You create a parser, tell it what to expect with `add_argument`, and call `parse_args()` to get the results back as a single object whose attributes are the argument values.

```
1 import argparse
2
3 parser = argparse.ArgumentParser(description="Repeat a word several times.")
4 parser.add_argument("word")
5 parser.add_argument("count", type=int)
6
7 args = parser.parse_args()
8 print((args.word + " ") * args.count)
```

Run as `python repeat.py hello 3`, this prints `hello hello hello`. Trace through it. The call `ArgumentParser(description=...)` builds the parser and stores the text that appears at the top of the help screen. Each `add_argument("word")` call declares a *positional* argument: one identified purely by its position on

the command line, with the name you give it becoming the attribute name on the result. `type=int` tells `argparse` to run `int()` on the raw string before handing it to you, so `args.count` is a real integer, not the string "3". Finally, `parse_args()` reads `sys.argv` for you, matches each argument to its declaration in order, and returns the filled-in object.

The real payoff appears when someone uses the tool incorrectly. Running `python repeat.py hello three` does not crash with a raw traceback; `argparse` prints a usage line plus argument count: `invalid int value: 'three'`, and exits. Running `python repeat.py --help` prints a complete, formatted help screen listing every argument, generated automatically from the declarations you wrote. None of that required a single line of error-handling code from you.

Options and Flags

Arguments starting with dashes are *optional* arguments, usually called options or flags. They can appear in any order, they can be left out entirely, and each can carry a default value.

```
1 import argparse
2
3 parser = argparse.ArgumentParser(description="Greet someone.")
4 parser.add_argument("name")
5 parser.add_argument("-g", "--greeting", default="Hello", help="word to open with")
6 parser.add_argument("-l", "--loud", action="store_true", help="shout the result")
7 parser.add_argument("--times", type=int, default=1)
8
9 args = parser.parse_args()
10
11 message = f"{args.greeting}, {args.name}!"
12 if args.loud:
13     message = message.upper()
14 for _ in range(args.times):
15     print(message)
```

Each option is given a short form (`-g`) and a long form (`--greeting`); the attribute name comes from the long form, with dashes converted to underscores. `default="Hello"` supplies the value when the option is omitted. `action="store_true"` creates a *flag*: an option that takes no value at all, and simply becomes `True` if it appears and `False` if it does not, which makes it the natural tool for on/off switches like `--loud` or `--verbose`. Each help string appears next to its option in the generated `--help` output. The command `python greet.py Ada -g Hi --loud --times 2` prints `HI, ADA!` twice.

Try It Yourself 3.1a: Write a script `shape.py` that takes a positional `width` and `height` (both floats) and an optional `--perimeter` flag. By default it prints the area; with the flag it prints the perimeter instead. Run it with `--help` and confirm your help strings appear.

Validating Input: choices, nargs, and Required Options

Much of what makes a tool trustworthy is refusing bad input early and clearly. `argparse` handles the most common cases declaratively.

```
1 import argparse
2
3 parser = argparse.ArgumentParser()
4 parser.add_argument("files", nargs="+", help="one or more files to process")
5 parser.add_argument("--format", choices=["json", "csv", "text"], default="text")
6 parser.add_argument("--output", required=True)
7
8 args = parser.parse_args()
9 print(args.files, args.format, args.output)
```

nargs="+" means one or more values, gathered into a list (nargs="*" would allow zero or more, and nargs="?" a single optional value). choices=[...] makes argparse reject any value outside the list, printing the allowed choices in the error message, so your own code never has to check. required=True makes an option mandatory even though options are normally optional. Together with type=, these four features cover the great majority of input checking a command-line tool ever needs, and every rejection happens before your own code runs a single line.

Exit Codes: Telling the Shell What Happened

When a program finishes, it hands the operating system a small integer called an *exit code*. By universal convention, 0 means success and any non-zero number means something went wrong. Shell scripts, schedulers (Lesson 3.7), and other programs decide what to do next based on it, so a tool that always exits with 0 even after failing quietly breaks everything built on top of it. `sys.exit()` sets it explicitly, and errors belong on `sys.stderr`, the separate error stream, so they never mix into output someone else is capturing.

```
1 import sys
2 from pathlib import Path
3
4 def main():
5     path = Path(sys.argv[1])
6     if not path.exists():
7         print(f"Error: {path} does not exist.", file=sys.stderr)
8         return 1
9     print(f"{path} is {path.stat().st_size} bytes.")
10    return 0
11
12 if __name__ == "__main__":
13     sys.exit(main())
```

This is the standard shape of a well-behaved command-line program, and it is worth memorizing. All the work lives inside `main()`, which *returns* an exit code rather than calling `sys.exit()` directly, which keeps `main` easy to test. The `if __name__ == "__main__":` guard from Lesson 1.15 runs it only when the file is executed directly, never when it is imported. `sys.exit(main())` passes whatever `main` returned to the operating system. Note also `print(..., file=sys.stderr)`: the `file=` keyword redirects the message to the error stream instead of ordinary output.

Subcommands

Tools such as `git commit`, `pip install`, and `docker run` are not one command with many options; they are families of subcommands, each with its own arguments. `argparse` supports this directly with `add_subparsers`.

```
1 import argparse
2
3 def cmd_add(args):
4     print(f"Adding task: {args.text}")
5
6 def cmd_list(args):
7     print("Listing all tasks" if args.all else "Listing open tasks")
8
9 parser = argparse.ArgumentParser(prog="tasks")
10 subparsers = parser.add_subparsers(dest="command", required=True)
11
12 add_parser = subparsers.add_parser("add", help="add a new task")
13 add_parser.add_argument("text")
14 add_parser.set_defaults(func=cmd_add)
15
16 list_parser = subparsers.add_parser("list", help="show tasks")
17 list_parser.add_argument("--all", action="store_true")
18 list_parser.set_defaults(func=cmd_list)
```

```

19
20 args = parser.parse_args()
21 args.func(args)

```

`add_subparsers` creates the mechanism, and each `add_parser` call creates one subcommand with its own independent set of arguments; tasks `add "buy milk"` and tasks `list --all` therefore accept completely different options. The line `set_defaults(func=cmd_add)` attaches a function to that subcommand, so after parsing, `args.func` is already the correct function for whichever subcommand the user typed, and `args.func(args)` calls it. This uses the idea from Lesson 2.7 that a function is an ordinary value that can be stored and passed around, and it replaces what would otherwise be a growing `if/elif` chain with one line that never needs to change when you add a new subcommand.

Try It Yourself 3.1b: Add a third subcommand done to the example above that takes a positional integer `task_id` and prints a confirmation message. Confirm that `python tasks.py --help` now lists all three subcommands, and that `python tasks.py done --help` shows the help for just that one.

Custom Types and Mutually Exclusive Options

The `type=` argument accepts *any function* that takes the raw string and returns the converted value, and raises `ValueError` or `argparse.ArgumentTypeError` if the text is unacceptable. This lets a parser enforce rules that `int` and `float` alone cannot, such as “a positive integer” or “a path that exists”, with `argparse` turning the failure into a clean usage error.

```

1 import argparse
2 from pathlib import Path
3
4 def positive_int(text):
5     value = int(text)
6     if value <= 0:
7         raise argparse.ArgumentTypeError(f"{text} is not a positive integer")
8     return value
9
10 def existing_folder(text):
11     path = Path(text)
12     if not path.is_dir():
13         raise argparse.ArgumentTypeError(f"{text} is not an existing folder")
14     return path
15
16 parser = argparse.ArgumentParser()
17 parser.add_argument("folder", type=existing_folder)
18 parser.add_argument("--workers", type=positive_int, default=4)
19
20 group = parser.add_mutually_exclusive_group()
21 group.add_argument("--verbose", action="store_true")
22 group.add_argument("--quiet", action="store_true")

```

Each converter is called with the raw text; whatever it returns becomes the attribute’s value, so `args.folder` is already a `Path` object and `args.workers` is already known to be positive by the time your own code runs. `add_mutually_exclusive_group` declares options that make no sense together: passing both `--verbose` and `--quiet` produces an automatic error, again with no checking code of your own. Putting rules like these in the parser, rather than scattered through the program, keeps every input check in one place, exactly the design habit that Lesson 3.5 applies to configuration.

Testing a Command-Line Program Without a Terminal

Running a script by hand after every change is slow. `parse_args` takes an optional list of strings to parse *instead of* `sys.argv`, which lets you test your parser, and your main function, directly from Python.

```

1 import argparse
2
3 def build_parser():
4     parser = argparse.ArgumentParser(prog="greet")
5     parser.add_argument("name")
6     parser.add_argument("--times", type=int, default=1)
7     return parser
8
9 def main(argv=None):
10     args = build_parser().parse_args(argv)
11     for _ in range(args.times):
12         print(f"Hello, {args.name}!")
13     return 0
14
15 if __name__ == "__main__":
16     raise SystemExit(main())
17
18 # elsewhere, in a test:
19 # assert main(["Ada", "--times", "2"]) == 0

```

Two small design choices make this possible. Building the parser inside a function, `build_parser`, means it can be created fresh whenever needed, and giving `main` an `argv=None` parameter means that when it is called with no argument, `parse_args(None)` falls back to the real `sys.argv`, while a caller can pass a list to simulate any command line. `raise SystemExit(main())` is equivalent to `sys.exit(main())`. Structuring every tool this way costs nothing and makes it testable from the start.

Try It Yourself 3.1c: Add a `--count` option to your `greet` tool using a `positive_int` converter. Call `main(["Ada", "--count", "0"])` inside a `try/except SystemExit` and confirm that the parser rejects the value with an error instead of running.

Common Mistakes to Avoid

- **Forgetting that `sys.argv` values are always strings.** Doing arithmetic on `sys.argv[1]` without converting it fails or, worse, silently repeats a string; with `argparse`, declare `type=int` or `type=float` instead.
- **Calling `parse_args()` at import time.** Code at the top level of a module runs when the file is imported, which will break anything that imports it; keep parsing inside `main()` under the `__name__` guard.
- **Always exiting with 0.** A tool that prints an error but still exits successfully looks like a success to every script or scheduler that calls it; return a non-zero code on failure.
- **Printing errors to normal output.** Error messages belong on `sys.stderr`, so they do not pollute output someone is redirecting to a file or piping to another program.
- **Naming an argument with dashes but expecting them in the attribute.** `--dry-run` becomes `args.dry_run`; `argparse` converts the dashes to underscores automatically.

Lesson 3.1 Project: Notes CLI

Write a program called `notes.py` that stores short notes in a JSON file, using the file handling from Lesson 1.13:

1. Uses `argparse` with three subcommands: `add`, `list`, and `delete`, each defined with its own `set_defaults(func=...)` handler.
2. `add` takes a positional note text and an optional `--tag` (default `"general"`) and appends a dictionary with an `id`, the text, and the tag to `notes.json`.
3. `list` takes an optional `--tag` filter and prints every matching note; `delete` takes a positional integer `id` and removes that note.

4. Wraps everything in a `main()` that returns `0` on success and `1` on failure, such as deleting a note that does not exist, printing the error to `sys.stderr`.
5. Handles a missing `notes.json` on the first run by treating it as an empty list, using `try/except` from Lesson 1.14.

This project deliberately combines subcommands, validated arguments, and correct exit codes into one tool that behaves like a real one.

Lesson Summary: Quick Reference

- `sys.argv` is the raw list of command-line strings; `argparse` builds a complete parser on top of it
- Positional arguments are identified by order; options and flags start with dashes and can carry defaults
- `type`, `choices`, `nargs`, and `required` validate input before your code runs, and `--help` is generated automatically
- `sys.exit(main())` with `0` for success and non-zero for failure, with errors sent to `sys.stderr`
- Subcommands (`add_subparsers`) plus `set_defaults(func=...)` keep a large tool organized without an `if/elif` chain

Lesson 3.2: subprocess & Sys Interaction

Why This Matters

Python is rarely the only tool on a machine. A real automation script needs to run `git`, call a compression utility, check a disk with a system command, or launch another Python program, and it needs to know afterward whether that command worked and what it printed. It also needs to ask the interpreter itself questions: what operating system is this, which Python version is running, where does it look for modules? This lesson covers both directions: launching other programs from Python with `subprocess`, and inspecting the environment your own program runs in with `sys`, `platform`, and `shutil`.

Learning Objectives

By the end of this lesson, you will be able to:

- Run an external command with `subprocess.run` and capture its output and exit code
- Explain why passing a list of arguments is safer than passing one shell string
- Handle failing commands, missing programs, and hung processes with `check`, `timeout`, and `try/except`
- Inspect the running interpreter and operating system with `sys`, `platform`, and `shutil.which`

Running a Command with `subprocess.run`

The `subprocess` module launches another program as a separate process, waits for it to finish, and hands you back an object describing what happened. The core function is `subprocess.run`, and the safest way to call it is with the command as a *list*: the program name first, then each argument as its own string.

```
1 import subprocess
2
3 result = subprocess.run(["python", "--version"], capture_output=True, text=True)
4
5 print(result.returncode)
6 print(result.stdout)
7 print(result.stderr)
```

Trace through each piece. The list `["python", "--version"]` is exactly what you would type in a terminal, split into separate words. `capture_output=True` tells Python to collect whatever the program prints instead of letting it appear on your screen. `text=True` decodes that captured output from raw bytes into ordinary strings, using the same encoding idea from Lesson 1.13; without it, `result.stdout` would be a bytes object. The returned `CompletedProcess` object carries three things you will use constantly: `returncode`, the exit code from Lesson 3.1 (0 means success); `stdout`, everything the program printed normally; and `stderr`, everything it printed to the error stream. Note that `run` blocks: your program pauses on that line until the command finishes.

Why a List, Not a String

It is tempting to write one string, `subprocess.run("ls -l my folder")`, and it can even work through a mode called `shell=True`, where Python hands the whole string to the system shell to interpret. The danger is that the shell interprets *everything* in that string, including characters that mean special things.

```
1 import subprocess
2
3 filename = input("File to show: ")
4
5 # DANGEROUS: the shell reads the whole string
6 subprocess.run(f"cat {filename}", shell=True)
7
```

```
8 # SAFE: filename is always one argument, never interpreted
9 subprocess.run(["cat", filename])
```

If a user types `notes.txt; rm -rf ~`, the first version hands the shell two commands separated by a semicolon and happily runs both. This class of bug is called *command injection*, and it is one of the most common serious security mistakes in real software. In the list form, no shell is involved at all: `filename` is passed to the program as exactly one argument, whatever characters it contains, so a hostile-looking string is just an odd filename. The rule is simple and worth treating as absolute: use a list, keep `shell=False` (the default), and only reach for `shell=True` for a fixed command you wrote yourself that genuinely needs shell features like pipes, never with any text that came from outside your program.

When Commands Fail

By default, `run` does not complain if the command fails; it simply records a non-zero `returncode` and carries on, which makes silent failures easy. Passing `check=True` turns a failing exit code into an exception, and two more failure modes deserve their own handling.

```
1 import subprocess
2
3 try:
4     result = subprocess.run(
5         ["git", "status"],
6         capture_output=True, text=True, check=True, timeout=10,
7     )
8     print(result.stdout)
9 except FileNotFoundError:
10     print("git is not installed or not on the PATH.")
11 except subprocess.CalledProcessError as error:
12     print(f"git failed with code {error.returncode}: {error.stderr}")
13 except subprocess.TimeoutExpired:
14     print("git took longer than 10 seconds and was stopped.")
```

Each `except` matches a genuinely different situation, all using the `try/except` structure from Lesson 1.14. `FileNotFoundError` means the program itself does not exist on this machine, before it ever ran. `CalledProcessError` means it ran but exited with a non-zero code, and the exception carries `returncode`, `stdout`, and `stderr` so you can report exactly why. `TimeoutExpired` means it ran longer than `timeout` seconds, at which point Python kills it and raises the exception, protecting your script from hanging forever on a stuck process. A production automation script should treat `timeout` as close to mandatory for any command whose runtime you do not fully control.

Try It Yourself 3.2a: Write a function `run_command(args)` that takes a list, runs it with `capture_output`, `text`, `check`, and a 5-second `timeout`, and returns the stripped `stdout`. Call it with a valid command, a command that fails (for example `["python", "-c", "raise SystemExit(3)"]`), and a program name that does not exist, catching each error and printing a clear message.

Sending Input and Running Python Itself

Some programs read from standard input. `run` accepts an `input=` string to feed them, and a common, robust way to launch another Python script is to use `sys.executable`, the exact path of the interpreter currently running, so the child process uses the same Python and the same virtual environment (Lesson 1.16) as its parent.

```
1 import subprocess
2 import sys
3
4 result = subprocess.run(
5     [sys.executable, "-c", "print(input().upper())"],
6     input="hello from the parent\n",
7     capture_output=True, text=True,
```

```

8 )
9 print(result.stdout.strip())      # HELLO FROM THE PARENT

```

Writing "python" instead of `sys.executable` would launch whichever python happens to be first on the PATH, which may be a different version, or a different environment, from the one your script is using, an extremely common source of confusing `ModuleNotFoundError` bugs. The `-c` option tells Python to run the string that follows as code, which is handy for tiny checks like this one.

Asking the Interpreter and the System Questions

The `sys` module you first used in Lesson 3.1 exposes facts about the running program, and the `platform` and `shutil` modules cover the operating system around it.

```

1 import platform
2 import shutil
3 import sys
4
5 print(sys.version_info >= (3, 10))      # True on Python 3.10 or newer
6 print(sys.executable)                  # path of the running interpreter
7 print(sys.platform)                    # 'win32', 'darwin', or 'linux'
8 print(platform.system())                # 'Windows', 'Darwin', or 'Linux'
9 print(platform.python_version())        # e.g. '3.12.4'
10 print(shutil.which("git"))              # full path to git, or None if missing
11 print(shutil.disk_usage("/").free)      # free bytes on that disk
12 print(sys.path[:2])                    # where Python looks for modules

```

`sys.version_info` compares like a tuple (Lesson 1.8), so `sys.version_info >= (3, 10)` is the standard way for a script to check it is running on a new enough Python. `sys.platform` and `platform.system()` let one script adapt to the operating system, for example choosing "cls" versus "clear", or a different path format. `shutil.which("git")` searches the PATH for a program and returns its full location, or None; checking it first is a cleaner way to detect a missing tool than waiting for `FileNotFoundError`. `sys.path` is the list of directories Python searches when you import something, the same search you learned about in Lesson 1.15, now visible directly.

Try It Yourself 3.2b: Write a script that prints a five-line environment report: the Python version, the operating system name, the interpreter path, whether git is installed (using `shutil.which`), and the free disk space in gigabytes rounded to one decimal place.

Controlling the Child’s Environment and Folder

`subprocess.run` can start the child process in a particular folder with `cwd=`, and with a modified environment through `env=`. This is how a script runs a tool “as if” from another project, or passes a setting down without changing its own environment.

```

1 import os
2 import subprocess
3 import sys
4
5 child_env = os.environ.copy()
6 child_env["APP_MODE"] = "test"
7
8 result = subprocess.run(
9     [sys.executable, "-c", "import os; print(os.getcwd(), os.environ['APP_MODE'])"],
10    cwd="/tmp", env=child_env, capture_output=True, text=True,
11 )
12 print(result.stdout.strip())          # /tmp test

```

The line `os.environ.copy()` is important. Environment variables, from Lesson 3.5, are inherited by every child, and `env=` replaces the child's whole environment with the dictionary you give. If you passed only `{"APP_MODE": "test"}`, the child would lose `PATH` and everything else, and often fail to find its own tools. Copying the current environment and adding to the copy keeps everything and changes just what you intend, without altering your own program's `os.environ`.

Watching Output as It Happens with Popen

`run` waits until the command has completely finished and only then gives you its output, which is unhelpful for a command that runs for minutes and prints progress as it goes. `subprocess.Popen` is the lower-level tool that `run` is built on: it starts the process and returns *immediately*, letting you read the output line by line while it is still running.

```

1 import subprocess
2 import sys
3
4 code = "import time\nfor i in range(3):\n    print('step', i, flush=True)\n    time.sleep(1)\n"
5
6 with subprocess.Popen(
7     [sys.executable, "-c", code],
8     stdout=subprocess.PIPE, stderr=subprocess.STDOUT, text=True,
9 ) as process:
10     for line in process.stdout:
11         print("child says:", line.rstrip())
12
13 print("exit code:", process.returncode)
```

`stdout=subprocess.PIPE` connects the child's output to a pipe you can read, and `stderr=subprocess.STDOUT` merges the error stream into it, so lines arrive in the order they were written. The `for` loop over `process.stdout` uses the line-by-line iteration you first saw with files in Lesson 1.13: each line is handed to you the moment the child prints it, so you see the three steps appear one second apart instead of all at once at the end. The `with` block, as in Lesson 3.11, waits for the process and cleans up when it ends, which is why `returncode` is set afterward. The child's `flush=True` matters: a program writing to a pipe usually buffers its output, so without flushing, the lines might be held back until the buffer fills.

Try It Yourself 3.2c: Write a function `stream(args)` that runs a command with `Popen`, prefixes every output line with the time it arrived (using `datetime.now()`), and returns the exit code. Try it on a Python one-liner that prints five lines with a short `time.sleep` between them.

Common Mistakes to Avoid

- **Building the command as one string with `shell=True`.** Any user-controlled text inside that string can inject extra commands; pass a list of arguments instead.
- **Assuming a failed command raises an error.** Without `check=True`, `run` quietly returns a non-zero `returncode` and your script continues as if nothing happened.
- **Forgetting `text=True`.** Captured output arrives as bytes, so string methods and `print` show `b'...'` instead of readable text.
- **Running a process with no timeout.** One stuck command can freeze an unattended script indefinitely.
- **Hardcoding "python" to launch Python.** Use `sys.executable` so the child runs under the same interpreter and virtual environment.

Lesson 3.2 Project: System Health Report

Write a program called `health_report.py` that:

1. Defines a function `run(args, timeout=10)` that wraps `subprocess.run` with `capture_output`, `text`, and `timeout`, and returns a tuple of `(returncode, stdout, stderr)`, catching `FileNotFoundError` and `TimeoutExpired` and turning each into a sensible tuple instead of crashing.
2. Prints the Python version, platform, and free disk space using `sys`, `platform`, and `shutil`.
3. Uses `shutil.which` to check whether `git` and `python` are available, and, for any that are, runs `[tool, "--version"]` through your `run` function and prints the result.
4. Ends by printing a one-line summary, such as "3 of 3 checks passed", and exits with 0 if everything passed and 1 otherwise, following the exit-code pattern from Lesson 3.1.

This project deliberately combines safe command execution, defensive error handling, and system inspection into one reusable diagnostic tool.

Lesson Summary: Quick Reference

- `subprocess.run(..., capture_output=True, text=True)` runs a command and returns `returncode`, `stdout`, and `stderr`
- Always pass a list, never a string with `shell=True`, so outside text cannot inject commands
- `check=True` raises `CalledProcessError`; `timeout` raises `TimeoutExpired`; a missing program raises `FileNotFoundError`
- Launch Python children with `sys.executable` so they share the parent's interpreter and environment
- `sys`, `platform`, and `shutil.which` let a script inspect its interpreter, operating system, and available tools

Lesson 3.3: Automation & Batch Processing

Why This Matters

The reason most people learn to program at all is to stop doing the same thing by hand: renaming two hundred photos, sorting a downloads folder into tidy subfolders, finding duplicate files, applying one change to every document in a directory. Doing that once takes an afternoon; a script does it in a second, and does it identically every time. This lesson combines the file handling from Lesson 1.13 with the `shutil` and `hashlib` modules into the core skill of automation: finding a batch of files, transforming each one safely, and reporting what happened, without ever letting one bad file ruin the whole run.

Learning Objectives

By the end of this lesson, you will be able to:

- Find batches of files with `Path.glob` and `Path.rglob`, using patterns
- Copy, move, and rename files safely with `shutil` and `Path`
- Build a *dry run* mode that shows what would happen without changing anything
- Process a batch so that one failing file is recorded but does not stop the rest, and finish with a summary

Finding Files: `glob` and `rglob`

Lesson 1.13 introduced `pathlib.Path`. Its `glob` method finds every path in a folder matching a *pattern*, where `*` stands for any run of characters and `?` for any single character. `rglob` does the same, but searches the folder and every subfolder beneath it, recursively.

```
1 from pathlib import Path
2
3 folder = Path("downloads")
4
5 for path in folder.glob("*.pdf"):          # PDFs directly inside downloads
6     print(path.name)
7
8 for path in folder.rglob("*.jpg"):         # JPGs anywhere below downloads
9     print(path)
10
11 files = [p for p in folder.iterdir() if p.is_file()]
12 print(len(files), "files at the top level")
```

Both methods return a lazy sequence of `Path` objects rather than a list, so wrap the call in `list(...)` when you need to count or reuse the results, a behavior Lesson 3.9 explains fully. The `if p.is_file()` filter uses the comprehension form from Lesson 1.10 to skip directories, since `iterdir()` returns everything inside a folder, files and subfolders alike. The attributes you learned in Lesson 1.13, `.name`, `.stem`, and `.suffix`, are exactly what a batch script reads to decide what to do with each file.

Copying, Moving, and Renaming Safely

`shutil` (short for shell utilities) provides the file operations `pathlib` does not: `shutil.copy2` copies a file while preserving its timestamps, `shutil.move` moves a file or folder, even across drives, and `shutil.rmtree` deletes an entire folder tree. `Path.rename` handles renaming within one location, and `Path.mkdir(parents=True, exist_ok=True)` creates a folder, along with any missing parents, without complaining if it already exists.

```
1 import shutil
2 from pathlib import Path
3
4 source = Path("downloads/report.pdf")
```

```
5 target_dir = Path("sorted/documents")
6
7 target_dir.mkdir(parents=True, exist_ok=True)
8 shutil.move(str(source), str(target_dir / source.name))
9
10 backup = Path("backup")
11 backup.mkdir(exist_ok=True)
12 shutil.copy2(target_dir / "report.pdf", backup / "report.pdf")
```

The `/` operator between a `Path` and a string, from Lesson 1.13, builds the destination path portably on any operating system. Notice `str(...)` around the arguments to `shutil.move`; recent Python versions accept `Path` objects directly, but converting keeps the code working on older ones. Most importantly, both `move` and `copy2` will silently *overwrite* a file that already exists at the destination, which is how a careless script destroys data. A safe batch script always checks `destination.exists()` first and decides what to do: skip the file, rename it, or stop.

The Dry Run Pattern

A script that changes hundreds of files should never be run for the first time on real data with no preview. The standard solution is a *dry run*: a mode where the script does all its thinking and prints every action it *would* take, but changes nothing. It is nearly free to build, because the decision logic and the action are already separate steps.

```
1 import shutil
2 from pathlib import Path
3
4 CATEGORIES = {
5     ".jpg": "images", ".png": "images",
6     ".pdf": "documents", ".docx": "documents",
7     ".mp3": "audio", ".zip": "archives",
8 }
9
10 def plan_moves(folder):
11     moves = []
12     for path in folder.iterdir():
13         if path.is_file():
14             category = CATEGORIES.get(path.suffix.lower(), "other")
15             moves.append((path, folder / category / path.name))
16     return moves
17
18 def organize(folder, dry_run=True):
19     for source, destination in plan_moves(folder):
20         if destination.exists():
21             print(f"SKIP (exists): {destination}")
22             continue
23         print(f"{'WOULD MOVE' if dry_run else 'MOVE'}: {source.name} -> {destination}")
24         if not dry_run:
25             destination.parent.mkdir(exist_ok=True)
26             shutil.move(str(source), str(destination))
27
28 organize(Path("downloads"), dry_run=True)
```

The important design decision is the split into `plan_moves`, which only *decides* and returns a list of (source, destination) pairs, and `organize`, which *acts* on them. Because the plan is built without touching anything, the dry-run version and the real version share exactly the same logic, so what the preview shows is guaranteed to be what the real run does. `dry_run=True` as the *default* is deliberate: forgetting to pass anything can only ever produce a harmless preview, never an accidental mass move. Combined with a `--apply` flag from Lesson 3.1, this is how careful real-world file tools are built.

Try It Yourself 3.3a: Extend `organize` so that, when a destination already exists, it appends a number before the extension (`report_1.pdf`, `report_2.pdf`) instead of skipping, using `path.stem` and `path.suffix`, and confirm the dry run prints the new names.

Never Let One File Stop the Batch

When a batch touches hundreds of files, something eventually goes wrong on one of them: a permission error, a file another program has open, a corrupt image. Wrapping the entire loop in one `try/except` means the first failure aborts everything after it. The correct structure is a `try/except` *inside* the loop, so each file succeeds or fails independently, with failures collected for a report at the end.

```

1  from pathlib import Path
2
3  def process_all(paths):
4      succeeded = []
5      failed = []
6      for path in paths:
7          try:
8              text = path.read_text(encoding="utf-8")
9              path.write_text(text.replace("colour", "color"), encoding="utf-8")
10             succeeded.append(path)
11         except (OSError, UnicodeDecodeError) as error:
12             failed.append((path, str(error)))
13
14     print(f"Processed {len(succeeded)} files, {len(failed)} failed.")
15     for path, reason in failed:
16         print(f"  FAILED {path}: {reason}")
17     return failed
18
19 process_all(list(Path("docs").rglob("*.txt")))

```

Catching `OSError` covers the whole family of file-system problems (missing file, no permission, disk full), and `UnicodeDecodeError` covers a file that is not valid UTF-8 text, both named in the tuple form from Lesson 1.14. The function collects every failure with its reason and prints a summary at the end, so an unattended run leaves behind a complete account of what worked and what did not. This *collect errors, keep going, report at the end* shape is the backbone of nearly every batch tool, and Lesson 3.14 extends it to concurrent code.

Finding Duplicates with Hashing

Comparing two large files byte by byte is slow, and comparing names is unreliable, since identical content can have different names. A *hash* is a short fixed-length fingerprint computed from a file's contents: identical content always produces an identical hash, and even a one-byte difference produces a completely different one. The `hashlib` module provides them.

```

1  import hashlib
2  from pathlib import Path
3
4  def file_hash(path, chunk_size=65536):
5      digest = hashlib.sha256()
6      with open(path, "rb") as file:
7          while chunk := file.read(chunk_size):
8              digest.update(chunk)
9      return digest.hexdigest()
10
11 def find_duplicates(folder):
12     seen = {}
13     duplicates = []

```

```

14     for path in folder.rglob("*"):
15         if path.is_file():
16             key = file_hash(path)
17             if key in seen:
18                 duplicates.append((path, seen[key]))
19             else:
20                 seen[key] = path
21     return duplicates

```

`file_hash` reads the file in 64 KB chunks, feeding each into the running hash with `update`, so even a multi-gigabyte file uses only a tiny, fixed amount of memory. The loop `while chunk := file.read(chunk_size)` uses the walrus operator, `:=`, which assigns and tests in one step; the loop stops when `read` returns an empty bytes object at the end of the file. `find_duplicates` maps each hash to the first path seen with it, using a dictionary from Lesson 1.9, so any later file with the same hash is recorded as a duplicate of that first one.

Try It Yourself 3.3b: Add a `report_size` function that takes the `duplicates` list from `find_duplicates` and prints how many bytes would be freed by deleting every duplicate, using `path.stat().st_size`. Do not delete anything; just report.

Selecting Files by Age and Size

Batch jobs very often need to act on *old* or *large* files: clearing logs older than a month, or listing what fills a folder. `Path.stat()` returns information about a file, including `st_size` in bytes and `st_mtime`, the time of last modification, as a number of seconds since a fixed starting point called the *epoch*. Comparing it with the current time from `time.time()` gives a file's age.

```

1  import time
2  from pathlib import Path
3
4  def old_files(folder, days):
5      cutoff = time.time() - days * 24 * 60 * 60
6      for path in folder.rglob("*"):
7          if path.is_file() and path.stat().st_mtime < cutoff:
8              yield path
9
10 def largest_files(folder, count=5):
11     files = [p for p in folder.rglob("*") if p.is_file()]
12     files.sort(key=lambda p: p.stat().st_size, reverse=True)
13     return files[:count]
14
15 for path in old_files(Path("logs"), days=30):
16     print(path)
17
18 for path in largest_files(Path("downloads")):
19     print(f"{path.stat().st_size / 1_000_000:.1f} MB {path}")

```

`old_files` computes the cutoff once, as a timestamp thirty days in the past, and then yields any file whose modification time is earlier; making it a generator, as in Lesson 3.9, means a folder with millions of files is examined without ever building a giant list. `largest_files` uses `key=` with a `lambda`, from Lesson 1.8 and Lesson 1.11, to sort by size, largest first. Combining these with the dry-run pattern of this lesson gives you a safe cleanup tool: print the candidates first, read them carefully, and only then, with an explicit flag, delete.

Renaming in Bulk, Safely

Renaming hundreds of files to a consistent pattern is a classic automation job, and it is also one where an error can scramble names irreversibly. The safe method has three steps: compute every new name, check the whole plan for collisions, and

only then rename.

```

1  from pathlib import Path
2
3  def plan_renames(folder, prefix):
4      plan = []
5      files = sorted(p for p in folder.iterdir() if p.is_file())
6      for number, path in enumerate(files, start=1):
7          new_name = f"{prefix}_{number:03d}{path.suffix.lower()}"
8          plan.append((path, path.with_name(new_name)))
9      return plan
10
11 def check_plan(plan):
12     targets = [new for _, new in plan]
13     if len(targets) != len(set(targets)):
14         raise ValueError("Two files would get the same new name")
15     existing = [new for old, new in plan if new.exists() and new != old]
16     if existing:
17         raise ValueError(f"Would overwrite: {existing[0]}")
18
19 plan = plan_renames(Path("photos"), "holiday")
20 check_plan(plan)
21 for old, new in plan:
22     print(f"{old.name} -> {new.name}")
23     # old.rename(new)      # uncomment only after reading the plan

```

`enumerate(files, start=1)`, from Lesson 1.8, numbers the files from 1, and the format specifier `:03d` from Lesson 1.7 pads the number to three digits with zeros so that the names sort correctly (`holiday_002` before `holiday_010`). `path.with_name(...)` builds a sibling path with a different file name, and `path.suffix.lower()` keeps the extension while tidying its case. The crucial part is `check_plan`: before anything is renamed, it verifies that no two files receive the same name, using the length comparison of a list against a set from Lesson 1.9, and that no target already exists. Only a plan that passes both checks is allowed to run. The same principle, validate the entire plan before touching anything, is what makes a bulk operation trustworthy.

Try It Yourself 3.3c: Write a function that, given a folder and a number of days, returns the total size in megabytes of all files older than that many days, without deleting anything. Test it on a folder of your own, then combine it with `old_files` to print each file that would be removed.

Common Mistakes to Avoid

- **Running a destructive script on real data with no preview.** Build in a dry run, defaulting to *on*, and read its output before running for real.
- **Overwriting files silently.** `shutil.move` and `copy2` replace existing destination files without warning; check `exists()` first.
- **One try/except around the whole loop.** The first failure aborts every remaining file; put the `try` inside the loop and collect errors instead.
- **Modifying a folder while iterating over it.** Moving files out of a directory you are still listing can skip or repeat items; build the plan first, then act on it.
- **Comparing files by name or size alone.** Names and sizes can match while contents differ; hash the contents to detect true duplicates.

Lesson 3.3 Project: File Organizer

Write a program called `organizer.py` that:

1. Uses `argparse` (Lesson 3.1) to accept a positional `folder` and an `--apply` flag; without `--apply`, it runs as a dry run.
2. Builds a plan of moves by file extension into category subfolders, using a dictionary of categories and a fallback "other", in a separate function that touches nothing.
3. Executes the plan (or prints it, in dry-run mode), never overwriting: if a destination exists, adds a numeric suffix to the name.
4. Wraps each individual move in its own `try/except`, collecting failures rather than stopping.
5. Finishes with a summary of files moved, skipped, and failed, and returns exit code 1 if anything failed.

This project deliberately separates deciding from acting, which is what makes a dry run trustworthy.

Lesson Summary: Quick Reference

- `Path.glob("*.ext")` matches in one folder; `rglob` searches every subfolder; both return lazy sequences
- `shutil.move`, `copy2`, and `rmtree` do the heavy lifting, but silently overwrite, so check `exists()` first
- A dry run (default on) prints planned actions without performing them, because planning and acting are separate steps
- Put `try/except` *inside* a batch loop, collect failures, and report a summary at the end
- `hashlib` fingerprints file contents in chunks, making duplicate detection reliable and memory-safe

Lesson 3.4: requests and APIs

Why This Matters

Most useful data does not live on your computer; it lives behind a web address, offered by a service that lets programs, not just browsers, ask for it. Weather, exchange rates, repository statistics, payment status, and the health of a website are all a few lines of Python away, once you know how to talk to an *API*. This lesson teaches you to use the `requests` library, the standard tool for the job, and, more importantly, to do it defensively, because a program that calls the network has to assume the network will sometimes be slow, broken, or wrong.

Learning Objectives

By the end of this lesson, you will be able to:

- Explain what an API, an HTTP request, and a status code are, in plain terms
- Send GET and POST requests with `requests`, including query parameters, headers, and JSON bodies
- Always set a timeout, check the status, and handle the exceptions a network call can raise
- Use a `Session` and a simple retry loop with backoff to make calls more reliable

What an API Actually Is

Your browser loads a web page by sending an *HTTP request* to a server and receiving a *response*. An API (application programming interface) is the same conversation, designed for programs: instead of returning a page of HTML for humans, the server returns structured data, almost always JSON, the format from Lesson 1.13. A request has a *method* saying what you want to do (GET to read data, POST to create it, PUT to replace it, DELETE to remove it), a *URL* saying where, and optionally headers and a body. The response has a *status code*, a three-digit number summarizing the outcome, plus headers and a body. The status codes worth memorizing are 200 (OK), 201 (created), 400 (you sent something invalid), 401 and 403 (not authenticated, or not allowed), 404 (not found), 429 (you are sending too many requests), and anything in the 500s (the server itself failed).

Installing and Making Your First Request

`requests` is not part of the standard library, so install it into your virtual environment with `pip install requests`, exactly as Lesson 1.16 taught, and import it like any other package (Lesson 1.15).

```
1 import requests
2
3 response = requests.get("https://jsonplaceholder.typicode.com/todos/1", timeout=10)
4
5 print(response.status_code)           # 200
6 print(response.headers["Content-Type"])
7 data = response.json()
8 print(data["title"])
```

`requests.get` sends the request and returns a `Response` object. `response.status_code` is the numeric outcome. `response.json()` parses the JSON body into ordinary Python dictionaries and lists, the same structures Lesson 1.9 taught you to work with, so `data["title"]` is a plain dictionary lookup. `response.text` gives the body as a raw string instead, useful for HTML or plain text. The service used here, JSONPlaceholder, is a free practice API that returns fake but realistic data, which makes it ideal for learning without any account or key. Most importantly, notice `timeout=10`. `requests` has *no default timeout at all*: without it, a server that never answers makes your program wait forever. Always pass one.

Query Parameters, Headers, and POST

Data added to a URL after a `?`, like `?userId=1&completed=true`, is called a query string. Building it by hand is error-prone, since special characters need encoding, so pass a dictionary to `params` and let `requests` do it correctly.

```

1 import requests
2
3 response = requests.get(
4     "https://jsonplaceholder.typicode.com/todos",
5     params={"userId": 1, "completed": "true"},
6     headers={"Accept": "application/json", "User-Agent": "learning-python/1.0"},
7     timeout=10,
8 )
9 print(response.url)           # shows the final URL with the query string
10 todos = response.json()
11 print(len(todos), "completed todos")
12
13 new_item = {"title": "Learn requests", "completed": False, "userId": 1}
14 created = requests.post(
15     "https://jsonplaceholder.typicode.com/todos", json=new_item, timeout=10
16 )
17 print(created.status_code)    # 201
18 print(created.json())

```

`params=` builds and encodes the query string, and `response.url` shows you exactly what was sent, which is invaluable for debugging. `headers=` adds request headers; APIs commonly use them for metadata and, as Lesson 3.5 covers, for credentials. `json=new_item` does two jobs at once: it converts the dictionary to a JSON string and sets the Content-Type header to match. That is the form to prefer over building the body by hand.

Try It Yourself 3.4a: Using `JSONPlaceholder`, request `/posts` with the parameter `userId=2`. Print how many posts came back and the title of the first one. Then request a post that does not exist (`/posts/99999`) and print its status code.

Checking the Result and Handling Errors

A 404 or 500 response is *not* an exception; `requests` considers the round trip successful because a response arrived. So parsing the body of a failed response as if it were good data is a classic bug. `raise_for_status()` converts any 4xx or 5xx status into an `HTTPError`. Network-level failures, where no response arrives at all, raise different exceptions, all of which inherit from a single base class, `requests.exceptions.RequestException`.

```

1 import requests
2
3 def fetch_json(url, **kwargs):
4     try:
5         response = requests.get(url, timeout=10, **kwargs)
6         response.raise_for_status()
7         return response.json()
8     except requests.exceptions.Timeout:
9         print("The request timed out.")
10    except requests.exceptions.ConnectionError:
11        print("Could not connect; check the network or the address.")
12    except requests.exceptions.HTTPError as error:
13        print(f"Server returned an error: {error.response.status_code}")
14    except requests.exceptions.JSONDecodeError:
15        print("The response was not valid JSON.")
16    return None
17
18 print(fetch_json("https://jsonplaceholder.typicode.com/todos/1"))
19 print(fetch_json("https://jsonplaceholder.typicode.com/nothing-here"))

```

Each `except` clause corresponds to a different real-world failure, and the order matters exactly as in Lesson 1.14: specific exceptions first. Timeout means the server did not answer in time. `ConnectionError` means the request never reached a server (no internet, wrong host). `HTTPError` fires only because `raise_for_status()` was called, and it carries the response so you can read `error.response.status_code`. The function returns `None` on any failure, so callers must check for it, a simple, explicit contract that beats a program crashing with an unrelated traceback three functions later.

Sessions, Retries, and Backoff

If you make many requests to the same service, create a `requests.Session`. It reuses the underlying connection, which is much faster, and it lets you set headers once for every request. Networks also fail *temporarily*: a server restarts, a connection drops for a second. Retrying such failures a few times, waiting longer between each attempt (*exponential backoff*), turns many transient errors into non-events, and waiting also avoids hammering an already struggling server.

```

1 import time
2 import requests
3
4 def get_with_retry(session, url, attempts=3, base_delay=1.0):
5     for attempt in range(1, attempts + 1):
6         try:
7             response = session.get(url, timeout=10)
8             if response.status_code in (429, 500, 502, 503, 504):
9                 raise requests.exceptions.HTTPError(response=response)
10            response.raise_for_status()
11            return response
12        except (requests.exceptions.ConnectionError,
13                requests.exceptions.Timeout,
14                requests.exceptions.HTTPError) as error:
15            if attempt == attempts:
16                raise
17            delay = base_delay * 2 ** (attempt - 1)    # 1s, 2s, 4s, ...
18            print(f"Attempt {attempt} failed ({error}); retrying in {delay}s")
19            time.sleep(delay)
20
21 with requests.Session() as session:
22     session.headers.update({"User-Agent": "learning-python/1.0"})
23     result = get_with_retry(session, "https://jsonplaceholder.typicode.com/todos/1")
24     print(result.json())

```

The loop tries up to `attempts` times. Only failures likely to be temporary trigger a retry: connection problems, time-outs, and the specific status codes 429 and 5xx; a 404 would not improve by asking again, so it goes straight to `raise_for_status()` and propagates. On the final attempt, a bare `raise` re-raises the original exception so the caller still learns what happened, the same bare `raise` from Lesson 1.14. The delay `base_delay * 2 ** (attempt - 1)` doubles each time. The `with` block, as Lesson 3.11 explains in depth, guarantees the session is closed properly even if an error occurs.

Try It Yourself 3.4b: Write a function `fetch_all_posts(session)` that retrieves `/posts` and returns a dictionary mapping each `userId` to the number of posts that user wrote, using a plain dictionary or `collections.Counter` from Lesson 1.17. Call it through your `get_with_retry` function.

Authentication: API Keys and Tokens

Many services refuse anonymous requests and require you to prove who you are, usually with a key or *token* sent in a header. The most common convention is an `Authorization` header carrying a *bearer token*. The key itself must never appear in your code, so it comes from the environment, as Lesson 3.5 taught.

```

1 import os
2 import requests

```

```

3 token = os.environ["API_TOKEN"]                # KeyError if it is missing: intentional
4
5 session = requests.Session()
6 session.headers.update({
7     "Authorization": f"Bearer {token}",
8     "Accept": "application/json",
9 })
10
11 response = session.get("https://api.example.com/v1/account", timeout=10)
12 if response.status_code == 401:
13     print("The token was rejected; check API_TOKEN.")
14

```

Setting the header once on the Session means every later request through it is authenticated automatically. The status codes from earlier in this lesson now become useful diagnostics: 401 means the credentials were missing or wrong, while 403 means they were valid but do not permit this particular action. Two habits protect the token. Never print it or include it in a log message, since logs are stored and shared, and never put it in a URL, where it is recorded by servers and browsers alike; the header is the correct place.

Pagination: Data That Arrives in Pages

An API will not return a million records in one response. It sends them in *pages*, and expects you to ask for the next one. The details vary by service, but a common form uses `page` and `per_page` parameters and returns an empty list when the pages run out. Turning that into a generator, from Lesson 3.9, hides the paging from the rest of your program completely.

```

1 import requests
2
3 def all_posts(session, per_page=20):
4     page = 1
5     while True:
6         response = session.get(
7             "https://jsonplaceholder.typicode.com/posts",
8             params={"_page": page, "_limit": per_page},
9             timeout=10,
10        )
11        response.raise_for_status()
12        items = response.json()
13        if not items:
14            return
15        yield from items
16        page += 1
17
18 with requests.Session() as session:
19     for count, post in enumerate(all_posts(session), start=1):
20         if count > 45:
21             break
22         print(post["id"], post["title"][:30])

```

Trace it. Each time the generator is asked for another item and has run out of the current page, it loops and fetches the next one, incrementing page. `yield from items` passes each record of a page to the caller one at a time. When a page comes back empty, the bare `return` ends the generator, which ends the caller's loop. The calling code, the `for` loop, is completely unaware that any paging occurs: it just sees a stream of posts, and, being lazy, it fetches pages only as it consumes them, so the `break` after 45 items stops further requests at once instead of downloading everything. Other services return a next link or a cursor token in each response rather than page numbers; the structure is the same, with the loop following the link until none is offered.

Being a Good Citizen: Rate Limits

Servers protect themselves by limiting how many requests a client may make in a given time, and they answer excess requests with status 429. Some include a `Retry-After` header stating how many seconds to wait. Respecting it is both polite and effective, since hammering a server that has told you to slow down usually leads to being blocked.

```

1 import time
2 import requests
3
4 def polite_get(session, url, **kwargs):
5     response = session.get(url, timeout=10, **kwargs)
6     if response.status_code == 429:
7         wait = int(response.headers.get("Retry-After", "5"))
8         print(f"Rate limited; waiting {wait} seconds")
9         time.sleep(wait)
10    response = session.get(url, timeout=10, **kwargs)
11    return response

```

`response.headers` behaves like a dictionary from Lesson 1.9 whose keys ignore capitalization, and `.get` supplies a default of five seconds when the server does not say how long to wait. The `retry` function you wrote earlier in this lesson can absorb this behavior by treating a 429 as retryable, which it already does; the point here is that the *server's* instruction is better information than your own guess.

Try It Yourself 3.4c: Using the `all_posts` generator, count how many posts each `userId` wrote across all pages, using `collections.Counter`. Print the three users with the most posts, and confirm the program made several separate requests by printing `response.url` inside the generator.

Common Mistakes to Avoid

- **Omitting `timeout`.** `requests` waits forever by default, so a single unresponsive server can freeze your program permanently.
- **Treating a 404 or 500 as a success.** These do not raise exceptions on their own; call `raise_for_status()` or check `status_code` before using the body.
- **Catching only `HTTPError` and ignoring connection failures.** Network problems raise different exceptions; catch `RequestException` or the specific subclasses.
- **Retrying everything.** Repeating a 404 or a 400 cannot succeed; retry only connection errors, timeouts, 429, and 5xx.
- **Retrying in a tight loop with no delay.** Hammering a struggling server makes the outage worse and can get you blocked; back off between attempts.

Lesson 3.4 Project: Cached API Client

Write a program called `api_client.py` that:

1. Defines `fetch_json(session, url, params=None)` that uses your `retry` logic, a `timeout`, and `raise_for_status()`, returning parsed JSON or `None` after all attempts fail.
2. Fetches the list of users from `https://jsonplaceholder.typicode.com/users` and each user's todos with a `userId` parameter, all through one shared `Session`.
3. Builds a summary dictionary mapping each user's name to how many todos they have completed and how many remain.
4. Saves the summary to `summary.json` using the JSON handling from Lesson 1.13, and prints a formatted table.
5. Handles a network failure gracefully: if a request fails after all retries, prints a message, skips that user, and continues with the others.

This project deliberately exercises a reliable network client end to end, with the timeout, status, and retry discipline this lesson is built around.

Lesson Summary: Quick Reference

- An API returns structured data (usually JSON) over HTTP; the status code summarizes the outcome
- `requests.get` and `requests.post` send requests; `params=`, `headers=`, and `json=` build them correctly
- *Always* pass `timeout=`, then call `raise_for_status()` before trusting `response.json()`
- Catch `Timeout`, `ConnectionError`, and `HTTPError` separately, or `RequestException` for all of them
- Use a `Session` for repeated calls, and retry only transient failures with exponentially growing delays

Lesson 3.5: Env Variables & Config

Why This Matters

A program that works on your laptop is only half finished. It also has to run on a colleague's machine, on a server, and inside a scheduled job, and each of those has a different database address, a different folder for its files, and, above all, different secrets: API keys, passwords, and tokens. Hardcoding any of these into your source code is one of the most common and most costly beginner mistakes, because code gets shared, copied, and uploaded, and a secret written into it goes with it. This lesson teaches the standard solution: keep configuration *outside* the code, and load it when the program starts.

Learning Objectives

By the end of this lesson, you will be able to:

- Read environment variables with `os.environ` and `os.getenv`, including defaults and required values
- Load a `.env` file with `python-dotenv` and keep secrets out of version control
- Read structured settings from a file with `tomllib` or `configparser`
- Layer defaults, a config file, environment variables, and command-line options into one predictable configuration

Environment Variables

Every process on your computer runs with a set of named text values called *environment variables*, provided by the operating system or by whoever launched the program. You have already met one indirectly: `PATH`, which tells the system where to look for programs. Python exposes them through `os.environ`, which behaves like a dictionary from Lesson 1.9, mapping variable names to string values.

```
1 import os
2
3 print(os.environ["HOME"])           # raises KeyError if missing
4 print(os.environ.get("API_KEY"))    # None if missing
5 print(os.getenv("API_KEY", "not-set")) # default if missing
6 print(os.getenv("DEBUG", "false").lower() == "true")
```

`os.environ["HOME"]` uses square brackets, so a missing name raises `KeyError`, the right choice for a *required* setting where continuing without it makes no sense. `os.getenv(name, default)` is the shortcut for `.get` and returns the default instead, the right choice for an *optional* one. Because every environment variable is a string, a setting like `DEBUG=true` arrives as the text `"true"`, not a boolean, and `PORT=8080` arrives as `"8080"`, so you convert them yourself with `int()` or a comparison, as the last line shows. A very common bug is testing `if os.getenv("DEBUG"):`, which is `True` even for the string `"false"`, since any non-empty string is *truthy* (Lesson 1.5).

Setting a variable depends on the shell: `export API_KEY=abc123` on macOS and Linux, and `set API_KEY=abc123` in a Windows Command Prompt. A variable set this way lasts only for that terminal session, and it is inherited by every program launched from it, including the child processes from Lesson 3.2.

Why Secrets Never Belong in Code

Consider this line: `API_KEY = "sk-live-8f3a..."`. The moment that file is committed to a shared repository, emailed, pasted into a chat for help, or uploaded, the key is exposed, and unlike a bug, an exposed secret cannot be fixed by editing the file afterward, because it remains in the history and in every copy. Automated scanners search public repositories for exactly these patterns within minutes. Reading the key from the environment separates the two concerns cleanly: the *code* is safe to share, and the *secret* lives only on the machine that needs it.

The .env File and python-dotenv

Typing `export` for every variable each time you open a terminal quickly becomes tedious, so a widespread convention is a plain text file named `.env` in your project folder, one `NAME=value` per line:

```
1 # contents of .env (a plain text file, NOT Python)
2 API_KEY=sk-test-12345
3 DATABASE_URL=sqlite:///app.db
4 DEBUG=true
5 REQUEST_TIMEOUT=15
```

The third-party package `python-dotenv` (install it with `pip install python-dotenv`) reads that file and copies its entries into `os.environ` when your program starts:

```
1 import os
2 from dotenv import load_dotenv
3
4 load_dotenv()                                # reads .env from the current folder, if it exists
5
6 api_key = os.environ["API_KEY"]
7 timeout = int(os.getenv("REQUEST_TIMEOUT", "10"))
8 debug = os.getenv("DEBUG", "false").lower() == "true"
9 print(timeout, debug)
```

By default, `load_dotenv()` does *not* override variables that are already set in the real environment, which is exactly what you want: on a production server, the real environment variables win, and the `.env` file is only a convenient stand-in during development. Two habits go with this file. First, add `.env` to your `.gitignore` so it is never committed. Second, commit a copy called `.env.example` containing the same names with fake values, so anyone cloning the project knows which variables to provide.

Try It Yourself 3.5a: Create a `.env` file with a `GREETING` and a `TIMES` value. Write a script that loads it, converts `TIMES` to an integer, and prints the greeting that many times. Then run the script with `TIMES=2` set in the terminal before it, and confirm the real environment value takes priority over the file.

Structured Configuration Files

Environment variables suit secrets and per-machine values, but they are awkward for structured, non-secret settings like a list of sites to check or nested options. For those, use a config file. Modern Python includes `tomllib` for reading TOML, a readable format with sections, from version 3.11 onward. Given this file:

```
1 # contents of config.toml
2 [monitor]
3 interval_seconds = 60
4 timeout = 10
5 sites = ["https://example.com", "https://python.org"]
6
7 [logging]
8 level = "INFO"
9 file = "monitor.log"
```

`tomllib` loads it into ordinary nested dictionaries:

```
1 import tomllib
2 from pathlib import Path
3
```

```

4 with open(Path("config.toml"), "rb") as file:      # note "rb": binary mode
5     config = tomlib.load(file)
6
7 print(config["monitor"]["interval_seconds"])      # 60 (a real int)
8 print(config["monitor"]["sites"])                # a real list
9 print(config["logging"]["level"])                 # 'INFO'

```

Unlike environment variables, TOML keeps types: 60 arrives as an integer and the sites arrive as a real list, with no conversion needed. The file must be opened in binary mode ("rb"), a requirement of `tomllib` itself. The standard library also includes `configparser` for the older INI format ([section] headings with key = value lines), and Lesson 1.13's `json` module works for configuration too; TOML is generally the friendliest to hand-edit, because it allows comments and does not demand commas and quotes around everything.

Layering: Defaults, File, Environment, Command Line

Real tools combine several sources, with a clear rule about which one wins. The usual order, from weakest to strongest, is: built-in defaults, then the config file, then environment variables, then command-line options. Someone typing a flag right now clearly means it more than a value from a file written last month.

```

1 import os
2 import tomlib
3 from dataclasses import dataclass
4 from pathlib import Path
5
6 @dataclass
7 class Settings:
8     interval: int = 60
9     timeout: int = 10
10    log_level: str = "INFO"
11
12 def load_settings(config_path="config.toml", cli_interval=None):
13     settings = Settings()                # 1. defaults
14
15     path = Path(config_path)
16     if path.exists():                    # 2. config file
17         with open(path, "rb") as file:
18             section = tomlib.load(file).get("monitor", {})
19             settings.interval = section.get("interval_seconds", settings.interval)
20             settings.timeout = section.get("timeout", settings.timeout)
21
22     if "MONITOR_TIMEOUT" in os.environ:  # 3. environment
23         settings.timeout = int(os.environ["MONITOR_TIMEOUT"])
24
25     if cli_interval is not None:         # 4. command line
26         settings.interval = cli_interval
27     return settings
28
29 print(load_settings())

```

Each numbered block overwrites only the fields it actually has a value for, so later sources override earlier ones without erasing the rest. Using a dataclass from Lesson 2.9 for `Settings` gives the whole program one well-named object to pass around, with a readable `repr` for free, and the defaults documented in a single place. Concentrating all this loading in one function is the key design habit: the rest of the program never touches `os.environ` or the file directly, so changing where settings come from later means editing exactly one function.

Try It Yourself 3.5b: Add a `MONITOR_LOG_LEVEL` environment override to `load_settings` and a `--timeout` option

(Lesson 3.1) that beats it. Run the script with the value set in `config.toml`, then in the environment, then on the command line, and confirm each source overrides the previous one.

Validating Configuration Before You Use It

A setting can be present and still be wrong: a timeout of -5, an interval of 0, a URL missing its scheme, a log level of "LOUD". If such a value is discovered ten minutes into a run, or, worse, is never discovered and silently produces bad behavior, the fault is far from its cause. The cure is to validate the complete configuration once, at startup, and to fail immediately with a message that says exactly what is wrong.

```

1 from dataclasses import dataclass, field
2
3 VALID_LEVELS = {"DEBUG", "INFO", "WARNING", "ERROR", "CRITICAL"}
4
5 @dataclass
6 class Settings:
7     sites: list = field(default_factory=list)
8     timeout: float = 10.0
9     interval: int = 60
10    log_level: str = "INFO"
11
12    def validate(self):
13        problems = []
14        if not self.sites:
15            problems.append("at least one site is required")
16        for site in self.sites:
17            if not site.startswith(("http://", "https://")):
18                problems.append(f"site {site!r} must start with http:// or https://")
19        if self.timeout <= 0:
20            problems.append(f"timeout must be positive, got {self.timeout}")
21        if self.interval < 1:
22            problems.append(f"interval must be at least 1, got {self.interval}")
23        if self.log_level.upper() not in VALID_LEVELS:
24            problems.append(f"log_level must be one of {sorted(VALID_LEVELS)}")
25        if problems:
26            raise ValueError("Invalid configuration:\n - " + "\n - ".join(problems))
27        return self

```

The important design choice is that `validate` does *not* stop at the first problem. It gathers every problem into a list, then raises a single `ValueError` whose message lists them all, so someone fixing a config file learns about all five mistakes in one run rather than discovering them one at a time. `str.startswith` accepts a tuple of prefixes (Lesson 1.7), which keeps the URL check to one line. The method returns `self` so a caller can write `settings = load_settings().validate()`. Validation belongs at the boundary, where outside data enters the program; once a `Settings` object has passed, the rest of the code may trust it without checking again.

Keeping Secrets Out of Logs and Output

Reading a secret from the environment solves only half of the problem. The other half is not leaking it afterward. Printing a whole `Settings` object for debugging, or logging a failing request with its headers, can copy a token straight into a file that is stored, backed up, and shared. Two practices prevent this. First, keep secrets out of any object you print by default: a dataclass field can be excluded from the automatic `__repr__` with `field(repr=False)`.

```

1 from dataclasses import dataclass, field
2
3 @dataclass
4 class Credentials:
5     username: str

```

```

6 token: str = field(repr=False)
7
8 creds = Credentials("ada", "sk-live-9f8e7d")
9 print(creds)                # Credentials(username='ada')
```

Second, when a value must be shown to prove it was loaded, show only enough to identify it, such as its last four characters, never the whole thing: `f"token loaded (ends in token[-4:])"`. Treat every log line and every error message as something that may one day be read by a stranger. The same caution applies to `.env` files and to any `config.toml` that contains a secret: they belong in `.gitignore`, and if a secret is ever committed by mistake, the only safe response is to revoke it and issue a new one, since removing it from later versions does not remove it from history.

Try It Yourself 3.5c: Extend `Settings.validate` so that it also checks that `interval` is at least as large as `timeout`, since a check that can take longer than the time between checks would overlap the next one. Test it with a config that has three separate mistakes and confirm all three are reported together.

Common Mistakes to Avoid

- **Hardcoding secrets in source code.** Once committed or shared, a key stays in history and copies forever; read it from the environment instead.
- **Committing the `.env` file.** Add it to `.gitignore` and share a `.env.example` with placeholder values instead.
- **Forgetting that environment values are strings.** `PORT` arrives as `"8080"`, and `if os.getenv("DEBUG"):` is true even for `"false"`; convert and compare explicitly.
- **Using `os.environ["X"]` for an optional setting.** A missing variable raises `KeyError`; use `os.getenv` with a default when the value is not required.
- **Opening a TOML file in text mode.** `tomllib.load` needs a file opened with `"rb"`; text mode raises a `TypeError`.

Lesson 3.5 Project: Layered Config Loader

Write a program called `config_loader.py` that:

1. Defines a `Settings` dataclass with at least four fields, each with a sensible default: for example a site list, a timeout, an interval, and a log level.
2. Loads `config.toml` if it exists, using `tomllib`, and overlays any values it defines.
3. Overlays environment variables next, loading a `.env` file first with `python-dotenv`, converting each value to the correct type.
4. Overlays command-line options last, using `argparse` from Lesson 3.1, where an option left out must not overwrite an earlier value.
5. Raises a clear `ValueError` if the final timeout is not a positive number, then prints the final settings and a line stating which source supplied each value.

This project deliberately builds the exact configuration system the course capstone will reuse.

Lesson Summary: Quick Reference

- `os.environ[name]` is required-and-strict; `os.getenv(name, default)` is optional; every value is a string
- Secrets belong in the environment or a `.env` file that is listed in `.gitignore`, never in source code
- `python-dotenv`'s `load_dotenv()` loads `.env` without overriding variables already set for real
- `tomllib` (opened in `"rb"` mode) reads structured, typed settings; `configparser` reads INI files
- Layer sources from weakest to strongest: defaults, file, environment, command line, all inside one loader func-

tion

Lesson 3.6: Logging and Error Tracking

Why This Matters

Every program in this course so far has reported what it is doing with `print()`. That works while you are sitting in front of the screen, and it collapses the moment a program runs unattended: at three in the morning on a server, as a scheduled job, or inside a tool someone else is using. When it fails there, `print` statements have either scrolled away or were never saved, and you are left guessing. Logging is the professional replacement. It records events with a timestamp and a severity, sends them wherever you choose, lets you turn detail up or down without editing code, and preserves the full traceback of every error so you can diagnose a failure long after it happened.

Learning Objectives

By the end of this lesson, you will be able to:

- Explain why logging beats `print` for anything beyond a quick experiment
- Choose the right level for a message, and filter by level
- Configure handlers and formatters to write to the console and a rotating file at once
- Record full tracebacks with `logger.exception` and capture unhandled crashes with `sys.excepthook`

Levels: Severity Built In

The logging module organizes messages by five standard levels, each with a numeric value: `DEBUG` (10) for detailed diagnostic information, `INFO` (20) for confirmation that things are working, `WARNING` (30) for something unexpected that did not stop the program, `ERROR` (40) for a failure that prevented something from working, and `CRITICAL` (50) for a failure so serious the program may not be able to continue. Each has a matching method on a logger.

```
1 import logging
2
3 logging.basicConfig(level=logging.INFO)
4
5 logging.debug("Loading configuration")           # hidden: below INFO
6 logging.info("Monitor started")
7 logging.warning("Disk space is low")
8 logging.error("Could not reach example.com")
```

`basicConfig(level=logging.INFO)` sets the threshold: only messages at that level or higher are shown, so the debug line disappears. That single setting is the heart of logging's value. During development you set `DEBUG` to see everything; in production you set `WARNING` to see only problems; the log statements in the code never change. The default output looks like `INFO:root:Monitor started`, with the level, the logger's name (here `root`), and the message.

Formatting and Named Loggers

A log line is far more useful with a timestamp and the place it came from. The format string controls this, using named fields wrapped in `%(...)`s. And instead of the module-level functions above, real code creates a named logger per module with `logging.getLogger(__name__)`.

```
1 import logging
2
3 logging.basicConfig(
4     level=logging.INFO,
5     format="%(asctime)s | %(levelname)-8s | %(name)s | %(message)s",
6     datefmt="%Y-%m-%d %H:%M:%S",
7 )
8
9 logger = logging.getLogger(__name__)
```

```

10
11 def check_site(url):
12     logger.info("Checking %s", url)
13     logger.warning("Slow response from %s", url)
14
15 check_site("https://example.com")
16 # 2026-09-29 14:03:11 | INFO      | __main__ | Checking https://example.com

```

`%(asctime)s`, `%(levelname)s`, `%(name)s`, and `%(message)s` are placeholders filled in for each record; `-8` pads the level name to a fixed width so columns line up. `__name__` is the module's own name from Lesson 1.15, so each file logs under its own label and you can see instantly where a message came from. Notice too that the messages use `logger.info("Checking %s", url)` with the value passed *separately*, rather than an f-string. The logging system only builds the final string if the message will actually be emitted, so a disabled `DEBUG` call costs almost nothing, and log-analysis tools can still group messages by their unchanging template.

Try It Yourself 3.6a: Configure logging with a format that includes the time, the level, and the function name (`%(funcName)s`). Write two small functions that each log an `INFO` and a `WARNING` message, call them both, then change the level to `WARNING` and confirm the `INFO` lines disappear without touching the functions.

Handlers: Console and File at Once

`basicConfig` is fine for quick scripts, but the general model has three parts. A *logger* is what your code calls. A *handler* decides where records go: the screen, a file, a network service. A *formatter* decides what each line looks like. One logger can have several handlers, each with its own level, which is how a program shows a calm summary on screen while writing every detail to a file.

```

1 import logging
2 from logging.handlers import RotatingFileHandler
3
4 def setup_logging(log_file="monitor.log", console_level=logging.INFO):
5     logger = logging.getLogger("monitor")
6     logger.setLevel(logging.DEBUG)
7
8     formatter = logging.Formatter(
9         "%(asctime)s | %(levelname)-8s | %(name)s | %(message)s"
10    )
11
12    console = logging.StreamHandler()
13    console.setLevel(console_level)
14    console.setFormatter(formatter)
15
16    file_handler = RotatingFileHandler(
17        log_file, maxBytes=1_000_000, backupCount=3, encoding="utf-8"
18    )
19    file_handler.setLevel(logging.DEBUG)
20    file_handler.setFormatter(formatter)
21
22    logger.addHandler(console)
23    logger.addHandler(file_handler)
24    return logger
25
26 log = setup_logging()
27 log.debug("Only in the file")
28 log.info("On screen and in the file")

```

The logger itself is set to `DEBUG` so that nothing is discarded early, and each handler then applies its own, stricter filter:

the console shows INFO and up, while the file receives everything. `RotatingFileHandler` solves a problem every long-running program eventually hits: a log file that grows forever. When the file reaches `maxBytes` (here about one megabyte) it is renamed to `monitor.log.1`, older copies shift up, and only `backupCount` old files are kept, so disk use has a hard ceiling. The number `1_000_000` uses underscores as digit separators, which Python ignores, purely to make large numbers readable.

Recording Errors with Full Tracebacks

When you catch an exception with `try/except`, as in Lesson 1.14, printing only `str(error)` throws away the most valuable information: *where* it happened. `logger.exception` logs a message at ERROR level and automatically appends the complete traceback.

```
1 import logging
2
3 logger = logging.getLogger("monitor")
4
5 def parse_interval(text):
6     try:
7         return int(text)
8     except ValueError:
9         logger.exception("Invalid interval %r, using 60", text)
10        return 60
11
12 parse_interval("soon")
```

`logger.exception` must be called from inside an `except` block, since it reads the exception currently being handled. The record it writes contains your message followed by the full traceback, exactly what Python would have printed had the error gone unhandled, but saved in the log with a timestamp. The `%r` placeholder inserts the value's `repr`, which shows the quotes around a string and so makes accidental spaces or empty strings visible. Using `logger.error(..., exc_info=True)` is the equivalent form when you want a level other than ERROR.

Capturing Crashes You Did Not Anticipate

You cannot wrap every line in `try/except`, and an unattended program that dies with an unexpected exception should leave a trace in its log, not just on a screen nobody is watching. Python calls `sys.excepthook` for any exception that nothing caught and that is about to end the program; replacing it lets you log the crash first.

```
1 import logging
2 import sys
3
4 logger = logging.getLogger("monitor")
5
6 def log_uncaught(exc_type, exc_value, exc_traceback):
7     if isinstance(exc_type, KeyboardInterrupt):
8         sys.__excepthook__(exc_type, exc_value, exc_traceback)
9         return
10    logger.critical("Unhandled exception", exc_info=(exc_type, exc_value, exc_traceback))
11
12 sys.excepthook = log_uncaught
```

The hook receives the exception's type, value, and traceback. The first branch deliberately hands `KeyboardInterrupt` (the user pressing Ctrl+C) back to Python's original hook, `sys.__excepthook__`, because that is a normal way to stop a program, not something to log as a critical failure. Otherwise it logs at CRITICAL with the full traceback supplied through `exc_info`. Dedicated error-tracking services build on precisely this idea, collecting every uncaught exception from every machine in one place, but the mechanism underneath is this hook.

Try It Yourself 3.6b: Set up a `RotatingFileHandler` with a very small `maxBytes` (such as 200) and a `backupCount` of 2. Write thirty log messages in a loop and list the folder to confirm the rotated files `app.log`, `app.log.1`, and `app.log.2` appear and nothing older is kept.

The Logger Hierarchy

Logger names form a tree, separated by dots, and this tree is what makes logging scale to a large program. A logger named `"monitor.checks"` is a *child* of `"monitor"`, which is a child of the *root* logger. A child passes its records up to its parent's handlers, a behavior called *propagation*, and inherits its level if none is set on itself. The consequence is that you configure handlers *once*, on the parent, and every module that uses a child logger is covered automatically.

```

1 import logging
2
3 # configured once, at the entry point
4 parent = logging.getLogger("monitor")
5 parent.setLevel(logging.INFO)
6 handler = logging.StreamHandler()
7 handler.setFormatter(logging.Formatter("%(name)s | %(levelname)s | %(message)s"))
8 parent.addHandler(handler)
9
10 # in checks.py
11 checks_log = logging.getLogger("monitor.checks")
12 # in alerts.py
13 alerts_log = logging.getLogger("monitor.alerts")
14
15 checks_log.info("Checking 3 sites")           # monitor.checks | INFO | Checking 3 sites
16 alerts_log.warning("Alert sent")             # monitor.alerts | WARNING | Alert sent
17
18 logging.getLogger("monitor.checks").setLevel(logging.DEBUG) # more detail from one module

```

Neither child has a handler of its own; both send their records up to `monitor`, whose handler prints them, with the child's full dotted name in the `%(name)s` field so you can tell the sources apart. Setting a level on just one child, as in the last line, raises the detail from a single module while the rest stay quiet, which is invaluable when hunting a bug in one part of a large program. Because every file simply calls `logging.getLogger(__name__)`, as in this lesson, and the entry point does the configuring, no module ever needs to know where the output goes. Libraries you install follow the same rule: `requests` logs through a logger named `"urllib3"`, and you can quiet or amplify it with `logging.getLogger("urllib3").setLevel(logging.WARNING)`.

Structured Logs: One JSON Object per Line

Plain-text log lines are easy for people to read and hard for programs to analyze. Log-collection tools prefer *structured* logs, where each line is a self-contained JSON object, so it can be searched and filtered by field. A custom `Formatter` produces them, using the `json` module from Lesson 1.13.

```

1 import json
2 import logging
3
4 class JsonFormatter(logging.Formatter):
5     def format(self, record):
6         entry = {
7             "time": self.formatTime(record, "%Y-%m-%dT%H:%M:%S"),
8             "level": record.levelname,
9             "logger": record.name,
10            "message": record.getMessage(),
11        }
12        if record.exc_info:
13            entry["traceback"] = self.formatException(record.exc_info)

```

```

14         return json.dumps(entry)
15
16 handler = logging.StreamHandler()
17 handler.setFormatter(JsonFormatter())
18 log = logging.getLogger("monitor")
19 log.addHandler(handler)
20 log.setLevel(logging.INFO)
21
22 log.info("Checked %s in %.2fs", "example.com", 0.31)
23 # {"time": "2026-09-29T14:03:11", "level": "INFO", "logger": "monitor", "message": "Checked
    example.com in 0.31s"}

```

Subclassing `Formatter`, using the inheritance of Lesson 2.4, and overriding its `format` method changes how *every* record is rendered while leaving loggers and handlers untouched. `record.getMessage()` performs the deferred %-style substitution from earlier in this lesson, and if an exception is attached, its traceback is included as one more field. Each output line can then be read back with `json.loads`, which makes it trivial to write a small script that counts errors per hour or extracts every message from one logger.

Try It Yourself 3.6c: Create loggers named "app.db" and "app.web" under a parent "app" that has one handler. Log from both, then set only "app.db" to `DEBUG` and confirm that its debug messages appear while "app.web" debug messages do not.

Common Mistakes to Avoid

- **Using `print` for diagnostics in long-running or unattended programs.** Output is unfiltered, unlabeled, and lost the moment the terminal closes; use a logger.
- **Building the message with an f-string.** Pass values as separate arguments (`logger.info("User %s", name)`) so formatting is skipped for disabled levels.
- **Logging `str(error)` instead of the traceback.** You lose the location of the failure; use `logger.exception` inside the `except` block.
- **Calling `basicConfig` in every module, or after handlers exist.** It only takes effect the first time; configure logging once, at the program's entry point.
- **Letting a log file grow without limit.** Use `RotatingFileHandler` (or a timed one) so a long-running job cannot fill the disk.

Lesson 3.6 Project: Logged Batch Runner

Write a program called `logged_runner.py` that:

1. Defines `setup_logging(verbose)` that creates a named logger with a console handler (`INFO`, or `DEBUG` if `verbose` is true) and a `RotatingFileHandler` that always records `DEBUG`.
2. Accepts a `--verbose` flag using `argparse` from Lesson 3.1.
3. Processes a list of at least six sample inputs (numbers as strings, including some invalid ones like "abc"), converting each to an integer and logging the result at `DEBUG`.
4. Uses `logger.exception` for each invalid value, then keeps going, and finishes with a summary logged at `INFO`.
5. Installs a `sys.excepthook` that logs uncaught exceptions at `CRITICAL`, and deliberately ends the run by triggering an uncaught error, to confirm it reaches the log file.

This project deliberately builds the logging setup that the capstone monitor will use throughout.

Lesson Summary: Quick Reference

- Levels (DEBUG to CRITICAL) let you change how much is recorded without editing any log calls
- Use `logging.getLogger(__name__)` per module, and pass values as arguments rather than f-strings
- Handlers choose the destination and Formatters choose the layout; each handler can have its own level
- `RotatingFileHandler` caps log-file growth; `logger.exception` saves the full traceback
- Replace `sys.excepthook` to log unhandled crashes, and configure logging once at the entry point

Lesson 3.7: Schedule & Cron

Why This Matters

A script you have to remember to run is only half automated. The real payoff comes when it runs by itself: a backup every night, a site check every minute, a report every Monday morning. There are two standard ways to get there. You can keep a Python program running and let it decide when to act, using the `schedule` library. Or you can let the operating system launch your script at the right moments, using `cron` on macOS and Linux, or Task Scheduler on Windows. Knowing both, and when each one fits, is what separates a script from a dependable piece of infrastructure.

Learning Objectives

By the end of this lesson, you will be able to:

- Run a task repeatedly from Python with the `schedule` library and a simple loop
- Read and write a cron expression, and install a job with `crontab`
- Explain the trade-offs between an in-process scheduler and an operating-system scheduler
- Write scripts that behave correctly when launched unattended: absolute paths, logging, and exit codes

The Simplest Scheduler: a Loop and a Sleep

Before using any library, it helps to see the idea in its plainest form. A scheduler is just a loop that does some work and then waits.

```
1 import time
2 from datetime import datetime
3
4 def check():
5     print(f"Checked at {datetime.now():%H:%M:%S}")
6
7 while True:
8     check()
9     time.sleep(60)           # wait one minute, then go around again
```

This runs `check` once a minute, forever, until you press Ctrl+C. `time.sleep(60)` pauses the program for sixty seconds without using any processor time. The `:%H:%M:%S` inside the f-string is the date format syntax from Lesson 1.17, applied directly to a `datetime` object. The approach is honest and it works, but it has real limits. The interval is the time *between the end of one run and the start of the next*, so a task that itself takes ten seconds drifts later each cycle. And it becomes clumsy the moment you want something like “every day at 09:30” or several different tasks on different rhythms.

The schedule Library

The third-party `schedule` package (`pip install schedule`) describes timing in nearly plain English and does the bookkeeping for you.

```
1 import time
2 import schedule
3
4 def check_sites():
5     print("Checking sites...")
6
7 def daily_report():
8     print("Sending report...")
9
10 schedule.every(30).seconds.do(check_sites)
11 schedule.every().hour.do(daily_report)
```

```

12 schedule.every().day.at("09:30").do(daily_report)
13 schedule.every().monday.at("08:00").do(daily_report)
14
15 while True:
16     schedule.run_pending()
17     time.sleep(1)

```

Each line builds a *job*: a rule about when, ending with `.do(function)`, which registers the function to be called. Note that you pass the function *itself*, with no parentheses, the same idea from Lesson 2.7 that a function is a value that can be handed around; writing `.do(check_sites())` would run it immediately and register its return value, a classic bug. Nothing runs by itself, though: `schedule` only *records* the jobs. The loop at the bottom is what brings them to life, because `schedule.run_pending()` looks at the clock, runs any job that is now due, and returns. Sleeping one second between calls keeps the loop light while still checking often enough to be on time to within about a second.

Jobs that need arguments use `do` with extra parameters: `schedule.every().hour.do(check_site, url="https://example.com")` passes the keyword argument through on each run.

Making a Scheduled Task Robust

An unattended loop that runs for weeks must survive its own tasks failing. If `check_sites` raises an exception, it propagates out of `run_pending` and ends the entire loop, so one bad night kills every future run. Wrapping the job in a decorator, using exactly the technique from Lesson 2.7, protects the scheduler from the tasks it runs.

```

1  import logging
2  import time
3  from functools import wraps
4
5  import schedule
6
7  logger = logging.getLogger("scheduler")
8
9  def safe_job(func):
10     @wraps(func)
11     def wrapper(*args, **kwargs):
12         try:
13             return func(*args, **kwargs)
14         except Exception:
15             logger.exception("Job %s failed", func.__name__)
16     return wrapper
17
18 @safe_job
19 def check_sites():
20     raise RuntimeError("network down")
21
22 schedule.every(10).seconds.do(check_sites)
23
24 while True:
25     schedule.run_pending()
26     time.sleep(1)

```

`safe_job` catches any `Exception` raised by the wrapped function, records the full traceback with `logger.exception` from Lesson 3.6, and lets the loop carry on to its next run. It catches `Exception` rather than the broader `BaseException`, so pressing Ctrl+C still stops the program as it should. Applying it as `@safe_job` above each task means no individual job can ever take down the scheduler.

Try It Yourself 3.7a: Write two functions, `tick` and `tock`, that each print a message. Schedule `tick` every 2 seconds and

tock every 5 seconds, run the loop for about 15 seconds, and confirm from the output that the two jobs run independently on their own rhythms.

cron: Letting the Operating System Do It

An in-process scheduler has one inherent weakness: it only works while your Python program is running. If the machine reboots, or the process is killed, nothing runs until someone starts it again. The alternative is to have the operating system launch a short-lived script at each scheduled time. On macOS and Linux, that service is cron, configured with a table called the *crontab*. Each line has five time fields followed by the command:

```
1 # minute hour day-of-month month day-of-week command
2 30 9 * * * /usr/bin/python3 /home/me/report.py
3 0 */6 * * * /usr/bin/python3 /home/me/backup.py
4 */5 * * * 1-5 /usr/bin/python3 /home/me/check.py
```

Reading left to right: minute (0 to 59), hour (0 to 23), day of the month (1 to 31), month (1 to 12), and day of the week (0 to 7, where both 0 and 7 mean Sunday). A *** means “every value”, **/5* means “every fifth value”, a dash gives a range, and a comma gives a list. So the first line runs the report at 09:30 every day, the second runs the backup at minute 0 of every sixth hour, and the third runs the check every five minutes on weekdays (Monday to Friday). You edit your crontab with `crontab -e`, list it with `crontab -l`, and cron handles the rest, including starting again after a reboot. Windows offers the same capability through Task Scheduler, configured through its own interface or the `schtasks` command; the ideas below apply equally.

Scripts That Behave Well When Run by cron

A script launched by the scheduler does not run in your familiar terminal. It has no interactive session, a minimal environment, and an unpredictable working directory. Several habits make a script robust there.

```
1 import logging
2 import sys
3 from pathlib import Path
4
5 BASE_DIR = Path(__file__).resolve().parent # folder containing THIS file
6 LOG_FILE = BASE_DIR / "check.log"
7 CONFIG_FILE = BASE_DIR / "config.toml"
8
9 logging.basicConfig(
10     filename=LOG_FILE, level=logging.INFO,
11     format="%(asctime)s %(levelname)s %(message)s",
12 )
13
14 def main():
15     logging.info("Run started")
16     try:
17         # ... the real work goes here ...
18         logging.info("Run finished")
19         return 0
20     except Exception:
21         logging.exception("Run failed")
22         return 1
23
24 if __name__ == "__main__":
25     sys.exit(main())
```

`__file__` is a built-in variable holding the path of the running script, and `Path(__file__).resolve().parent` turns it into the absolute folder the script lives in. Building every other path from that `BASE_DIR`, rather than a relative name

like "config.toml", means the script finds its files no matter which directory cron happens to start it from, the single most common cause of “it works when I run it but not from cron”. Logging to a file, since there is no screen, and returning the exit code from Lesson 3.1 complete the picture: the log records what happened, and the exit code tells the scheduler whether the run succeeded. In a crontab, use full absolute paths to both the interpreter and the script, and for a virtual environment (Lesson 1.16) use the environment’s own python directly, such as /home/me/project/.venv/bin/python.

Choosing Between Them

Use an in-process scheduler like `schedule` when the program is a long-running service that already stays alive (a bot, a monitor, a dashboard), when you want sub-minute intervals, or when the timing logic is part of the application. Use `cron` or Task Scheduler when the job is a short, self-contained script, when it must survive reboots and crashes without you, or when you simply want a proven scheduler that needs no code of its own. A reliable pattern combines the two ideas: write the script so it does one complete unit of work and exits, then run it from whichever scheduler you like, since a script built that way works under both.

Try It Yourself 3.7b: Write the crontab lines (as plain text; you do not need to install them) for: a script every day at 06:15; a script every 10 minutes; a script at 23:00 only on Sundays; and a script on the first day of every month at midnight. Check each against the five-field order.

Preventing Overlapping Runs with a Lock File

A scheduler does not care whether the previous run has finished. If a job scheduled every five minutes occasionally takes seven, cron starts a second copy while the first is still working, and now two copies are writing to the same files and sending the same alerts. The standard defence is a *lock file*: at the start, the script tries to create a file that marks it as running; if the file already exists, another copy is active and this one exits quietly.

```

1  import os
2  import sys
3  from contextlib import contextmanager
4  from pathlib import Path
5
6  @contextmanager
7  def single_instance(lock_path):
8      lock_path = Path(lock_path)
9      try:
10         descriptor = os.open(lock_path, os.O_CREAT | os.O_EXCL | os.O_WRONLY)
11     except FileExistsError:
12         raise SystemExit("Another run is already in progress; exiting.")
13     try:
14         os.write(descriptor, str(os.getpid()).encode())
15         yield
16     finally:
17         os.close(descriptor)
18         lock_path.unlink(missing_ok=True)
19
20 def main():
21     with single_instance(Path(__file__).with_name("job.lock")):
22         print("Doing the work...")
23     return 0
24
25 if __name__ == "__main__":
26     sys.exit(main())

```

The heart of it is the flag combination `os.O_CREAT | os.O_EXCL`, which means “create this file, but fail if it already exists.” The operating system performs that check-and-create as a single indivisible step, so two copies starting at the same instant cannot both succeed; one gets `FileExistsError`. Writing the process id into the file is a courtesy that lets a human see which process holds the lock. The `@contextmanager` from Lesson 3.11 guarantees the lock is removed in

the `finally` block even if the work raises an error, and `unlink(missing_ok=True)` avoids a second error if the file has somehow already gone. One limitation to know about: if the program is killed outright, the `finally` never runs and a stale lock file is left behind, blocking every later run until someone deletes it. Production tools handle this by checking whether the process id stored in the file is still alive, but the simple version above is right for most personal automation.

Deciding When Not to Run: Skipping and Catching Up

Scheduled jobs meet real-world conditions: the machine was off at the scheduled moment, the network is down, or a run is only sensible on business days. Handling these belongs in the job itself. A job can check a simple condition first and return early, keeping the schedule simple and the decision explicit.

```

1 import datetime
2 import socket
3
4 def has_internet(host="1.1.1.1", port=53, timeout=3):
5     try:
6         with socket.create_connection((host, port), timeout=timeout):
7             return True
8     except OSError:
9         return False
10
11 def weekday_report():
12     today = datetime.date.today()
13     if today.weekday() >= 5:                # 5 = Saturday, 6 = Sunday
14         print("Weekend: skipping the report.")
15         return
16     if not has_internet():
17         print("No network: skipping this run.")
18         return
19     print(f"Running the report for {today}")

```

`date.weekday()`, from the `datetime` tools of Lesson 1.17, returns 0 for Monday through 6 for Sunday, so a value of 5 or more means the weekend. The network check tries to open a connection with a short timeout and treats any `OSError`, the base of the connection errors, as “offline”. Both checks make the job safe to schedule without any cleverness in the scheduler itself. Note the trade-off with `cron`: an operating-system scheduler does not run a job that was missed while the computer was off, so anything that must not be skipped either needs a machine that stays on, or a job that on start-up looks at when it last succeeded, for example a timestamp saved in a file, and catches up.

Try It Yourself 3.7c: Write a script that acquires the lock file using `single_instance`, sleeps for 10 seconds, then exits. Start it in two terminals at nearly the same time and confirm that the second one exits immediately with the “already in progress” message.

Common Mistakes to Avoid

- **Writing `.do(func())` instead of `.do(func)`.** The parentheses call the function immediately; pass the function itself so the scheduler calls it later.
- **Forgetting the `run_pending()` loop.** `schedule` only records jobs; without the loop nothing ever runs.
- **Letting one failed job crash the scheduler.** An uncaught exception ends the whole loop; wrap tasks so failures are logged and the loop continues.
- **Using relative paths in a scheduled script.** The working directory under `cron` is unpredictable; build paths from `Path(__file__).resolve().parent`.
- **Assuming `cron` has your terminal’s environment.** It starts with a minimal `PATH` and no activated virtual environment; use absolute paths to the interpreter and script.

Lesson 3.7 Project: Scheduled Reporter

Write a program called `scheduled_reporter.py` that:

1. Builds every file path from the script's own folder, using `Path(__file__).resolve().parent`, and logs to a file with the logging setup from Lesson 3.6.
2. Defines a `safe_job` decorator, as in this lesson, and applies it to at least three tasks with different rhythms: for example a quick heartbeat, a periodic check, and a daily summary.
3. Schedules the tasks with `schedule`, at short intervals so you can watch them run, and runs the standard `run_pending` loop.
4. Deliberately makes one task raise an exception every third call, and confirms from the log file that the failure was recorded and the other jobs kept running.
5. Ends the file with a comment block containing the exact `crontab` line that would run the same script's single-run mode every 15 minutes, with all paths absolute.

This project deliberately shows one script working under both scheduling styles, and surviving failure in either.

Lesson Summary: Quick Reference

- A scheduler is a loop plus a sleep; `schedule` adds readable rules (`every().day.at("09:30").do(func)`) and needs a `run_pending()` loop
- Pass the function itself to `.do`, and wrap tasks in a decorator so one failure cannot stop the loop
- A cron line has five fields: minute, hour, day of month, month, day of week; `*`, `*/n`, ranges, and lists express repetition
- Unattended scripts need absolute paths, file logging, and a meaningful exit code
- Prefer cron or Task Scheduler for short jobs that must survive reboots; prefer `schedule` inside a long-running service

Lesson 3.8: Iterators & itertools

Why This Matters

You have written for loops since Lesson 1.6, over lists, strings, dictionaries, ranges, and even files, without ever asking how a single loop can work on so many different things. The answer is a small, uniform protocol called the *iterator protocol*, and it is the foundation for a whole family of Python's most powerful features: generators (Lesson 3.9), lazy file processing, and the `itertools` module, a toolkit for looping, combining, grouping, and slicing data efficiently. Understanding it turns for from magic into something you can extend to your own classes.

Learning Objectives

By the end of this lesson, you will be able to:

- Explain the difference between an *iterable* and an *iterator*, and what `iter()` and `next()` do
- Explain what a for loop does behind the scenes, including how it knows to stop
- Write your own iterator class with `__iter__` and `__next__`
- Use `itertools` to chain, slice, combine, group, and accumulate data without building intermediate lists

Iterables and Iterators

An *iterable* is anything you can loop over: a list, a string, a dictionary, a file. Formally, it is an object that can produce an iterator when asked. An *iterator* is the object that actually does the walking; it remembers its position and hands out one item at a time. The built-in `iter()` converts an iterable into an iterator, and `next()` asks an iterator for its following item.

```
1 colors = ["red", "green", "blue"]
2
3 iterator = iter(colors)
4 print(next(iterator))      # red
5 print(next(iterator))      # green
6 print(next(iterator))      # blue
7 print(next(iterator))      # raises StopIteration
```

Each call to `next` advances the iterator by one position and returns the item there. When nothing is left, it raises `StopIteration`, a special exception whose only job is to say “I am finished”. Notice that the list `colors` and its iterator are two separate things: the list holds the data, while the iterator holds only a bookmark into it. That is why you can create several independent iterators over the same list, each with its own position, and why an iterator, once exhausted, stays exhausted: to loop again you need a fresh one from `iter()`.

What a for Loop Really Does

Now the mystery of `for` dissolves. This loop:

```
1 for color in colors:
2     print(color)
```

is exactly equivalent to the following, which spells out every step:

```
1 iterator = iter(colors)
2 while True:
3     try:
4         color = next(iterator)
5     except StopIteration:
6         break
```

```
print(color)
```

Python calls `iter()` on the thing after `in` once, then repeatedly calls `next()`, running the loop body each time, and quietly catching `StopIteration` to end the loop cleanly. This is precisely why `for` works on so many types: it does not care what the object is, only that `iter()` works on it and the result answers to `next()`. It is also why comprehensions, `list()`, `sum()`, `sorted()`, and the `in` operator all accept any iterable; they are all built on the same two calls.

Writing Your Own Iterator

A class becomes an iterator by defining two magic methods from Lesson 2.6: `__iter__`, which returns the iterator object (usually just `self`), and `__next__`, which returns the next item or raises `StopIteration`.

```
1 class Countdown:
2     def __init__(self, start):
3         self.current = start
4
5     def __iter__(self):
6         return self
7
8     def __next__(self):
9         if self.current <= 0:
10            raise StopIteration
11        value = self.current
12        self.current -= 1
13        return value
14
15 for number in Countdown(3):
16     print(number)           # 3, 2, 1
17
18 print(list(Countdown(5)))  # [5, 4, 3, 2, 1]
```

Trace the loop on `Countdown(3)`. `for` calls `iter()`, which calls `__iter__` and gets `self` back. It then calls `__next__`, which sees `current` is 3, stores it, decrements `current` to 2, and returns 3. This repeats until `current` reaches 0, at which point `__next__` raises `StopIteration` and the loop ends. The object carries its own state in `self.current`, using the instance attributes from Lesson 2.1 and the state-changing methods from Lesson 2.2. Because the countdown is computed on demand, `Countdown(1_000_000)` takes no more memory than `Countdown(3)`: it never builds a list of a million numbers, it produces each one only when asked. This idea, computing values *lazily*, is the central theme of the next two lessons.

Try It Yourself 3.8a: Write an iterator class `Evens` that takes a `limit` and yields the even numbers from 0 up to but not including the limit. Loop over `Evens(10)` with a `for` loop, and confirm it prints 0, 2, 4, 6, 8.

itertools: Building Blocks for Loops

The standard library's `itertools` module contains functions that take iterables and return iterators, all lazy. They cover the loops people otherwise write by hand, more concisely and with far less memory. Start with the ones for producing and combining sequences.

```
1 from itertools import chain, count, cycle, islice
2
3 print(list(chain([1, 2], (3, 4), "ab")))  # [1, 2, 3, 4, 'a', 'b']
4
5 print(list(islice(count(10, 5), 4)))      # [10, 15, 20, 25]
6
7 colors = cycle(["red", "green"])
8 print([next(colors) for _ in range(5)])  # ['red', 'green', 'red', 'green', 'red']
```

chain joins several iterables into one continuous stream without copying them into a new list. `count(start, step)` is an *infinite* counter; on its own it would never end, which is safe here only because `islice(iterable, stop)` takes just the first few items, the lazy equivalent of the slice `items[:4]` from Lesson 1.8, but working on any iterator. `cycle` repeats a sequence endlessly, handy for round-robin assignment. The rule to remember with every infinite iterator is that something downstream must stop it, whether `islice`, a `break`, or a condition.

Combinations, Grouping, and Running Totals

Another group handles the combinatorial and summarizing jobs.

```

1 from itertools import accumulate, combinations, groupby, product, zip_longest
2
3 print(list(product("AB", [1, 2])))
4 # [('A', 1), ('A', 2), ('B', 1), ('B', 2)]
5
6 print(list(combinations(["x", "y", "z"], 2)))
7 # [('x', 'y'), ('x', 'z'), ('y', 'z')]
8
9 print(list(accumulate([1, 2, 3, 4])))          # [1, 3, 6, 10]
10
11 print(list(zip_longest([1, 2, 3], ["a"], fillvalue="-")))
12 # [(1, 'a'), (2, '-'), (3, '-')]

```

`product` produces every pairing across its inputs, the nested loops you would otherwise write by hand. `combinations(items, r)` produces every way to choose `r` items regardless of order, each combination only once. `accumulate` yields the running total after each item. `zip_longest` is the version of `zip` from Lesson 1.8 that keeps going until the *longest* input is exhausted, padding the shorter ones with `fillvalue`, where plain `zip` silently stops at the shortest.

`groupby` deserves extra care, because it has one behavior that surprises almost everyone. It groups *consecutive* items that share a key, and therefore the data must be sorted by that key first.

```

1 from itertools import groupby
2
3 logs = [("ERROR", "disk"), ("INFO", "start"), ("ERROR", "net"), ("INFO", "stop")]
4
5 logs.sort(key=lambda entry: entry[0])
6 for level, group in groupby(logs, key=lambda entry: entry[0]):
7     print(level, [message for _, message in group])
8 # ERROR ['disk', 'net']
9 # INFO ['start', 'stop']

```

Without the `sort`, the two `ERROR` entries are not adjacent, so `groupby` would produce four separate groups instead of two. Each group is itself a lazy iterator that is only valid until you advance to the next one, so consume it (here with a comprehension) before moving on.

Try It Yourself 3.8b: Use `itertools.product` to generate every combination of the sizes `["S", "M", "L"]` and the colors `["red", "blue"]`, printing each as a string like `"M-blue"`. Then use `accumulate` on the list `[5, 10, 15, 20]` to print a running total.

Iterable versus Iterator: Why the Difference Matters

The Countdown class earlier in this lesson is its own iterator: `__iter__` returns `self`, so once the loop has used it up, it is finished for good. That is a fault when the object is meant to represent a *collection* you can loop over many times. The remedy is to separate the two roles: the collection is an iterable whose `__iter__` returns a *fresh iterator* every time it is called.

```

1 class Playlist:
2     def __init__(self, songs):
3         self.songs = songs
4
5     def __iter__(self):
6         return PlaylistIterator(self.songs)
7
8 class PlaylistIterator:
9     def __init__(self, songs):
10        self.songs = songs
11        self.position = 0
12
13    def __next__(self):
14        if self.position >= len(self.songs):
15            raise StopIteration
16        song = self.songs[self.position]
17        self.position += 1
18        return song
19
20 playlist = Playlist(["A", "B", "C"])
21 print(list(playlist))      # ['A', 'B', 'C']
22 print(list(playlist))      # ['A', 'B', 'C'] again: a new iterator was made

```

Each for loop calls `iter(playlist)`, which builds a new `PlaylistIterator` with its own position, so two loops, or two nested loops, over the same playlist do not interfere. This is exactly how a list behaves: it is an iterable, while the object returned by `iter(my_list)` is a separate, single-use iterator. It also explains a fact from earlier in this lesson: iterators are single-use precisely because they *are* the position marker. When you design your own class, a good rule is to make the collection an iterable, and let a generator provide the iterator, since `def __iter__(self): yield from self.songs` does the whole job in one line, as Lesson 3.9 shows.

More itertools Recipes

Four more functions from `itertools` solve everyday problems that otherwise need hand-written loops.

```

1 from itertools import chain, dropwhile, pairwise, starmap, takewhile
2
3 readings = [3, 5, 9, 14, 20, 8, 4]
4
5 print(list(takewhile(lambda r: r < 15, readings)))    # [3, 5, 9, 14]
6 print(list(dropwhile(lambda r: r < 15, readings)))    # [20, 8, 4]
7
8 print(list(pairwise(readings)))
9 # [(3, 5), (5, 9), (9, 14), (14, 20), (20, 8), (8, 4)]
10 changes = [b - a for a, b in pairwise(readings)]
11 print(changes)                                         # [2, 4, 5, 6, -12, -4]
12
13 print(list(starmap(pow, [(2, 3), (3, 2), (10, 3)])))    # [8, 9, 1000]
14
15 files = ["a.txt", "b.txt"], ["c.txt"], []
16 print(list(chain.from_iterable(files)))                # ['a.txt', 'b.txt', 'c.txt']

```

`takewhile` yields items *while* a condition holds and stops permanently at the first failure, and `dropwhile` skips items while the condition holds and then yields everything that remains; neither looks past the first change. `pairwise`, available from Python 3.10, yields each item together with its successor, which turns the common “compare each value with the next one” loop into a one-line comprehension. `starmap` is `map` for functions that take several arguments: each tuple in the input is *unpacked* into the call, so `(2, 3)` becomes `pow(2, 3)`. `chain.from_iterable` flattens one level of

nesting lazily, taking a single iterable of iterables, where plain `chain` needs them as separate arguments. Reaching for these before writing a manual loop makes code shorter, and the intent is visible from the function name.

Try It Yourself 3.8c: Given a list of daily temperatures, use `pairwise` to find the largest single-day increase, and use `takewhile` to collect the temperatures up to the first one above 30. Use `chain.from_iterable` to flatten a list of three word lists into one.

Common Mistakes to Avoid

- **Expecting an iterator to be reusable.** Once exhausted, an iterator yields nothing; create a new one with `iter()` (or call the function again) to loop a second time.
- **Forgetting `raise StopIteration` in a custom iterator.** Without it, `__next__` never signals the end and the loop runs forever.
- **Running `list()` on an infinite iterator.** `count()` and `cycle()` never end; limit them first with `islice` or a `break`.
- **Using `groupby` on unsorted data.** It groups only *adjacent* items with equal keys, so sort by the same key beforehand.
- **Keeping a `groupby` group after advancing.** Each group is invalidated when the loop moves on; convert it with `list()` or use it immediately.

Lesson 3.8 Project: Lazy Data Toolkit

Write a program called `lazy_toolkit.py` that:

1. Defines an iterator class `Fibonacci` that produces Fibonacci numbers forever, using `__iter__` and `__next__`, and uses `itertools.islice` to print the first fifteen.
2. Defines a function `batches(iterable, size)` using `islice` inside a `while` loop that yields successive lists of size items, and demonstrates it on `range(23)`.
3. Uses `groupby` (after sorting) to group a list of at least ten words by their first letter, printing each group.
4. Uses `product` to generate all two-character codes from the letters "ABC" and digits "12", and combinations to list every pair from a list of five names.
5. Prints the running total of a list of daily sales figures with `accumulate`.

This project deliberately keeps every step lazy, so nothing is turned into a list until the moment its result is printed.

Lesson Summary: Quick Reference

- An iterable can produce an iterator with `iter()`; an iterator hands out items with `next()` and raises `StopIteration` when finished
- A `for` loop is just repeated `next()` calls with `StopIteration` handled for you
- A class becomes an iterator by defining `__iter__` (return `self`) and `__next__`
- `chain`, `islice`, `count`, `cycle`, `product`, `combinations`, `accumulate`, and `zip_longest` build and combine sequences lazily
- `groupby` only groups adjacent equal keys, so sort first; always limit infinite iterators

Lesson 3.9: Generators & yield

Why This Matters

Lesson 3.8 ended with a custom iterator class: about ten lines of setup just to count down. It worked, but it had a lot of ceremony, a class, two magic methods, and hand-managed state, all to say something simple. Generators are Python's answer. A generator function looks like an ordinary function with one new keyword, `yield`, and Python builds the entire iterator machinery for you behind the scenes. Generators are how professional Python code processes files too large to fit in memory, streams data through a series of steps, and represents sequences that never end.

Learning Objectives

By the end of this lesson, you will be able to:

- Write a generator function with `yield` and explain how it differs from a function that `returns`
- Explain lazy evaluation and why generators use a tiny, constant amount of memory
- Write generator expressions, and delegate to another generator with `yield from`
- Build a multi-stage data pipeline out of generators that processes a large file line by line

yield: A Function That Pauses

Compare two functions that produce the numbers from 1 to `n`. The first, using the tools from Lesson 1.11, builds a whole list and hands it back:

```
1 def numbers_list(n):
2     result = []
3     for i in range(1, n + 1):
4         result.append(i)
5     return result
6
7 def numbers_gen(n):
8     for i in range(1, n + 1):
9         yield i
10
11 gen = numbers_gen(3)
12 print(next(gen))      # 1
13 print(next(gen))      # 2
14 print(next(gen))      # 3
15 print(next(gen))      # raises StopIteration
```

A function containing `yield` is a *generator function*, and calling it does *not* run its body. It returns a generator object, a ready-made iterator of exactly the kind from Lesson 3.8. Each `next()` runs the body until it reaches a `yield`, hands that value back, and then *pauses* right there, remembering every local variable and the exact line it stopped on. The next `next()` resumes from that very spot. When the function finally runs off its end, Python raises `StopIteration` automatically. So `numbers_gen` does everything the `Countdown` class did, in three lines, with no class and no bookkeeping, because the paused function itself *is* the state.

The contrast with `return` is the heart of the lesson. `return` ends the function permanently and hands back one final value. `yield` hands back a value and suspends the function, ready to continue, and a single generator can yield thousands of values over its lifetime. Because it is a genuine iterator, it works directly in `for` loops, in `list()`, in `sum()`, and everywhere else an iterable does.

```
1 def countdown(start):
2     print("Starting")
3     while start > 0:
```

```

4         yield start
5         start -= 1
6         print("Done")
7
8     for n in countdown(3):
9         print(n)
10 # Starting
11 # 3
12 # 2
13 # 1
14 # Done

```

Watching where the prints appear reveals the pausing exactly: "Starting" prints only when the loop first asks for a value, each number arrives one at a time, and "Done" prints only when the loop asks for a value after the last `yield`, so the function runs off its end.

Why Lazy Matters: Memory

The practical payoff is memory. The list version above stores every number at once; the generator version stores exactly one at a time.

```

1 import sys
2
3 as_list = [n for n in range(1_000_000)]
4 as_gen = (n for n in range(1_000_000))
5
6 print(sys.getsizeof(as_list))      # about 8 million bytes
7 print(sys.getsizeof(as_gen))      # about 200 bytes, no matter how large the range

```

The second line, with round parentheses in place of square brackets, is a *generator expression*: the comprehension syntax from Lesson 1.10, producing values lazily instead of building a list. `sys.getsizeof` reports an object's size in bytes. The list holds a million numbers; the generator holds only the recipe for producing them. This is the difference between a program that comfortably processes a hundred-gigabyte log file and one that crashes with a `MemoryError`. A generator expression can also be passed straight into a function that consumes an iterable, with the extra parentheses optional: `sum(n * n for n in range(1000))` adds up a thousand squares without ever building the list of them.

There is one trade-off to remember: a generator can only be consumed *once*. After it is exhausted, it yields nothing more, exactly like the iterators in Lesson 3.8, so a second loop over the same generator object sees an empty sequence. When you need to loop twice, either call the generator function again to get a fresh generator, or store the results in a list.

Try It Yourself 3.9a: Write a generator function `squares(limit)` that yields the square of each number from 1 up to and including `limit`. Use it in a `for` loop, then use the same function inside `sum(...)`. Finally, loop over the *same* generator object twice and confirm the second loop prints nothing.

Infinite Sequences and Early Stopping

Because a generator only computes a value when asked, it can represent a sequence with no end at all, something a list never could. The consumer decides when to stop.

```

1 from itertools import islice
2
3 def fibonacci():
4     a, b = 0, 1
5     while True:
6         yield a
7         a, b = b, a + b
8

```

```

9 print(list(islice(fibonacci(), 10)))
10 # [0, 1, 1, 2, 3, 5, 8, 13, 21, 34]
11
12 for value in fibonacci():
13     if value > 100:
14         break
15     print(value, end=" ")

```

The `while True` would be an infinite loop in an ordinary function, but here each pass through it pauses at `yield`, so the code never runs away: it advances exactly as far as the consumer pulls. `islice` from Lesson 3.8 takes just the first ten, and the second loop simply breaks when it has seen enough. Compare this to the class from the previous lesson: the multiple assignment `a, b = b, a + b` from Lesson 1.3 does the state update in a single line, with the two variables living naturally as local variables of the paused function.

Delegating with `yield from`

When one generator needs to hand over to another, the naive way is a loop that re-yields each item. `yield from` does exactly that in a single statement.

```

1 def first_lines(paths):
2     for path in paths:
3         with open(path, encoding="utf-8") as file:
4             yield from file
5
6 def flatten(nested):
7     for item in nested:
8         if isinstance(item, list):
9             yield from flatten(item)      # recursion, Lesson 1.11
10        else:
11            yield item
12
13 print(list(flatten([1, [2, [3, 4]], 5]))) # [1, 2, 3, 4, 5]

```

`yield from` iterable passes every item of the iterable straight through to the caller, as if the outer generator had yielded them itself. In `first_lines`, it means iterating a file (which yields one line at a time, as in Lesson 1.13) can be forwarded in one word. In `flatten`, it lets a recursive generator, as in Lesson 1.11, yield the flattened contents of each nested list, an otherwise fiddly problem that becomes a few clear lines.

Generator Pipelines

The most powerful use of generators is chaining them: each stage takes an iterable and yields transformed items, and the stages are wired end to end. Because every stage is lazy, data flows through the whole pipeline *one item at a time*, and no stage ever holds the full dataset. Here is a pipeline that analyzes a log file of any size.

```

1 def read_lines(path):
2     with open(path, encoding="utf-8") as file:
3         for line in file:
4             yield line.rstrip("\n")
5
6 def only_errors(lines):
7     for line in lines:
8         if "ERROR" in line:
9             yield line
10
11 def extract_message(lines):
12     for line in lines:

```

```

13     yield line.split("|", 3)[-1].strip()
14
15 errors = extract_message(only_errors(read_lines("monitor.log")))
16
17 for message in errors:
18     print(message)

```

Trace one item through. When the `for` loop asks `errors` for a value, `extract_message` asks `only_errors` for a line, which asks `read_lines`, which reads exactly one line from the file and yields it. If that line has no "ERROR", `only_errors` silently pulls another one; when it finds a match, it passes it up, `extract_message` trims it down to the message, and the loop finally receives it. At no moment does more than one line sit in memory, so the same code that handles a ten-line log handles a ten-gigabyte one. Each stage is short, does one thing, and can be tested on its own, which is precisely the design benefit of the function-composition ideas you first met in Lesson 1.11.

Try It Yourself 3.9b: Write a pipeline of three generators for a text file: one that yields each line, one that yields only the non-empty lines, and one that yields each remaining line converted to uppercase. Chain them together and print the first five results using `islice`.

Generators That Keep State Between Yields

Because a generator is a paused function, it can hold any amount of state in ordinary local variables, and that makes it the natural tool for stream processing where each output depends on what came before. A running average is a small example: each value produced needs the total and count of everything seen so far.

```

1  def running_average(values):
2      total = 0
3      count = 0
4      for value in values:
5          total += value
6          count += 1
7          yield total / count
8
9  def new_highs(values):
10     highest = None
11     for value in values:
12         if highest is None or value > highest:
13             highest = value
14             yield value
15
16 readings = [10, 12, 9, 15, 14, 20, 8]
17 print(list(running_average(readings))) # [10.0, 11.0, 10.33..., 11.5, 12.0, 13.33...,
18                                         12.57...]
19 print(list(new_highs(readings)))      # [10, 12, 15, 20]

```

Neither function stores the whole input; each keeps only the few numbers it needs (`total` and `count`, or `highest`). That is why they work equally well on a list of seven readings or on an endless stream of sensor data. Notice also that `new_highs` yields *fewer* values than it receives: a generator is free to yield zero, one, or many values for each input, which is why the filtering and transforming stages of a pipeline are both easy to express in this form.

Cleaning Up When a Generator Is Abandoned

A generator that opens a resource, like a file, should release it even if the caller stops early. If a `for` loop is left by a `break`, or the generator is simply no longer needed, Python calls the generator's `close()` method, which raises a special `GeneratorExit` exception inside the generator at its paused `yield`. A `try/finally` or a `with` block around the `yield` then runs its cleanup.

```

1 def read_records(path):
2     print("opening file")
3     try:
4         with open(path, encoding="utf-8") as file:
5             for line in file:
6                 yield line.rstrip("\n")
7     finally:
8         print("closing file")
9
10 for number, record in enumerate(read_records("data.txt")):
11     if number == 2:
12         break                # stops early; the generator is closed and cleaned up

```

When the `break` abandons the generator, and it is garbage collected, "closing file" is printed, proving that the `finally` block ran even though the loop never reached the end. For generators that hold a resource, the safest habit is the `with` statement inside the generator, exactly as above, since the context manager of Lesson 3.11 takes care of the release. When you want the cleanup to happen at a precise moment rather than whenever Python collects the object, call `close()` yourself, or use `contextlib.closing` in a `with` block.

Choosing Between a List and a Generator

A generator is not automatically better; it is a choice with trade-offs. Use a generator when the data is large or unbounded, when you will only pass over it once, or when producing every item up front would waste effort because the consumer may stop early. Use a list when the data is small, when you need to loop over it several times, when you need its length or indexing, or when you need to print or inspect it. A frequent, sensible compromise is to write the pipeline lazily and call `list()` once, at the very end, on the small final result. The mistake to avoid is turning a generator into a list *early* in a pipeline, because that forces every stage before that point to finish completely, losing the constant-memory advantage.

Try It Yourself 3.9c: Write a generator `moving_sum(values, window)` that yields the sum of the last `window` values seen so far, using a `collections.deque(maxlen=window)` to hold them. Test it on the numbers 1 through 8 with a window of 3.

Common Mistakes to Avoid

- **Looping over the same generator twice.** A generator is exhausted after one pass; call the function again for a fresh one, or store the results in a list.
- **Using `return` where `yield` was meant.** A `return` ends the function and hands back one value; only `yield` produces a sequence of values over time.
- **Calling `list()` on a huge or infinite generator.** It forces every value into memory at once and defeats the purpose; iterate lazily or limit with `islice`.
- **Expecting a generator's body to run when the function is called.** Calling a generator function only creates the generator; nothing executes until the first `next()`.
- **Using `len()` or indexing on a generator.** A generator has no length and no `[i]`; convert to a list first if you truly need them.

Lesson 3.9 Project: Log Analyzer Pipeline

Write a program called `log_pipeline.py` that:

1. Creates a sample log file of at least 30 lines, each in the form `2026-09-29 10:15:02 | LEVEL | message`, with a mix of `INFO`, `WARNING`, and `ERROR` levels.
2. Defines four generator functions: `read_lines(path)`, `parse(lines)` that yields a dictionary with `time`, `level`, and `message` for each well-formed line, `filter_level(records, level)`, and a `take(records,`

n) stage built on `islice`.

3. Chains them into a pipeline and prints the first five `ERROR` messages, without ever loading the whole file into a list.
4. Counts how many records there are of each level in a single pass, using `collections.Counter` from Lesson 1.17 fed directly from the generator.
5. Skips malformed lines inside `parse` rather than crashing, and confirms with a comment where in the pipeline a single line lives at any moment.

This project deliberately keeps memory use constant regardless of file size, the defining advantage of generators.

Lesson Summary: Quick Reference

- A function containing `yield` is a generator function; calling it returns a lazy iterator without running the body
- `yield` pauses the function and remembers its state; `return` ends it permanently
- Generator expressions, `(x for x in items)`, produce values on demand and use almost no memory
- A generator is consumed once; `yield from` delegates to another iterable or generator
- Chaining generators builds a pipeline that processes one item at a time, so huge data fits in constant memory

Lesson 3.10: functools (partial, lru_cache)

Why This Matters

Lesson 1.11 established that functions are values, and Lesson 2.7 showed how to build decorators on that idea. The `functools` module is the standard library's toolbox built on the same foundation: ready-made helpers for functions that take other functions. Two of them, `partial` and `lru_cache`, appear constantly in real code. `partial` fixes some arguments of a function in advance, creating a specialized version of it. `lru_cache` makes a function remember its previous answers, so the same expensive computation is never performed twice. The second, in particular, can turn a program that takes minutes into one that takes milliseconds by adding a single line.

Learning Objectives

By the end of this lesson, you will be able to:

- Use `functools.partial` to create specialized versions of a function
- Use `@lru_cache` to memoize a function, and read its hit and miss statistics
- Explain what makes a function safe to cache, and why its arguments must be hashable
- Use `reduce` and `cached_property` appropriately

partial: Fixing Some Arguments in Advance

Suppose you have a general function and you keep calling it with the same first argument. `partial` builds a new function with some arguments already filled in.

```
1 from functools import partial
2
3 def power(base, exponent):
4     return base ** exponent
5
6 square = partial(power, exponent=2)
7 cube = partial(power, exponent=3)
8
9 print(square(5))           # 25    same as power(5, exponent=2)
10 print(cube(2))            # 8     same as power(2, exponent=3)
11 print(square)              # functools.partial(<function power ...>, exponent=2)
```

`partial(power, exponent=2)` returns a new callable object that remembers `power` and the fixed keyword argument. Calling `square(5)` passes 5 for the remaining parameter, `base`, and adds the stored `exponent=2`, so the call becomes `power(5, exponent=2)`. No new function was written by hand; the general one was specialized. You can fix positional arguments too, as in `partial(power, 2)`, which pins `base=2` and leaves `exponent` open, giving a “powers of two” function.

This is especially valuable wherever an API expects a function that takes a fixed number of arguments but yours needs more. Recall from Lesson 3.7 that `schedule` calls jobs with no arguments; `partial` is the neat way to bind them ahead of time:

```
1 from functools import partial
2 import schedule
3
4 def check_site(url, timeout):
5     print(f"Checking {url} (timeout {timeout}s)")
6
7 schedule.every(5).minutes.do(partial(check_site, "https://example.com", 10))
```

Compared with writing `lambda: check_site("https://example.com", 10)` from Lesson 1.11, `partial` has a subtle advantage: it evaluates the fixed arguments *immediately*, when the `partial` is created, whereas a `lambda` looks variables up later, at call time. That difference matters in a loop, as the mistakes section shows.

Try It Yourself 3.10a: Write a function `log(level, message)` that prints `[LEVEL] message`. Use `partial` to create `info` and `warn` functions that fix the `level` argument, then call each with a message.

`lru_cache`: Remembering Answers

Some functions are expensive to compute and get called repeatedly with the same inputs. *Memoization* means storing each result the first time it is computed so later calls with the same inputs return the stored answer instantly. The classic demonstration is the recursive Fibonacci function from Lesson 1.11, which is disastrously slow because it recomputes the same values over and over.

```

1 import time
2 from functools import lru_cache
3
4 def fib_slow(n):
5     if n < 2:
6         return n
7     return fib_slow(n - 1) + fib_slow(n - 2)
8
9 @lru_cache(maxsize=None)
10 def fib_fast(n):
11     if n < 2:
12         return n
13     return fib_fast(n - 1) + fib_fast(n - 2)
14
15 start = time.perf_counter()
16 print(fib_slow(32))
17 print(f"slow: {time.perf_counter() - start:.2f}s")
18
19 start = time.perf_counter()
20 print(fib_fast(32))
21 print(f"fast: {time.perf_counter() - start:.6f}s")
22
23 print(fib_fast.cache_info())
24 # CacheInfo(hits=30, misses=33, maxsize=None, cursize=33)

```

`fib_slow(32)` makes millions of calls, since `fib_slow(30)` is worked out from scratch inside both `fib_slow(31)` and `fib_slow(32)`, and so on down the tree. `@lru_cache`, applied with the decorator syntax from Lesson 2.7, wraps the function with a dictionary mapping the arguments to the result. On each call it checks the dictionary first; only on a *miss* does it run the real function and store the answer. Now `fib_fast(32)` computes each value from 0 to 32 exactly once, thirty-three misses, and every other request is a *hit* served from the dictionary. The timing turns roughly a second into a few microseconds. `cache_info()` reports the hits, the misses, and the current size, which is how you confirm that a cache is actually helping; `cache_clear()` empties it.

`maxsize` caps how many results are kept, discarding the *least recently used* entries first (the “LRU” in the name) once the limit is reached. The default is 128. Use `maxsize=None` for unlimited, only when the number of distinct inputs is known to be small. Since Python 3.9 the shorthand `@cache` (also from `functools`) means exactly `@lru_cache(maxsize=None)`.

What Is Safe to Cache

Caching is only correct under two conditions, and forgetting them causes some of the most confusing bugs there are.

```

1 from functools import lru_cache
2 import random

```

```

3
4 @lru_cache(maxsize=None)
5 def roll():
6     return random.randint(1, 6)
7
8 print(roll(), roll(), roll())      # the same number three times!
9
10 @lru_cache(maxsize=None)
11 def total(numbers):
12     return sum(numbers)
13
14 print(total((1, 2, 3)))             # 6, works with a tuple
15 print(total([1, 2, 3]))             # TypeError: unhashable type: 'list'

```

First, the function must be *pure*: its result must depend only on its arguments, with no randomness, no reading the clock, no fetching from the network, and no side effects you rely on. `roll` fails this test, so caching it freezes the first random number forever. Second, every argument must be *hashable*, because the cache uses the arguments as dictionary keys, exactly the restriction from Lesson 1.9 on what can be a key. Tuples, strings, numbers, and frozen dataclasses (Lesson 2.9) qualify; lists, dictionaries, and sets do not, and raise the `TypeError` shown. In practice, cache functions that compute something from plain values, and convert a list to a tuple before passing it in.

Try It Yourself 3.10b: Write a recursive function `count_paths(rows, cols)` that returns the number of ways to move from the top-left to the bottom-right of a grid taking only steps right or down (return 1 if either dimension is 1, otherwise the sum of `count_paths(rows - 1, cols)` and `count_paths(rows, cols - 1)`). Time `count_paths(16, 16)` with and without `@lru_cache`, and print `cache_info()`.

reduce and cached_property

`reduce` repeatedly applies a two-argument function to a running result and the next item, collapsing an iterable into a single value. It is the general idea behind `sum` and `max`.

```

1 from functools import reduce
2
3 print(reduce(lambda total, n: total + n, [1, 2, 3, 4], 0))    # 10
4 print(reduce(lambda a, b: a * b, [1, 2, 3, 4, 5]))           # 120

```

The optional third argument is the starting value, which also serves as the result for an empty iterable. For sums, products, and extremes, the built-in functions from Lesson 1.8 are clearer; `reduce` earns its place for custom combining rules, like merging a list of dictionaries into one.

`cached_property` is the class-based cousin of caching. It looks like the `@property` from Lesson 2.5, but it computes the value only once per object, on first access, and stores it on the instance.

```

1 from functools import cached_property
2
3 class Report:
4     def __init__(self, path):
5         self.path = path
6
7     @cached_property
8     def lines(self):
9         print("Reading file...")
10        with open(self.path, encoding="utf-8") as file:
11            return file.readlines()
12
13 report = Report("data.txt")

```

```

14 print(len(report.lines))      # prints "Reading file..." then the count
15 print(len(report.lines))      # no re-read: the stored list is reused

```

It is the right tool for an attribute that is expensive to compute, does not change afterward, and might never be needed at all, so it should be computed lazily.

The Trap of Caching Methods

Placing `@lru_cache` on an *instance method* has a consequence that catches many people out. The cache is keyed on all the arguments, including `self`, so every instance that ever called the method is kept alive by the cache, and cannot be garbage collected, until the entry is evicted. On a class whose objects are created often, this quietly leaks memory. It also requires the instance to be hashable, which a plain class is by default but a dataclass with `eq=True` and no `frozen` is not.

```

1  from functools import cache, cached_property
2
3  class Circle:
4      def __init__(self, radius):
5          self.radius = radius
6
7      @cached_property
8      def area(self):
9          print("computing area")
10         return 3.14159 * self.radius ** 2
11
12  @cache
13  def area_of(radius):
14      print("computing area")
15      return 3.14159 * radius ** 2
16
17  c = Circle(2)
18  print(c.area, c.area)      # computes once
19  print(area_of(2), area_of(2))

```

The two remedies show the general approach. For a value derived from one object's own data, use `cached_property`, which stores its result *on that object*, so it lives and dies with it. For a calculation that depends only on plain values, move it to a module-level function, or a static method (Lesson 2.3), and cache *that*: the cache then holds numbers rather than instances. The underlying rule is that a cache is a piece of long-lived state, so always ask what it keeps alive and how large it can become.

Caching with an Expiry Time

`lru_cache` never expires an entry on its own, which is wrong for data that goes stale: a currency rate cached forever is a currency rate that is wrong tomorrow. A small decorator, written with the techniques of Lesson 2.7, adds a time limit.

```

1  import time
2  from functools import wraps
3
4  def cache_for(seconds):
5      def decorator(func):
6          stored = {}
7          @wraps(func)
8          def wrapper(*args):
9              now = time.monotonic()
10             if args in stored:
11                 value, saved_at = stored[args]
12                 if now - saved_at < seconds:
13                     return value

```

```

14         value = func(*args)
15         stored[args] = (value, now)
16         return value
17     return wrapper
18 return decorator
19
20 @cache_for(seconds=30)
21 def exchange_rate(currency):
22     print(f"fetching {currency}...")
23     return 1.09          # imagine a network call here
24
25 print(exchange_rate("EUR"))    # fetches
26 print(exchange_rate("EUR"))    # served from the cache for the next 30 seconds

```

This is a decorator with its own argument, so it has the three layers you saw with repeat in Lesson 2.7. The dictionary stored maps each argument tuple to a pair: the value, and the moment it was saved. On each call, an entry is used only if it is younger than seconds; otherwise the function runs again and the entry is replaced. `time.monotonic()` is a clock that never goes backward, which makes it the right choice for measuring an interval, unlike the wall clock, which the system may adjust. Wrapping a real API call from Lesson 3.4 this way protects both your speed and the service's rate limit.

total_ordering: Fewer Comparison Methods to Write

Lesson 2.6 showed that sorting needs only `__lt__`, but that comparing objects with `<=`, `>`, and `>=` needs all the other methods too. `functools.total_ordering` fills in the rest: you define `__eq__` and *one* ordering method, and the decorator derives the others.

```

1 from functools import total_ordering
2
3 @total_ordering
4 class Version:
5     def __init__(self, major, minor):
6         self.major = major
7         self.minor = minor
8
9     def __eq__(self, other):
10        return (self.major, self.minor) == (other.major, other.minor)
11
12    def __lt__(self, other):
13        return (self.major, self.minor) < (other.major, other.minor)
14
15 print(Version(2, 1) >= Version(2, 0))    # True, though only __eq__ and __lt__ exist

```

Comparing tuples, as in Lesson 1.8, does the real work: two versions compare by major number first, then minor. For a class that is mostly plain data, the `order=True` option of a dataclass (Lesson 2.9) does the same thing with no code at all; `total_ordering` is for the classes where the comparison rule needs to be written by hand.

Try It Yourself 3.10c: Apply `@cache_for(seconds=2)` to a function that returns `time.time()`. Call it twice immediately and confirm both results are identical, then sleep three seconds, call again, and confirm a new value is produced.

Common Mistakes to Avoid

- **Caching a function that is not pure.** Anything depending on randomness, the current time, or external data returns stale results forever; cache only deterministic functions.
- **Passing a list, dictionary, or set to a cached function.** Arguments must be hashable; convert to a tuple or

frozen structure first.

- **Using `maxsize=None` on an unbounded set of inputs.** The cache grows without limit and can use all available memory; set a sensible `maxsize`.
- **Building lambda functions in a loop that all use the loop variable.** A lambda looks the variable up when it is called, so every one sees the last value; `partial` fixes the value at creation.
- **Reaching for `reduce` when a built-in exists.** `sum`, `max`, `min`, and `"".join` are clearer than an equivalent `reduce`.

Lesson 3.10 Project: Caching Toolkit

Write a program called `cache_toolkit.py` that:

1. Implements the recursive Fibonacci function twice, once plain and once with `@lru_cache`, times both for a suitably large input with `time.perf_counter()`, and prints the speedup.
2. Prints `cache_info()` before and after several repeated calls, and demonstrates `cache_clear()` resetting the statistics.
3. Defines a function `apply_discount(price, percent)` and uses `partial` to create `ten_percent_off` and `half_price`, applying each to a list of prices with `map` from Lesson 1.11.
4. Demonstrates the hashability restriction by catching the `TypeError` raised when a list is passed to a cached function, then fixing the call by converting the list to a tuple.
5. Defines a class with a `cached_property` that simulates an expensive computation with a print statement, and shows it running only once across several accesses.

This project deliberately demonstrates the huge gain from caching and the two rules that keep it correct.

Lesson Summary: Quick Reference

- `partial(func, ...)` pre-fills some arguments of a function and returns a new, specialized callable
- `@lru_cache` memoizes results keyed on the arguments; `cache_info()` shows hits and misses, `maxsize` bounds memory
- Only cache pure functions with hashable arguments; `@cache` is shorthand for an unbounded cache
- `reduce` folds an iterable into one value; prefer built-ins like `sum` when they fit
- `@cached_property` computes an expensive attribute once per instance, on first access

Lesson 3.11: Context Managers

Why This Matters

You have been writing with `open(...)` as `file`: since Lesson 1.13, and each time it quietly did something valuable: it guaranteed that the file was closed, even if the code inside the block crashed halfway through. That guarantee is not special to files. Any time a program acquires something that must later be released, a file, a network connection, a lock, a temporary directory, a timer that must stop, the same problem appears, and forgetting the cleanup is a bug that may not show up for hours. A *context manager* packages the setup and the cleanup together so the cleanup can never be forgotten. This lesson shows you how the `with` statement works, and how to build your own.

Learning Objectives

By the end of this lesson, you will be able to:

- Explain what the `with` statement guarantees, and why it is safer than manual cleanup
- Write a context manager as a class with `__enter__` and `__exit__`
- Write a context manager as a generator with `@contextmanager`
- Use `contextlib.suppress` and `tempfile` for common everyday cases

The Problem: Cleanup That Gets Skipped

Consider a version of file handling with no `with` statement, written the way Lesson 1.13 warned against:

```
1 file = open("data.txt", "w", encoding="utf-8")
2 file.write("first line\n")
3 result = 10 / 0                      # raises ZeroDivisionError
4 file.write("second line\n")
5 file.close()                        # never reached!
```

The exception jumps straight out of the program, so `file.close()` never runs, and the file is left open, its buffered data possibly never saved. The manual fix is a `try/finally` from Lesson 1.14, putting `close()` in the `finally` block so it always runs. That works, but it is wordy, and it depends on every programmer remembering to write it every single time. The `with` statement is exactly that pattern, built into the language:

```
1 with open("data.txt", "w", encoding="utf-8") as file:
2     file.write("first line\n")
3     result = 10 / 0
4     file.write("second line\n")
5 # the file is closed here, even though ZeroDivisionError was raised
```

Whether the block finishes normally, raises an error, hits a `return`, or hits a `break`, the file is closed the moment execution leaves the indented block. The error still propagates afterward, so nothing is hidden; only the guaranteed cleanup was added.

How with Works: `__enter__` and `__exit__`

The `with` statement works with any object that defines two magic methods, in the same family as the ones from Lesson 2.6. `__enter__` runs at the start of the block, and whatever it returns is bound to the name after `as`. `__exit__` runs when the block ends, no matter how.

```
1 import time
2
3 class Timer:
```

```

4     def __enter__(self):
5         self.start = time.perf_counter()
6         print("Timer started")
7         return self
8
9     def __exit__(self, exc_type, exc_value, traceback):
10        self.elapsed = time.perf_counter() - self.start
11        print(f"Timer stopped: {self.elapsed:.3f}s")
12        return False
13
14 with Timer() as timer:
15     total = sum(range(5_000_000))
16
17 print(timer.elapsed)

```

Trace the order. `Timer()` creates the object, and `with` calls its `__enter__`, which records the start time and returns `self`, so `timer` refers to the object. The block body runs. When it ends, `with` calls `__exit__`, which computes the elapsed time. The object stays available afterward, so `timer.elapsed` can be read after the block, useful for reporting.

The three parameters of `__exit__` describe any exception raised inside the block: its type, its value, and its traceback. If the block finished cleanly, all three are `None`. The *return value* of `__exit__` decides what happens to the exception. Returning `False` (or nothing, which is `None`) lets the exception continue propagating, the correct default, since you should not hide errors you do not understand. Returning `True` *suppresses* the exception, as though it had been caught, and execution continues after the `with` block. Suppressing is occasionally right, but should always be deliberate.

Try It Yourself 3.11a: Write a context manager class `Announce` that takes a name, prints "Entering <name>" in `__enter__` and "Leaving <name>" in `__exit__`. Use it around a block that raises a `ValueError`, and confirm "Leaving" still prints before the error appears.

A Realistic Example: Changing Directory Temporarily

A pattern that scripts need constantly is doing something in another folder and then returning to the original one. Forgetting to return is a nasty bug, since every later relative path silently points somewhere else.

```

1 import os
2 from pathlib import Path
3
4 class WorkingDirectory:
5     def __init__(self, path):
6         self.path = Path(path)
7
8     def __enter__(self):
9         self.previous = Path.cwd()
10        os.chdir(self.path)
11        return self.path
12
13    def __exit__(self, exc_type, exc_value, traceback):
14        os.chdir(self.previous)
15        return False
16
17 print(Path.cwd())
18 with WorkingDirectory("/tmp"):
19     print(Path.cwd())           # /tmp
20 print(Path.cwd())              # back where we started, even after an error

```

`__enter__` remembers where you were with `Path.cwd()` and moves; `__exit__` always moves back. Because `__exit__` runs even when the body fails, the program can never be left stranded in the wrong directory. This “acquire, use, always

release” shape is the reason context managers exist.

The Shortcut: `@contextmanager`

Writing a class with two magic methods is a lot of structure for something so small. The `contextlib` module lets you write the same thing as a single generator function, using `yield` from Lesson 3.9 to mark the point where the `with` body runs.

```

1 import os
2 import time
3 from contextlib import contextmanager
4 from pathlib import Path
5
6 @contextmanager
7 def working_directory(path):
8     previous = Path.cwd()
9     os.chdir(path)
10    try:
11        yield Path(path)
12    finally:
13        os.chdir(previous)
14
15 @contextmanager
16 def timed(label):
17     start = time.perf_counter()
18     try:
19         yield
20     finally:
21         print(f"{label}: {time.perf_counter() - start:.3f}s")
22
23 with timed("summing"):
24     sum(range(5_000_000))

```

Everything before the `yield` is the setup, the equivalent of `__enter__`. The value after `yield` is what `as` receives (`timed` yields nothing, so it is used without `as`). The block body runs *at* the `yield`, and everything after it is the cleanup, the equivalent of `__exit__`. The `try/finally` around the `yield` is essential: if the body raises, the exception is thrown into the generator at the `yield` point, and without `finally` the cleanup lines would be skipped. With it, the cleanup always runs, and the exception carries on afterward. Compare the two versions of `working_directory`: the generator form says the same thing in a third of the space, which is why `@contextmanager` is what most real code uses.

Try It Yourself 3.11b: Write a `@contextmanager` function `suppress_output()` that temporarily replaces `sys.stdout` with `io.StringIO()`, yields the buffer, and restores the original in a `finally`. Print something inside the block, then print the captured text afterward using the yielded buffer.

Ready-Made Helpers: `suppress` and `tempfile`

The standard library provides several context managers so common that you should know them by name. `contextlib.suppress` ignores the listed exceptions, replacing a `try/except/pass` block from Lesson 1.14 with one clear line.

```

1 from contextlib import suppress
2 from pathlib import Path
3
4 with suppress(FileNotFoundError):
5     Path("old_cache.txt").unlink()           # fine if it is already gone
6
7 print("continues here either way")

```

The intent, “delete this if it exists, and I do not mind if it does not”, reads straight off the code. Use `suppress` only for exceptions you genuinely expect and genuinely do not care about, and always name specific ones, never `Exception`.

The `tempfile` module creates files and folders that clean themselves up, ideal for scratch work and for tests:

```

1 import tempfile
2 from pathlib import Path
3
4 with tempfile.TemporaryDirectory() as folder:
5     scratch = Path(folder) / "scratch.txt"
6     scratch.write_text("temporary data", encoding="utf-8")
7     print(scratch.exists())           # True
8
9 print(scratch.exists())               # False: the whole folder was deleted

```

The temporary folder exists only inside the block; when the block ends, it and everything in it are removed automatically. A file opened with `tempfile.NamedTemporaryFile` behaves the same way. In Lesson 3.4, the `with requests.Session() as session:` line used exactly this mechanism to guarantee that the network connections were closed, and the same is true of `subprocess.Popen`, threading locks in Lesson 3.12, and database connections you will meet in your own projects. The pattern is everywhere because the problem is everywhere.

Several Context Managers at Once, and `ExitStack`

A single `with` statement can manage several resources by separating them with commas, and they are released in the reverse order of acquisition, like closing a stack of doors behind you.

```

1 from pathlib import Path
2
3 with open("input.txt", encoding="utf-8") as source, \
4     open("output.txt", "w", encoding="utf-8") as target:
5     for line in source:
6         target.write(line.upper())
7 # target is closed first, then source

```

Both files are guaranteed to be closed even if the loop raises an error, and if opening the *second* file fails, the first is still closed properly. This handles a fixed, known number of resources. But what if the number is not known until the program runs, for example opening one output file for every site in a configuration? `contextlib.ExitStack` is a context manager that manages a *dynamic* collection of other context managers.

```

1 from contextlib import ExitStack
2
3 sites = ["alpha", "beta", "gamma"]
4
5 with ExitStack() as stack:
6     files = {
7         name: stack.enter_context(open(f"{name}.log", "w", encoding="utf-8"))
8         for name in sites
9     }
10    for name, file in files.items():
11        file.write(f"log for {name}\n")
12 # all three files are closed here, in reverse order

```

`stack.enter_context(manager)` enters a context manager, records that it must be exited later, and returns what its `__enter__` returned, so the dictionary comprehension from Lesson 1.10 opens all the files and keeps their handles. When the `with ExitStack()` block ends, every registered manager is exited, in reverse order, and if one exit raises an error the

rest are still attempted. `stack.callback(function, *args)` registers an ordinary function to be called at exit, which is convenient for cleanup that has no context manager of its own.

Building a Context Manager That Handles Errors

So far the cleanup has been unconditional. The exception information passed to `__exit__` also lets a context manager *react* to failure, for example rolling back a partly finished change. The pattern is common enough that database libraries build their transactions on it.

```

1  from contextlib import contextmanager
2  import shutil
3  from pathlib import Path
4
5  @contextmanager
6  def safe_edit(path):
7      path = Path(path)
8      backup = path.with_suffix(path.suffix + ".bak")
9      shutil.copy2(path, backup)
10     try:
11         yield path
12     except Exception:
13         shutil.copy2(backup, path)          # roll back to the saved copy
14         print(f"Edit failed; {path.name} restored from backup")
15         raise
16     else:
17         print(f"Edit of {path.name} succeeded")
18     finally:
19         backup.unlink(missing_ok=True)
20
21 with safe_edit("settings.txt") as file_path:
22     file_path.write_text("new value\n", encoding="utf-8")
23     # if anything raises here, settings.txt is restored automatically

```

Inside a generator-based context manager, an exception raised in the `with` body appears at the `yield`, so an ordinary `try/except/else/finally`, the full structure from Lesson 1.14, expresses every case. If the body fails, the `except` block restores the backup and then `raise` sends the error onward so the problem is not hidden. If the body succeeds, the `else` block runs. In both cases the `finally` block deletes the backup. This “do it, and undo it if anything goes wrong” guarantee is the strongest use of a context manager.

Try It Yourself 3.11c: Use `ExitStack` to open every `.txt` file in a folder for reading, and print the number of lines in each. Then write a `safe_edit`-style context manager of your own for a JSON file, and confirm that raising an exception inside the block restores the original contents.

Common Mistakes to Avoid

- **Opening resources without `with`.** A skipped `close()` after an exception leaks the resource; use `with`, or at minimum `try/finally`.
- **Forgetting `try/finally` around the `yield` in `@contextmanager`.** Without it, the cleanup lines are skipped whenever the body raises an error.
- **Returning `True` from `__exit__` by accident.** That suppresses every exception in the block and hides real bugs; return `False` unless suppression is deliberate.
- **Using `suppress(Exception)`.** Silencing all errors hides genuine problems; name only the specific exception you expect.
- **Using the resource after the `with` block.** A file, session, or temporary folder is already closed or deleted by

then; finish using it inside the block.

Lesson 3.11 Project: Context Manager Toolkit

Write a program called `context_toolkit.py` that:

1. Defines a `Timer` context manager as a class with `__enter__` and `__exit__` that records elapsed and exposes it after the block.
2. Defines a `working_directory(path)` context manager using `@contextmanager` and `try/finally`, and demonstrates it returning to the original folder even when the block raises an exception.
3. Defines a `log_block(name)` context manager that logs entry and exit with the `logging` module from Lesson 3.6, and logs a failure message with `logger.exception` if the block raises, before re-raising.
4. Uses `tempfile.TemporaryDirectory` to create a scratch folder, writes three files into it, lists them, and confirms the folder no longer exists afterward.
5. Uses `contextlib.suppress(FileNotFoundError)` to delete a file that may not exist.

This project deliberately builds the small `guarantee-of-cleanup` tools that appear throughout the capstone.

Lesson Summary: Quick Reference

- `with` guarantees cleanup runs whether the block finishes, raises, returns, or breaks
- A class-based context manager defines `__enter__` (setup, value for `as`) and `__exit__` (cleanup, may suppress errors)
- Returning `False` or `None` from `__exit__` lets exceptions continue; returning `True` suppresses them
- `@contextmanager` turns a generator with one `yield` into a context manager; wrap the `yield` in `try/finally`
- `contextlib.suppress` and `tempfile.TemporaryDirectory` cover two of the most common everyday cases

Lesson 3.12: threading & multiprocessing

Why This Matters

So far, every program in this course has done one thing at a time: finish this step, then start the next. That is fine for most scripts, but consider a monitor checking a hundred websites. If each check spends half a second waiting for a server to answer, doing them one after another takes fifty seconds, nearly all of it spent *waiting*. Doing many of them at the same time would take about half a second. Concurrency is the set of techniques for making a program do several things at once, and Python offers more than one, because there are two very different kinds of slow: work that is slow because it waits, and work that is slow because it computes. Choosing correctly between them is the entire skill.

Learning Objectives

By the end of this lesson, you will be able to:

- Distinguish I/O-bound work from CPU-bound work, and explain what the GIL means for each
- Run functions concurrently with `threading.Thread` and, more conveniently, `ThreadPoolExecutor`
- Explain a race condition and prevent one with a `Lock`
- Use `multiprocessing` and `ProcessPoolExecutor` to speed up CPU-bound work

I/O-Bound versus CPU-Bound

A task is *I/O-bound* when most of its time is spent waiting for something outside the processor: a network response, a disk read, a subprocess. While it waits, the processor is idle. A task is *CPU-bound* when most of its time is spent actually computing, like resizing images, compressing data, or a heavy calculation, and the processor is busy the whole time. The distinction matters because Python's standard interpreter has a mechanism called the *Global Interpreter Lock*, or GIL, which allows only one thread to execute Python bytecode at any instant. The consequence is precise and worth memorizing. Threads *do* speed up I/O-bound work, because a waiting thread releases the lock and lets another thread run. Threads *do not* speed up CPU-bound Python code, because only one of them can compute at a time. For CPU-bound work, you need separate *processes*, each with its own interpreter and its own GIL, which is what `multiprocessing` provides.

Threads: Doing Waiting Work Concurrently

The `threading` module runs a function in a separate thread of execution within your program.

```
1 import threading
2 import time
3
4 def check(name, delay):
5     print(f"{name}: start")
6     time.sleep(delay)           # stands in for waiting on a network reply
7     print(f"{name}: done after {delay}s")
8
9 start = time.perf_counter()
10
11 threads = [
12     threading.Thread(target=check, args=("site-a", 1)),
13     threading.Thread(target=check, args=("site-b", 1)),
14     threading.Thread(target=check, args=("site-c", 1)),
15 ]
16 for thread in threads:
17     thread.start()
18 for thread in threads:
19     thread.join()
20
21 print(f"Total: {time.perf_counter() - start:.1f}s")    # about 1.0s, not 3.0s
```

`Thread(target=function, args=...)` creates a thread but does not run it. `start()` launches it, and the main program immediately carries on. `join()` makes the main program wait until that thread finishes, and without the `join` loop the final print would run before the threads were done. The three one-second waits overlap, so the total is about one second instead of three. The output lines from different threads may appear interleaved in a different order from run to run, since the operating system decides which thread runs when, an important reminder that concurrent programs are not deterministic the way sequential ones are.

ThreadPoolExecutor: The Practical Way

Creating and joining threads by hand gets tedious quickly. The `concurrent.futures` module provides a higher-level tool: a *pool* of worker threads that you hand tasks to.

```

1 from concurrent.futures import ThreadPoolExecutor, as_completed
2 import time
3
4 def fetch(url):
5     time.sleep(1)                # pretend this is a network call
6     return f"{url}: OK"
7
8 urls = [f"site-{i}.example" for i in range(8)]
9
10 with ThreadPoolExecutor(max_workers=4) as pool:
11     results = list(pool.map(fetch, urls))
12     print(results)
13
14 with ThreadPoolExecutor(max_workers=4) as pool:
15     futures = {pool.submit(fetch, url): url for url in urls}
16     for future in as_completed(futures):
17         print(futures[future], "->", future.result())

```

`max_workers=4` caps how many run at once, so eight one-second tasks finish in about two seconds. `pool.map(function, items)` works like the built-in `map` from Lesson 1.11, but runs the calls concurrently and returns results in the *original order*. `pool.submit(function, arg)` schedules one call and returns a `Future`, a placeholder for a result that may not exist yet; `future.result()` waits for it and returns the value. `as_completed` yields the futures in the order they *finish*, which is what you want when you would like to process results as soon as they arrive. The dictionary maps each future back to its URL. The `with` block, from Lesson 3.11, waits for all work to complete and shuts the pool down cleanly.

Try It Yourself 3.12a: Write a function `slow_square(n)` that sleeps for 1 second and returns `n * n`. Time computing the squares of `range(6)` sequentially, then with `ThreadPoolExecutor(max_workers=6)`, and print both durations.

Race Conditions and Locks

Threads share the same memory, which is convenient and dangerous. When two threads read and modify the same variable, the result can depend on exactly how their steps interleave, a bug called a *race condition*.

```

1 import threading
2
3 counter = 0
4
5 def add_many():
6     global counter
7     for _ in range(100_000):
8         counter += 1                # read, add, write: three steps, not one

```

```

9  threads = [threading.Thread(target=add_many) for _ in range(4)]
10 for thread in threads:
11     thread.start()
12 for thread in threads:
13     thread.join()
14
15 print(counter)                                # expected 400000, but often less

```

The statement `counter += 1` looks like one action but is really three: read the current value, add one, write the result back. If a thread is paused between the read and the write, another thread can update the counter in the meantime, and the first then overwrites that update with a stale value; increments are silently lost. Such bugs are especially nasty because they depend on timing and may not appear in most runs. The fix is a `Lock`, which lets only one thread at a time into a protected section.

```

1  import threading
2
3  counter = 0
4  lock = threading.Lock()
5
6  def add_many():
7      global counter
8      for _ in range(100_000):
9          with lock:                # only one thread at a time may run this block
10             counter += 1

```

The `with lock:` block, using the context manager mechanism from Lesson 3.11, acquires the lock on entry and releases it on exit, even if an error occurs, so the read-add-write sequence can no longer be interrupted. The general rule: whenever more than one thread modifies shared data, that access must be protected. Better still, avoid sharing altogether where you can, for example by having each task return its result rather than updating a global, as `ThreadPoolExecutor` naturally encourages.

Queue: Passing Work Safely Between Threads

When threads must cooperate, the standard tool is `queue`. `Queue`, a collection that is already safe to use from many threads at once. One common design has producer threads putting jobs in and worker threads taking them out.

```

1  import queue
2  import threading
3
4  jobs = queue.Queue()
5
6  def worker():
7      while True:
8          item = jobs.get()
9          if item is None:        # sentinel value: no more work
10             break
11             print(f"processed {item}")
12             jobs.task_done()
13
14  thread = threading.Thread(target=worker)
15  thread.start()
16
17  for item in range(5):
18      jobs.put(item)
19  jobs.put(None)                # tell the worker to stop

```

```
20 thread.join()
```

put adds an item and get removes one, waiting if the queue is empty, and both handle their own locking. A special value such as None, called a *sentinel*, is the usual way to tell a worker that the work is finished, since it cannot be mistaken for a real job.

multiprocessing: Real Parallelism for CPU Work

Because of the GIL, a CPU-bound function gains nothing from threads. Processes sidestep the problem: each has its own interpreter, so several truly run at the same time on different processor cores. ProcessPoolExecutor has the same interface as the thread version, which makes switching a one-word change.

```
1 from concurrent.futures import ProcessPoolExecutor
2 import time
3
4 def heavy(n):
5     return sum(i * i for i in range(n))
6
7 if __name__ == "__main__":
8     numbers = [3_000_000] * 8
9
10    start = time.perf_counter()
11    sequential = [heavy(n) for n in numbers]
12    print(f"sequential: {time.perf_counter() - start:.2f}s")
13
14    start = time.perf_counter()
15    with ProcessPoolExecutor() as pool:
16        parallel = list(pool.map(heavy, numbers))
17    print(f"processes: {time.perf_counter() - start:.2f}s")
```

On a machine with several cores, the process version finishes in a fraction of the sequential time; a thread pool here would be no faster than the plain loop. The `if __name__ == "__main__":` guard is *mandatory* for multiprocessing on Windows and macOS, because each new process re-imports your script, and without the guard every child would try to start more children. Processes come with different costs than threads: starting one is slower, and they do *not* share memory. Arguments and results are copied between processes by converting them to bytes (a process called *pickling*), so they must be picklable, and a change one process makes to a global variable is invisible to the others.

Try It Yourself 3.12b: Run the CPU-bound comparison above with `ThreadPoolExecutor` in place of `ProcessPoolExecutor`, and compare the three timings: sequential, threads, and processes. Explain in a comment why the thread version is not faster.

Stopping Threads Cleanly with an Event

A worker thread that loops forever, such as a background monitor, needs a way to be told to stop. Killing a thread from outside is not supported in Python, and would be dangerous anyway, since it could stop in the middle of writing a file. The safe design is *cooperative*: the thread checks a flag regularly and ends itself. `threading.Event` is a flag built for exactly this, and safe to share between threads.

```
1 import threading
2 import time
3
4 stop = threading.Event()
5
6 def monitor():
7     while not stop.is_set():
8         print("checking...")
```

```

9         stop.wait(timeout=1.0)           # sleeps up to 1 second, but wakes early if stopped
10        print("monitor stopping cleanly")
11
12    thread = threading.Thread(target=monitor)
13    thread.start()
14
15    time.sleep(3.5)
16    stop.set()                             # ask the thread to finish
17    thread.join()
18    print("main done")

```

The loop condition `not stop.is_set()` keeps it running until someone calls `stop.set()`. The line `stop.wait(timeout=1.0)` plays two roles at once: it is the pause between checks, replacing `time.sleep`, and it returns immediately if the event is set during the wait, so the thread reacts to the stop request within moments instead of finishing a whole one-second sleep. After `set()`, the main thread calls `join()` to wait for the worker to finish its current pass and exit. This is the same pattern that a long-running scheduler from Lesson 3.7 would use to shut down gracefully when you press Ctrl+C, catching `KeyboardInterrupt` in the main thread and setting the event. A thread started with `daemon=True` is killed abruptly when the main program exits, which suits throwaway helpers but not anything that must clean up after itself.

Sharing Data Between Processes

Processes do not share memory, so the tools of the thread sections do not apply. For giving work to processes and collecting results, the pool tools already shown are the simplest answer, and they deliver each result back to you by copying it. A frequent mistake is trying to update a global variable from inside a worker process, which changes only that process's private copy.

```

1  from concurrent.futures import ProcessPoolExecutor
2
3  total = 0
4
5  def add_to_total(n):
6      global total
7      total += n           # changes THIS process's copy only
8      return n
9
10 def chunked(items, size):
11     for start in range(0, len(items), size):
12         yield items[start:start + size]
13
14 def sum_chunk(chunk):
15     return sum(x * x for x in chunk)
16
17 if __name__ == "__main__":
18     numbers = list(range(1_000_000))
19
20     with ProcessPoolExecutor() as pool:
21         partial_sums = list(pool.map(sum_chunk, chunked(numbers, 100_000)))
22
23     print(sum(partial_sums))           # combine the results in the main process

```

The correct design is the one the second half of the code uses: split the data into chunks, hand each chunk to a worker that returns its *own* result, and combine the partial results in the main process. Splitting the million numbers into ten chunks means the workers receive ten reasonably large tasks rather than a million tiny ones, which matters because every task sent to another process carries the pickling and communication cost mentioned earlier. As a rule of thumb, a task should

be large enough that the useful work clearly outweighs that overhead; if a function finishes in microseconds, sending it to another process will be slower than simply calling it. This is the map-and-combine shape, and it is the same one used for processing very large datasets across entire clusters of machines.

How Many Workers?

More workers are not always faster. For a process pool doing CPU-bound work, the useful count is about the number of processor cores, which `os.cpu_count()` reports, and it is what `ProcessPoolExecutor` uses by default. For a thread pool doing I/O-bound work, the limit is set by the *other side*: a website, an API with a rate limit, or a database can only serve so many simultaneous requests before it slows down or rejects you. Starting from a modest number, such as 8 to 20 threads for network checks, and measuring is far better than guessing, in the spirit of Lesson 3.16. If doubling the workers does not shorten the total time, you have reached the limit and can stop.

Try It Yourself 3.12c: Write a background thread that appends the current time to a list once per second until a `threading.Event` is set. Let it run for five seconds from the main thread, stop it with the event, join it, and print how many entries were collected.

Common Mistakes to Avoid

- **Using threads to speed up CPU-bound code.** The GIL prevents true parallel computation in threads; use processes for heavy calculation and threads for waiting.
- **Modifying shared variables from several threads without a lock.** Operations like `counter += 1` are not atomic, and updates are lost unpredictably.
- **Forgetting `join()` or leaving the pool's with block early.** The main program may continue, or exit, before the work is finished.
- **Omitting the `__name__` guard when using multiprocessing.** Child processes re-import the script and can spawn endlessly or crash.
- **Passing unpicklable objects to a process pool.** Lambdas, open files, and locks cannot be sent to another process; use plain top-level functions and simple data.

Lesson 3.12 Project: Parallel Site Checker

Write a program called `parallel_checker.py` that:

1. Defines `check_site(url)` that uses `requests` from Lesson 3.4 with a timeout, returning a dictionary with the URL, the status code (or `None`), and the elapsed time, and never raising an exception.
2. Checks a list of at least eight URLs sequentially, timing the whole run with `time.perf_counter()`.
3. Checks the same list with `ThreadPoolExecutor`, printing each result as soon as it completes using `as_completed`, and timing the whole run.
4. Prints both totals and the speedup, and includes a comment explaining why threads suit this task.
5. Adds a separate CPU-bound function, compares sequential, thread, and process versions on it, inside a proper `__name__` guard, and records the three timings.

This project deliberately measures both kinds of workload, so you see with your own numbers when threads help and when only processes do.

Lesson Summary: Quick Reference

- I/O-bound tasks wait; CPU-bound tasks compute; the GIL lets threads help only the first kind
- `ThreadPoolExecutor` runs waiting work concurrently: `map` keeps order, `submit` plus `as_completed` gives results as they finish
- Shared data modified by several threads needs a `Lock`, used with `with lock::`; `queue.Queue` is thread-safe

- `ProcessPoolExecutor` gives real parallelism for CPU work, at the cost of pickling and no shared memory
- Always guard multiprocessing code with `if __name__ == "__main__":`

Lesson 3.13: `async/await` (`asyncio`)

Why This Matters

Threads (Lesson 3.12) are one way to wait on many things at once, but they carry real costs: each thread uses memory, the operating system decides when to switch between them, and shared data invites race conditions. For programs that juggle hundreds or thousands of slow network operations, Python offers a lighter model called *asynchronous programming*, built into the standard library as `asyncio`. A single thread runs many tasks by switching between them only at points *you* mark, which makes the switching predictable and the memory cost tiny. It is the foundation of modern Python web servers, bots, and high-volume network tools.

Learning Objectives

By the end of this lesson, you will be able to:

- Explain the event loop model and how `async def` and `await` fit into it
- Write coroutines and run them concurrently with `asyncio.gather` and `create_task`
- Explain why `time.sleep` freezes an async program and `asyncio.sleep` does not
- Limit concurrency with a Semaphore, add timeouts, and run blocking code with `asyncio.to_thread`

The Model: One Thread, Cooperative Switching

Imagine a chef cooking several dishes. A thread-based kitchen hires several chefs who work simultaneously. An async kitchen has one chef who is very good at not standing idle: while the pasta water heats, she chops vegetables; while the sauce simmers, she plates the salad. The chef only switches to another dish when the current one is *waiting*. In `asyncio`, that chef is the *event loop*. Your job is to mark, with the word `await`, each place where a task is waiting and the loop is free to run something else.

Coroutines: `async def` and `await`

A function defined with `async def` is a *coroutine function*. Calling it does not run it; like the generator functions of Lesson 3.9, it returns an object, a coroutine, that does nothing until something drives it. The `await` keyword, usable only inside an `async def`, pauses the current coroutine until the awaited operation finishes, and lets the event loop run other work meanwhile. `asyncio.run` starts the loop and runs a top-level coroutine to completion.

```
1 import asyncio
2
3 async def greet(name, delay):
4     print(f"{name}: waiting")
5     await asyncio.sleep(delay)
6     print(f"{name}: done")
7     return f"hello {name}"
8
9 async def main():
10     result = await greet("Ada", 1)
11     print(result)
12
13 asyncio.run(main())
```

Nothing here is concurrent yet. `await greet("Ada", 1)` runs the coroutine and waits for its result, just like an ordinary call would, and takes one second. The value of `async` appears when several coroutines are in flight at once.

Running Coroutines Concurrently

`asyncio.gather` takes several coroutines, starts them all together, and returns their results as a list, in the order you passed them.

```

1 import asyncio
2 import time
3
4 async def fetch(name, delay):
5     await asyncio.sleep(delay)           # stands in for a network call
6     return f"{name} finished in {delay}s"
7
8 async def main():
9     start = time.perf_counter()
10
11     results = await asyncio.gather(
12         fetch("a", 1), fetch("b", 2), fetch("c", 1.5),
13     )
14     print(results)
15     print(f"Total: {time.perf_counter() - start:.1f}s")    # about 2.0s, not 4.5s
16
17 asyncio.run(main())

```

The three waits overlap, so the total is the longest single wait, about two seconds, not the sum of 4.5. Trace the flow: `gather` schedules all three coroutines; the first runs until its `await asyncio.sleep(1)`, at which point control returns to the loop, which starts the second, and so on. Whenever a sleep expires, the loop resumes that coroutine right where it left off, exactly like resuming a paused generator. Nothing ever runs at the same instant; the concurrency comes entirely from never wasting the time spent waiting.

When you want a coroutine to start running in the background while you carry on with other things, `asyncio.create_task` schedules it immediately and returns a `Task` you can `await` later:

```

1 import asyncio
2
3 async def background_check():
4     await asyncio.sleep(1)
5     return "check complete"
6
7 async def main():
8     task = asyncio.create_task(background_check())    # starts running now
9     print("doing other work meanwhile")
10    await asyncio.sleep(0.5)
11    print("still working")
12    print(await task)                                # wait for the result now
13
14 asyncio.run(main())

```

Try It Yourself 3.13a: Write a coroutine `slow_double(n)` that awaits `asyncio.sleep(1)` and returns `n * 2`. Use `asyncio.gather` to process the numbers 1 through 5 at once, print the results, and print the elapsed time to confirm it is about one second.

The Golden Rule: Never Block the Loop

Because there is only one thread, a single blocking call stops *everything*. The most common way to do this by accident is `time.sleep`, or any ordinary blocking function such as `requests.get`.

```

1 import asyncio
2 import time
3
4 async def bad():
5     time.sleep(2)                                # BLOCKS the whole loop: nothing else can run

```

```

6     return "bad"
7
8     async def good():
9         await asyncio.sleep(2)      # yields control: other tasks run meanwhile
10        return "good"
11
12    async def main():
13        start = time.perf_counter()
14        await asyncio.gather(bad(), bad(), bad())
15        print(f"bad: {time.perf_counter() - start:.1f}s")    # about 6.0s
16
17        start = time.perf_counter()
18        await asyncio.gather(good(), good(), good())
19        print(f"good: {time.perf_counter() - start:.1f}s")    # about 2.0s
20
21    asyncio.run(main())

```

`time.sleep(2)` halts the entire thread, and with it the event loop, so the three “concurrent” calls run one after another. `await asyncio.sleep(2)` hands control back so the others can proceed. The same principle applies to any slow operation: to be useful in async code, it must itself be awaitable. That is why async programs use libraries built for the purpose, such as `httpx` and `aiohttp` for HTTP, rather than `requests`.

Sometimes you must call blocking code anyway, from a library that has no async version. `asyncio.to_thread` runs an ordinary function in a separate thread and gives you back something you can `await`, keeping the loop free.

```

1     import asyncio
2     import requests
3
4     def blocking_fetch(url):
5         return requests.get(url, timeout=10).status_code
6
7     async def main():
8         codes = await asyncio.gather(
9             asyncio.to_thread(blocking_fetch, "https://python.org"),
10            asyncio.to_thread(blocking_fetch, "https://example.com"),
11        )
12        print(codes)
13
14    asyncio.run(main())

```

Limiting Concurrency and Adding Timeouts

Starting a thousand requests at the same instant can overwhelm both your machine and the server. An `asyncio.Semaphore` works like a set of tickets: only a fixed number of coroutines may hold one at a time, and the rest wait their turn.

```

1     import asyncio
2
3     async def fetch(number, semaphore):
4         async with semaphore:      # at most 3 inside at once
5             print(f"start {number}")
6             await asyncio.sleep(1)
7             print(f"end {number}")
8             return number
9
10    async def main():

```

```

11     semaphore = asyncio.Semaphore(3)
12     results = await asyncio.gather(*(fetch(n, semaphore) for n in range(9)))
13     print(results)
14
15 asyncio.run(main())

```

`async with` is the asynchronous form of the `with` statement from Lesson 3.11: it acquires the semaphore on entry and releases it on exit, even if an error occurs. Nine tasks run in three waves of three, taking about three seconds. The expression `gather(*(...))` uses the unpacking from Lesson 1.11 to pass a generator's items as separate arguments, so `gather` receives one coroutine per number. Timeouts are equally simple with `asyncio.wait_for`, which cancels the operation and raises `TimeoutError` if it takes too long:

```

1 import asyncio
2
3 async def slow():
4     await asyncio.sleep(10)
5
6 async def main():
7     try:
8         await asyncio.wait_for(slow(), timeout=2)
9     except TimeoutError:
10        print("gave up after 2 seconds")
11
12 asyncio.run(main())

```

Threads, Processes, or asyncio?

The three tools solve overlapping problems, and choosing between them follows directly from the nature of the work.

Tool	Best for	How it switches	Main cost
Threads	Moderate amounts of waiting work, using ordinary blocking libraries	The operating system, at any moment	Race conditions; needs locks
Processes	CPU-bound computation across cores	Separate interpreters run truly in parallel	Startup cost; data must be pickled
asyncio	Very many slow network operations at once	You, at each <code>await</code>	Everything must be <code>async</code> -aware

Try It Yourself 3.13b: Write a coroutine `fake_request(id, semaphore)` that acquires a semaphore of size 2, awaits a random sleep between 0.2 and 1 second, and returns the id. Launch ten of them with `gather`, and print each id as it finishes to confirm no more than two are ever active.

Producers and Consumers with `asyncio.Queue`

When work arrives over time rather than all at once, such as URLs discovered while crawling, a queue connects the code that *produces* work to a fixed set of *workers* that consume it. `asyncio.Queue` is the `async` counterpart of the thread queue in Lesson 3.12; its `put` and `get` methods are awaited, so a worker waiting for work does not block the loop.

```

1 import asyncio
2 import random
3
4 async def worker(name, queue, results):
5     while True:

```

```

6         item = await queue.get()           # waits here when the queue is empty
7         try:
8             await asyncio.sleep(random.uniform(0.1, 0.4))
9             results.append((name, item * item))
10        finally:
11            queue.task_done()               # tell the queue this item is finished
12
13    async def main():
14        queue = asyncio.Queue()
15        results = []
16
17        workers = [asyncio.create_task(worker(f"w{i}", queue, results)) for i in range(3)]
18
19        for number in range(10):
20            await queue.put(number)
21
22        await queue.join()                 # wait until every item is processed
23
24        for task in workers:
25            task.cancel()                   # the workers loop forever; stop them
26        await asyncio.gather(*workers, return_exceptions=True)
27
28        print(sorted(item for _, item in results))
29
30    asyncio.run(main())

```

Trace the design. Three worker tasks start and immediately wait on `queue.get()`. As `main` puts ten items in, each waiting worker takes one, processes it, and comes back for more, so the three share the ten items among them, however long each takes. Every `get` must be matched by a `task_done()` call, placed in a `finally` so it happens even if processing fails; `await queue.join()` then returns only when every item that was ever put has been marked done. The workers themselves never end, because `while True` has no exit, so the main code cancels them afterwards, using the cancellation behavior from Lesson 3.14, and gathers them with `return_exceptions=True` to swallow the resulting `CancelledError`. This structure, a fixed number of workers pulling from a shared queue, is the async equivalent of a thread pool, and it limits concurrency to three just as a semaphore would, while also handling work that is added while others are running. Passing a `maxsize` to `Queue` makes `put` wait when the queue is full, which prevents a fast producer from overwhelming slow consumers.

Async Iteration: `async for`

Some sources of data arrive gradually and need awaiting between items: lines from a network stream, pages from an API, messages from a chat connection. Python has an asynchronous form of the iteration protocol from Lesson 3.8, and an asynchronous form of the generator from Lesson 3.9: an `async def` function that contains `yield`, consumed with `async for`.

```

1    import asyncio
2
3    async def ticker(count, delay):
4        for number in range(1, count + 1):
5            await asyncio.sleep(delay)      # wait for the next item to "arrive"
6            yield number
7
8    async def main():
9        async for value in ticker(4, 0.5):
10            print("received", value)
11
12        squares = [n * n async for n in ticker(4, 0.1)]
13        print(squares)

```

```

14
15 asyncio.run(main())

```

An *async generator* may await between yields, so while it waits for the next value, the event loop is free to run other tasks. `async for` can only be used inside an `async def`, and the comprehension form, the last-but-one line, follows Lesson 1.10 with `async` added. You would rewrite the paginated API reader of Lesson 3.4 this way when using an `async HTTP` library, so that fetching the next page never freezes the rest of the program.

Try It Yourself 3.13c: Write an `async generator` `countdown(n)` that yields `n`, `n-1`, down to 1, with a short `asyncio.sleep` between values, and consume it with `async for` while a second task prints a dot every 0.2 seconds, demonstrating that the loop stays responsive.

Common Mistakes to Avoid

- **Calling a coroutine without `await`.** `greet("Ada", 1)` alone only creates a coroutine object and runs nothing; Python warns that it was never awaited.
- **Using `time.sleep` or `requests.get` inside a coroutine.** Blocking calls freeze the whole event loop; use `asyncio.sleep`, an `async` library, or `asyncio.to_thread`.
- **Awaiting tasks one at a time and expecting concurrency.** Use `gather` or `create_task` so they are all started before any is waited on.
- **Launching thousands of tasks with no limit.** Use a `Semaphore` to cap how many run at once.
- **Calling `asyncio.run` inside a running loop.** It can be called only once, from ordinary code; inside a coroutine, `await` instead.

Lesson 3.13 Project: Async Downloader Simulation

Write a program called `async_downloader.py` that:

1. Defines a coroutine `download(name, size, semaphore)` that acquires the semaphore, prints a start message, sleeps for a time proportional to `size`, prints a finish message, and returns the name and size.
2. Builds a list of at least twelve simulated downloads with different sizes, and runs them all with `asyncio.gather`, with a semaphore limiting the number active at once to four.
3. Times the whole run and prints it alongside the time the same downloads would have taken sequentially, computed as the sum of their delays.
4. Wraps each download in `asyncio.wait_for` with a timeout, so that one deliberately slow download times out; catches `TimeoutError` inside the wrapper and reports that item as failed instead of crashing everything.
5. Uses `asyncio.to_thread` to run one ordinary blocking function alongside the `async` downloads, proving the loop stays responsive.

This project deliberately combines concurrency, a concurrency limit, and timeouts, the three ingredients of every real `async` network tool.

Lesson Summary: Quick Reference

- `asyncio` runs many tasks on one thread; the event loop switches only at each `await`
- `async def` defines a coroutine; `asyncio.run(main())` starts the loop; `gather` and `create_task` run coroutines concurrently
- Never block the loop: use `asyncio.sleep`, `async` libraries, or `asyncio.to_thread` for blocking calls
- `Semaphore` limits concurrency; `wait_for` adds a timeout and raises `TimeoutError`
- `Threads` suit blocking libraries, `processes` suit CPU work, and `asyncio` suits very many network operations

Lesson 3.14: Error Handling in Concurrency

Why This Matters

Lesson 1.14 taught you that an exception travels up the call stack until something catches it. Concurrency breaks that model. When a function runs in another thread, another process, or a background task, there is no call stack connecting it to the code that started it, so an exception has nowhere to travel: it can vanish, be silently discarded, or surface far from its cause. A monitor that runs a hundred checks at once and quietly loses the ones that crashed is worse than useless, since it reports health it never verified. This lesson shows how errors behave in each concurrency tool from Lesson 3.12 and Lesson 3.13, and how to make sure none of them disappears.

Learning Objectives

By the end of this lesson, you will be able to:

- Explain why an exception in a thread does not reach the code that started it, and how to capture it
- Retrieve errors from futures in thread and process pools, and collect results and failures together
- Handle errors in `asyncio`, including `gather(return_exceptions=True)` and cancellation
- Choose between fail-fast and collect-everything strategies, and log concurrent errors usefully

Threads: Errors That Go Nowhere

When a plain `threading.Thread` raises an exception, the thread simply ends, printing a traceback to the error stream, while the main program carries on completely unaware.

```
1 import threading
2
3 def worker():
4     raise ValueError("something broke in the thread")
5
6 thread = threading.Thread(target=worker)
7 thread.start()
8 thread.join()
9 print("main program continues as if nothing happened")
```

Wrapping `thread.start()` in a `try/except` would catch nothing, because `start()` returns immediately, long before `worker` runs. The exception happens later, in a different thread. To capture it, catch it *inside* the thread's own function and store it where the main program can look, or use a pool, which does exactly this for you.

```
1 import threading
2
3 errors = []
4
5 def safe_worker():
6     try:
7         raise ValueError("something broke in the thread")
8     except Exception as error:
9         errors.append(error)
10
11 thread = threading.Thread(target=safe_worker)
12 thread.start()
13 thread.join()
14 print("errors collected:", errors)
```

Appending to a list from several threads is safe enough for this purpose, since a single `append` is atomic, but the safer

general answer is to use a `ThreadPoolExecutor`, described next, where the machinery for carrying exceptions across is already built in.

Pools: Errors Travel Inside the Future

When you submit work to a `ThreadPoolExecutor` or `ProcessPoolExecutor` from Lesson 3.12, an exception raised in the worker is *captured and stored* in the `Future`. It is re-raised in your code when you call `future.result()`, so `try/except` works there exactly as you would expect.

```

1 from concurrent.futures import ThreadPoolExecutor, as_completed
2
3 def check(n):
4     if n % 3 == 0:
5         raise ValueError(f"item {n} is invalid")
6     return n * 10
7
8 results = {}
9 failures = {}
10
11 with ThreadPoolExecutor(max_workers=4) as pool:
12     futures = {pool.submit(check, n): n for n in range(1, 8)}
13     for future in as_completed(futures):
14         n = futures[future]
15         try:
16             results[n] = future.result()
17         except Exception as error:
18             failures[n] = str(error)
19
20 print(results)          # {1: 10, 2: 20, 4: 40, 5: 50, 7: 70} (order may vary)
21 print(failures)         # {3: 'item 3 is invalid', 6: 'item 6 is invalid'}
```

Each task either produces a result or a stored exception. The `try/except` wraps `future.result()` *inside* the loop, so one failure is recorded and the others are still collected: the same “never let one item stop the batch” structure from Lesson 3.3, now applied to concurrent work. The dictionary `futures` maps each future back to its input, so the failure report can say *which* item failed. By contrast, `pool.map` re-raises the first worker exception when you iterate over its results, and abandons the remaining results at that point, so it suits all-or-nothing work but not batches where you want every outcome. `future.exception()` returns the exception object (or `None`) without raising it, a handy alternative when you would rather inspect than catch.

Process pools behave the same way, with one extra wrinkle. The exception must be sent back from another process, which means it is pickled, so custom exception classes with unusual constructors can fail to transfer. Stick to standard exception types, or exceptions whose constructor takes a single message, when errors need to cross a process boundary.

Try It Yourself 3.14a: Write a function `parse_price(text)` that converts a string like `"12.50"` to a float and raises a `ValueError` for bad input. Submit six strings (some invalid) to a thread pool, and print two dictionaries: successful parses and error messages, keyed by the original text.

asyncio: Gathering Errors

In `async` code, the behavior of `asyncio.gather` on failure surprises people. By default, as soon as one awaited coroutine raises, `gather` re-raises that exception to you immediately, while the other coroutines keep running unobserved in the background. Passing `return_exceptions=True` changes this: every coroutine is allowed to finish, and exceptions come back *as ordinary values in the result list*, alongside the successes.

```

1 import asyncio
2
3 async def fetch(n):
```

```

4     await asyncio.sleep(0.1)
5     if n == 2:
6         raise ConnectionError("server unreachable")
7     return n * 100
8
9 async def main():
10     results = await asyncio.gather(*(fetch(n) for n in range(4)), return_exceptions=True)
11     for n, result in enumerate(results):
12         if isinstance(result, Exception):
13             print(f"task {n} failed: {result}")
14         else:
15             print(f"task {n} succeeded: {result}")
16
17 asyncio.run(main())

```

The results list keeps the order of the inputs, so `results[2]` is the `ConnectionError` object itself, and `isinstance`, from Lesson 2.4, sorts failures from successes. This is the async equivalent of the collect-everything loop above. An unobserved failed task is a related trap: if you create `task` and never `await` it, an exception inside it is only reported, as a “Task exception was never retrieved” message, when the task is garbage collected, if at all. Always keep a reference to every task you create, and `await` or `gather` it.

Cancellation and Cleanup

Async tasks can be *cancelled*, for example when `wait_for` times out or the program shuts down. Cancellation is delivered by raising `asyncio.CancelledError` inside the coroutine at its current `await`. This means a `finally` block, or a context manager from Lesson 3.11, will still run and release resources properly.

```

1 import asyncio
2
3 async def worker():
4     try:
5         print("working...")
6         await asyncio.sleep(10)
7     except asyncio.CancelledError:
8         print("cancelled: cleaning up")
9         raise # always re-raise cancellation
10    finally:
11        print("resources released")
12
13 async def main():
14     task = asyncio.create_task(worker())
15     await asyncio.sleep(1)
16     task.cancel()
17     try:
18         await task
19     except asyncio.CancelledError:
20         print("main saw the cancellation")
21
22 asyncio.run(main())

```

The one hard rule is to *re-raise* `CancelledError` after doing any cleanup. Swallowing it makes the task ignore cancellation, which can hang a timeout or a shutdown indefinitely. Note also that a bare `except Exception:` does not catch it, since `CancelledError` deliberately inherits from `BaseException`, so ordinary error handlers do not accidentally intercept a cancellation request.

Retries, Timeouts, and Fail-Fast versus Collect

Two design questions come up every time you run work concurrently. First: when one task fails, should the others stop, or continue? *Fail-fast* suits tasks that only make sense together, where one failure invalidates the whole batch. *Collect everything* suits independent tasks, like checking many websites, where each result stands on its own; that is the structure of every example above. Second: how long should you wait? Every concurrent operation that touches the outside world needs a timeout, because a hung task in a pool occupies a worker permanently, and enough hung tasks silently freeze the whole pool.

```

1 import asyncio
2
3 async def with_retry(operation, attempts=3, delay=0.5):
4     for attempt in range(1, attempts + 1):
5         try:
6             return await asyncio.wait_for(operation(), timeout=5)
7         except (TimeoutError, ConnectionError) as error:
8             if attempt == attempts:
9                 raise
10            await asyncio.sleep(delay * 2 ** (attempt - 1))

```

This is the retry-with-backoff structure from Lesson 3.4, translated into async form: the `await asyncio.sleep` replaces `time.sleep`, so waiting between attempts does not freeze other tasks. Since Python 3.11, `asyncio.TaskGroup` offers a structured alternative to `gather`: tasks are started inside an `async with` block, and if any fails, the rest are cancelled and the errors are raised together as an `ExceptionGroup`, handled with `except*`. It is the modern fail-fast tool, worth exploring once the basics here feel natural.

Finally, log concurrent errors so you can tell where they came from. Adding `%(threadName)s` to a logging format string, from Lesson 3.6, tags every line with the thread that wrote it, and including the input that failed in the message tells you *which* task went wrong. In concurrent code, a traceback alone is often not enough to reconstruct what happened.

Try It Yourself 3.14b: Write three async coroutines that each take a number, sleep briefly, and raise a `RuntimeError` for one of the numbers. Run them with `gather(..., return_exceptions=True)`, then print a summary line: how many succeeded and how many failed.

Timeouts on Futures

A pool task that never returns holds a worker forever, and a `future.result()` with no limit waits just as long. Both the wait and the wider design need a bound. `future.result(timeout=seconds)` raises `concurrent.futures.TimeoutError` if the answer is not ready in time.

```

1 from concurrent.futures import ThreadPoolExecutor, TimeoutError as FutureTimeout
2 import time
3
4 def slow(n):
5     time.sleep(n)
6     return n
7
8 with ThreadPoolExecutor(max_workers=2) as pool:
9     futures = [pool.submit(slow, n) for n in (0.2, 3)]
10    for future in futures:
11        try:
12            print("result:", future.result(timeout=1))
13        except FutureTimeout:
14            print("gave up waiting")
15            future.cancel()

```

Read the behavior carefully, because it contains a subtlety. The timeout applies only to *your waiting*, not to the work: when the wait expires, the worker thread is still running `slow(3)`, and Python has no way to stop a running thread. `future.cancel()` succeeds only for a task that has not started yet. Furthermore, the `with` block will still wait for that thread to finish before the program moves past it. The practical lessons are these: put the timeout *inside* the work itself wherever possible, as you did with `requests.get(timeout=...)` in Lesson 3.4, since that stops the operation for real; and treat a pool-level timeout as a way to stop *waiting*, not a way to stop *working*. For work that must be forcibly stoppable, a separate process is the tool, because a process can be terminated from outside.

TaskGroup and ExceptionGroup

Python 3.11 added `asyncio.TaskGroup`, a structured way to run several tasks and to handle their failures. Tasks are created inside an `async with` block; the block does not finish until every task has, and if any task fails, the others are cancelled and the errors are raised together as an `ExceptionGroup`, which is caught with the `except*` form.

```

1 import asyncio
2
3 async def job(name, fail):
4     await asyncio.sleep(0.1)
5     if fail:
6         raise ConnectionError(f"{name} lost its connection")
7     return name
8
9 async def main():
10     try:
11         async with asyncio.TaskGroup() as group:
12             first = group.create_task(job("a", False))
13             second = group.create_task(job("b", True))
14             third = group.create_task(job("c", True))
15     except* ConnectionError as errors:
16         for error in errors.exceptions:
17             print("failed:", error)
18
19 asyncio.run(main())

```

Because two tasks failed at about the same time, a single exception could not describe what happened, so Python raises an `ExceptionGroup` holding both. `except* ConnectionError` matches the group's members of that type, and `errors.exceptions` is the tuple of them. A plain `except` would not do here, because the exception that arrives is the *group*, not the individual errors. Compared with `gather`, a `TaskGroup` guarantees that no task is left running unobserved when the block exits, which removes the forgotten-task problem described earlier in this lesson. It is the right default for tasks that succeed or fail together; for independent tasks where you want every outcome regardless, `gather(return_exceptions=True)` remains the better fit.

Try It Yourself 3.14c: Write three coroutines where one raises `ValueError` and one raises `KeyError`, and run all three in a `TaskGroup`. Use two `except*` clauses, one for each type, and print a message from each to see that both are handled in one pass.

Common Mistakes to Avoid

- **Wrapping `thread.start()` in `try/except`.** The exception happens later in another thread and is never seen; catch it inside the thread function or use a pool.
- **Never calling `future.result()`.** A worker's exception stays hidden inside the future forever, and the failure goes unreported.
- **Using `pool.map` when you need every outcome.** It stops at the first raised error; use `submit` and `as_completed` with `try/except` to gather all results.

- **Swallowing `CancelledError`.** Catching it without re-raising makes a task ignore cancellation and can hang shutdowns and timeouts.
- **Creating tasks and forgetting them.** An un-awaited failed task reports its error late or never; keep references and await or gather every task.

Lesson 3.14 Project: Resilient Batch Runner

Write a program called `resilient_runner.py` that:

1. Defines a function `flaky_task(n)` that raises a `ValueError` for multiples of 4, a `TimeoutError` for multiples of 5, and otherwise returns a result after a short `time.sleep`.
2. Runs the numbers 1 to 20 through a `ThreadPoolExecutor`, using `submit` and `as_completed`, collecting results and failures into two dictionaries and logging each failure with the number and the thread name.
3. Adds a retry wrapper, with exponential backoff, that retries only the `TimeoutError` cases, then shows in the output which items succeeded after retrying.
4. Reimplements the same batch in `asyncio` using `gather(return_exceptions=True)`, and prints both summaries side by side.
5. Ends with a report listing every failure, and exits with code 1 if any item failed permanently, as in Lesson 3.1.

This project deliberately implements the same collect-everything design in two concurrency models, with no error allowed to disappear.

Lesson Summary: Quick Reference

- An exception in a thread or task has no call stack to travel up, so it must be captured on purpose
- Pools store exceptions in the `Future`; they are re-raised at `future.result()`, where `try/except` works normally
- `pool.map` stops at the first error; `submit` plus `as_completed` collects every result and failure
- `asyncio.gather(..., return_exceptions=True)` returns errors as values; always re-raise `CancelledError` after cleanup
- Every concurrent operation needs a timeout, and failures should be logged with enough context to identify the task

Lesson 3.15: Debugging with pdb

Why This Matters

Until now, when a program misbehaved you have probably done what every beginner does: sprinkled `print()` statements through the code, run it again, read the output, guessed, and repeated. It works, and it scales badly. Every print requires editing the file and rerunning, and you must decide in advance what to look at, often before you understand what is wrong. A debugger reverses this. It lets you stop a running program at any line, look at every variable, step through the code one line at a time, and try out expressions, all while the program is frozen exactly where the bug lives. Python includes one, `pdb`, in the standard library, and knowing it is the difference between hunting for a bug and simply walking up to it.

Learning Objectives

By the end of this lesson, you will be able to:

- Start a debugging session with `breakpoint()` and with `python -m pdb`
- Step through code with `n`, `s`, `c`, and `r`, and inspect state with `p`, `pp`, `l`, `w`, and `u`
- Set, list, and clear breakpoints, including conditional ones
- Perform a post-mortem on a crashed program, and follow a systematic method for finding a bug

Stopping the Program: `breakpoint()`

The simplest way into the debugger is one built-in function. When Python reaches a call to `breakpoint()`, it pauses the program and opens an interactive `pdb` prompt at that exact line.

```
1 def average(numbers):
2     total = 0
3     for n in numbers:
4         total += n
5     breakpoint()           # execution pauses here
6     return total / len(numbers)
7
8 print(average([4, 8, 15, 16]))
```

Running this script stops at the `breakpoint()` line and shows a prompt that looks like this:

```
> /home/me/demo.py(6)average()
-> return total / len(numbers)
(Pdb) p total
43
(Pdb) p numbers
[4, 8, 15, 16]
(Pdb) p total / len(numbers)
10.75
(Pdb) c
10.75
```

The line beginning with `>` tells you the file, the line number, and the function; the line beginning with `->` shows the next line *about to run*, not one that has already run. At the `(Pdb)` prompt you type commands. `p total` prints a value, `p` accepts any expression (even one that calls functions), and `c` means continue, resuming the program until it ends or reaches another breakpoint. Remove the `breakpoint()` line when you finish; leaving one in code that reaches production would freeze it waiting for someone to type at a terminal that does not exist.

The Commands You Will Use Every Time

The debugger has dozens of commands, but a small core covers nearly all real work. Each has a one-letter shortcut, and pressing Enter on an empty line repeats the previous command.

Command	What it does
n (next)	Run the current line, staying in this function; a call on the line runs to completion
s (step)	Run the current line, but step <i>into</i> any function it calls
c (continue)	Resume running until the next breakpoint or the end
r (return)	Run until the current function returns
l / ll	List source code around the current line / the whole current function
p expr / pp expr	Print / pretty-print the value of any expression
w (where)	Show the call stack, the chain of calls that led here
u / d	Move up / down one level in the call stack, to inspect the caller's variables
q (quit)	Stop the debugger and end the program

The difference between n and s is the one worth practicing. Stepping over a function call with n treats it as a single action, right when you already trust that function. Stepping into it with s takes you inside to watch it work, right when you suspect that function is the culprit. w and u answer the question every bug eventually raises: “how did we get here, and what did the caller pass in?”

Try It Yourself 3.15a: Create a script with a function `total_price(items)` that loops over a list of (name, price) tuples and returns the sum. Add `breakpoint()` inside the loop, run it, and use n, p price, and c to watch the running total change on each pass. Try q to stop early.

Running a Whole Script Under pdb

You do not have to edit the source. Launching a script as `python -m pdb script.py` starts the debugger on its first line, from where you can set your own breakpoints. The command b sets one at a line number (or function name), b alone lists them, cl clears them, and c runs to the first one.

```
$ python -m pdb report.py
> report.py(1)<module>()
-> import json
(Pdb) b 24
Breakpoint 1 at report.py:24
(Pdb) b summarize
Breakpoint 2 at report.py:15
(Pdb) c
> report.py(15)summarize()
-> total = 0
(Pdb) w
    report.py(40)<module>()
-> summarize(data)
> report.py(15)summarize()
-> total = 0
```

A powerful refinement is the *conditional* breakpoint: `b 24, price < 0` pauses at line 24 only when price is negative. If a loop runs a hundred thousand times and the bug appears on the ten-thousandth pass, this stops you exactly there instead of making you type c ten thousand times. Similarly, if you are debugging code you cannot easily edit, the environment variable `PYTHONBREAKPOINT=0` disables every `breakpoint()` call without touching the source, which is a handy safety switch.

Post-Mortem: Debugging After the Crash

A crash gives you a traceback, but the traceback shows only where the program died, not what the variables held at that moment. Post-mortem debugging opens the debugger *at the point of the crash*, with all the state still intact. Running `python -m pdb script.py` does this automatically: if the script raises an uncaught exception, pdb catches it and drops you into a prompt inside the failing function instead of exiting. You can also request it from code, using the `try/except` structure from Lesson 1.14:

```

1 import pdb
2
3 def parse(record):
4     return int(record["count"]) * 2
5
6 records = [{"count": "5"}, {"count": "7"}, {"count": "n/a"}]
7
8 try:
9     for record in records:
10        print(parse(record))
11 except Exception:
12     pdb.post_mortem()           # opens pdb inside the crashed frame

```

When the third record raises a `ValueError`, `pdb.post_mortem()` opens the debugger at the exact line inside `parse` where `int(...)` failed. There you can type `p record` and see `{'count': 'n/a'}` immediately, then `u` to move up into the loop and check which record it was. This is often the fastest possible route from a crash to a cause.

A Method, Not Just a Tool

A debugger only helps if you use it deliberately. The reliable approach is the same every time. First, *reproduce* the bug consistently; a bug you cannot trigger on demand is very hard to fix. Second, *form a hypothesis*: what specific wrong value or wrong path do you suspect? Third, *stop just before the suspect line* and check that hypothesis by looking at the actual values, not what you assumed they were. Fourth, *narrow it down*: if the values there are already wrong, the bug is earlier, so move the breakpoint up; if they are correct, the bug is later, so step forward. Each round cuts the search in half, which is why this beats scattering prints. When you find the cause, fix it and then confirm with the original reproduction that the bug is really gone.

Try It Yourself 3.15b: Write a short function with a deliberate off-by-one error, for example a loop `range(1, len(items))` that skips the first item. Run it with `python -m pdb`, set a breakpoint in the loop, and use `p` to discover which item is being skipped. Then fix it and rerun.

Reading a Traceback Properly

Before reaching for any tool, learn to read what Python already tells you. A traceback is the record of the chain of calls that led to an error, printed with the *oldest* call first and the point of failure last. Beginners often read from the top and stop, when the answer is nearly always at the bottom.

```

Traceback (most recent call last):
  File "report.py", line 22, in <module>
    print(summarize(load("data.json")))
  File "report.py", line 14, in summarize
    return total / count
ZeroDivisionError: division by zero

```

Read it from the bottom. The last line names the error type and its message: `ZeroDivisionError: division by zero`. The pair of lines above it show where it was raised: line 14 of `report.py`, inside the function `summarize`, on the code `total / count`. Each pair above that is the caller: line 22 called `summarize`, and that was in the top-level code of the module. So the question to ask is not “why did line 22 fail?”, it is “how did `count` become zero when `summarize` was

called with this data?” The traceback has just told you where to set a `breakpoint()`: on line 14, or in the caller if you suspect the bad value arrived from outside. When one error causes another, Python prints both, with the line “During handling of the above exception, another exception occurred” between them, and the *first* error is usually the root cause.

assert: Catching Bugs Where They Start

A large share of debugging time is spent tracing a wrong value back to where it first went wrong. An `assert` statement checks a condition that must always be true at that point and stops the program immediately, at the exact place the assumption is broken, if it is not.

```

1 def average(numbers):
2     assert len(numbers) > 0, "average() needs at least one number"
3     return sum(numbers) / len(numbers)
4
5 def apply_discount(price, percent):
6     assert 0 <= percent <= 100, f"percent out of range: {percent}"
7     return price * (1 - percent / 100)
8
9 print(apply_discount(80, 25))          # 60.0
10 print(apply_discount(80, 250))        # AssertionError: percent out of range: 250

```

`assert` condition, message does nothing when the condition is true, and raises `AssertionError` with the message when it is false. The error now appears in `apply_discount`, at the call that supplied the bad percentage, rather than three functions later as a mysteriously negative total. Assertions document your assumptions in code, and they make bugs loud and close to their source. Two cautions apply. Assertions are for catching *programmer* mistakes, conditions that should be impossible if the code is correct; they are not for validating user input or a config file, where errors are expected and deserve a proper `ValueError` and a helpful message, as in Lesson 3.5. And Python can be run with the `-O` flag, which removes every `assert` completely, so never put anything essential inside one, such as `assert do_important_work()`, since it would vanish.

Printing Well When You Must Print

There are moments when a quick `print` really is the right tool, and it helps to do it well. An f-string with the `=` specifier prints both the expression and its value, and `repr` formatting shows exactly what a value is.

```

1 count = 0
2 name = "  Ada  "
3 items = ["a", "b"]
4
5 print(f"{count=}")          # count=0
6 print(f"{name=}")           # name='  Ada  '    the quotes and spaces are now visible
7 print(f"{len(items)=}")     # len(items)=2
8 print(f"{name!r}")          # '  Ada  '

```

`{count=}` prints `count=0`, saving you from typing the label twice and from mislabelling the output, and because it uses the `repr` form, the whitespace hidden in `name` becomes visible. This is a small habit with a large payoff, since a great many bugs turn out to be a stray space, an empty string, or the string `"5"` where the number 5 was expected, and only `repr` shows the difference. Use `prints` for a quick look, the debugger when you need to explore, logging when the message should stay in the program, and assertions when you want the program to guard itself.

Try It Yourself 3.15c: Take a small function of your own that you have written in an earlier lesson and add two meaningful `assert` statements that state what must be true about its inputs. Then call it with an input that breaks one, and read the resulting traceback from the bottom up, writing one sentence saying what it tells you.

Common Mistakes to Avoid

- **Leaving breakpoint() in committed code.** The program will freeze waiting for input wherever it runs; remove it, or set PYTHONBREAKPOINT=0.
- **Confusing n and s.** n runs a function call as one step; use s when you actually want to go inside it.
- **Assuming the -> line has already run.** It shows the line *about to* execute; its effects are not yet visible in the variables.
- **Naming a variable after a debugger command.** Typing n or c at the prompt runs the command, so to see a variable called n, use p n.
- **Debugging without a hypothesis.** Stepping aimlessly through every line wastes time; decide what you suspect, then test exactly that.

Lesson 3.15 Project: Bug Hunt

Write a program called `bug_hunt.py` containing at least four small functions, each with a different deliberate bug, and then debug your own code:

1. A function with an off-by-one error in a loop range, a function that mutates a list while looping over it, a function that returns None because of a missing return, and a function that divides by zero for one specific input.
2. For each bug, add a comment that says which pdb commands you used to find it, for example the breakpoint you set, the expressions you printed, and the level you moved up to.
3. Use `breakpoint()` for one, `python -m pdb` with a line-number breakpoint for another, a conditional breakpoint for the third, and `pdb.post_mortem()` for the crash.
4. Fix every bug, add a comment showing the corrected line, and end the file with assertions that confirm every function now returns the right answer.

This project deliberately practices each entry point into the debugger, so you know which one fits which kind of bug.

Lesson Summary: Quick Reference

- `breakpoint()` pauses a running program at that line; `python -m pdb script.py` starts the debugger without editing the source
- n steps over, s steps into, c continues, r runs to the return, and q quits
- p and pp print any expression; l, w, u, and d show the code and the call stack
- b line, condition sets a conditional breakpoint; `pdb.post_mortem()` debugs from the point of a crash
- Reproduce, hypothesize, check the real values, and narrow the search: debugging is a method, not guessing

Lesson 3.16: Performance Profiling

Why This Matters

Every programmer eventually meets a program that is too slow, and nearly every programmer's first instinct is to guess: it must be that loop, or that function call, or Python itself. Guessing is wrong far more often than it is right. Programs spend their time in surprising places, and an hour spent speeding up a function that accounts for two percent of the runtime is an hour wasted. Profiling replaces guessing with measurement: it tells you exactly where the time and the memory actually go, so you fix the one thing that matters. The rule that guides this whole lesson is old and reliable: *measure first, then optimize, then measure again*.

Learning Objectives

By the end of this lesson, you will be able to:

- Time small pieces of code reliably with `time.perf_counter()` and `timeit`
- Profile a whole program with `cProfile` and read the report, telling `tottime` from `cumtime`
- Measure memory use with `tracemalloc`
- Recognize the common causes of slow Python code and fix them, confirming each fix with a measurement

Timing a Piece of Code

The simplest measurement is the clock. `time.perf_counter()` returns a high-resolution timestamp in seconds, meant specifically for measuring intervals, and you have already used it in Lessons 3.10 and 3.11. Subtracting two readings gives the elapsed time.

```
1 import time
2
3 start = time.perf_counter()
4 total = sum(i * i for i in range(1_000_000))
5 elapsed = time.perf_counter() - start
6 print(f"{elapsed:.4f} seconds")
```

This works, but a single reading is unreliable. Other programs compete for the processor, the first run may be slower while caches warm up, and tiny operations finish faster than the clock can resolve. The `timeit` module solves this by running the code many times and reporting the total, so that noise averages out.

```
1 import timeit
2
3 loop_time = timeit.timeit(
4     "result = []\nfor i in range(1000):\n    result.append(i * 2)",
5     number=2000,
6 )
7 comp_time = timeit.timeit("[i * 2 for i in range(1000)]", number=2000)
8
9 print(f"loop:           {loop_time:.3f}s")
10 print(f"comprehension: {comp_time:.3f}s")
```

`timeit.timeit(code, number=n)` runs the code string `n` times and returns the total seconds. Comparing the two totals settles a question that opinion cannot: on a typical machine the comprehension is noticeably faster, because it avoids the repeated method lookup and call that `append` requires on every pass. For a more trustworthy figure, `timeit.repeat` runs the whole measurement several times, and the convention is to take the *minimum*, since slower results are usually caused by interference rather than by your code. Notice the shape of the workflow: form two candidate versions, time both under identical conditions, and let the numbers decide.

Try It Yourself 3.16a: Use `timeit` to compare building a string of the numbers 0 to 999 by repeated `+=` inside a loop against `"".join(str(i) for i in range(1000))`. Run each 500 times and print both totals.

Profiling a Whole Program with cProfile

Timing tells you how long something takes, but only once you already know what to time. When a program of many functions is slow, the first question is *which* function. `cProfile`, built into Python, records every function call while the program runs and reports how much time each one consumed.

```

1 import cProfile
2 import pstats
3
4 def is_prime(n):
5     if n < 2:
6         return False
7     for d in range(2, n):
8         if n % d == 0:
9             return False
10    return True
11
12 def count_primes(limit):
13     return sum(1 for n in range(limit) if is_prime(n))
14
15 def report(limit):
16     total = count_primes(limit)
17     return f"{total} primes below {limit}"
18
19 with cProfile.Profile() as profiler:
20     print(report(20_000))
21
22 stats = pstats.Stats(profiler)
23 stats.sort_stats("cumulative").print_stats(6)

```

The `with` block, using the context manager idea from Lesson 3.11, starts recording on entry and stops on exit. `pstats.Stats` then formats the recorded data, and `sort_stats("cumulative")` orders it so that the most expensive functions come first. You can also profile an entire script without editing it, with `python -m cProfile -s cumulative script.py`. The report looks like this:

ncalls	totttime	percall	cumtime	percall	filename:lineno(function)
1	0.000	0.000	1.812	1.812	demo.py:16(report)
1	0.004	0.004	1.812	1.812	demo.py:12(count_primes)
20000	0.004	0.000	1.808	0.000	demo.py:6(is_prime)
17986262	1.801	0.000	1.801	0.000	{built-in method builtins.range}

Reading the columns is the skill. `ncalls` is how many times the function ran. `totttime` is the time spent *inside that function's own code*, excluding anything it called. `cumtime` is the *cumulative* time, including every function it called. `percall` divides each by the number of calls. The distinction between `totttime` and `cumtime` is what tells you where to look. `report` has a large `cumtime` but almost no `totttime`: it does no work itself, it merely calls things that do. To find the real cost, sort by `totttime` (`sort_stats("totttime")`) and read from the top, which exposes the functions doing the actual work. Here, the profile points straight at `is_prime`, and the millions of iterations inside its loop.

Acting on the Profile

The profile identified the target; now improve it, and measure again. `is_prime` tests every possible divisor up to `n`, but a number has no divisor larger than its square root unless it also has a smaller one, so checking beyond `sqrt(n)` is wasted work.

```

1 import math
2 import time
3
4 def is_prime_fast(n):
5     if n < 2:
6         return False
7     for d in range(2, math.isqrt(n) + 1):
8         if n % d == 0:
9             return False
10    return True
11
12 start = time.perf_counter()
13 count = sum(1 for n in range(20_000) if is_prime_fast(n))
14 print(count, f"{time.perf_counter() - start:.3f}s")

```

The change is one line, and it reduces the work from a number of steps proportional to n to one proportional to its square root. The measured time drops from over a second to a few hundredths. This illustrates the central truth of optimization: the biggest gains come from a better *algorithm*, not from clever tricks at the level of individual statements, and only a profile tells you which part deserves a better algorithm. Notice too the discipline of the loop: profile, change one thing, and measure again. If the second measurement had shown no improvement, you would revert the change rather than keep complexity that buys nothing.

Try It Yourself 3.16b: Profile a script that builds a list of 5,000 random numbers and then sorts it using a hand-written bubble sort (two nested loops). Print the top five entries sorted by `tottime`, then replace the bubble sort with the built-in `sorted` and confirm with a second measurement how much faster it is.

Measuring Memory with `tracemalloc`

Speed is one resource; memory is the other. A program that runs fast but consumes gigabytes will crash on a smaller machine. The standard library's `tracemalloc` tracks Python's memory allocations, and reports both the current and the peak usage.

```

1 import tracemalloc
2
3 tracemalloc.start()
4
5 data = [str(i) * 10 for i in range(200_000)]    # build a big list of strings
6
7 current, peak = tracemalloc.get_traced_memory()
8 print(f"current: {current / 1_000_000:.1f} MB")
9 print(f"peak:    {peak / 1_000_000:.1f} MB")
10
11 snapshot = tracemalloc.take_snapshot()
12 for stat in snapshot.statistics("lineno")[:3]:
13     print(stat)
14 tracemalloc.stop()

```

`start()` begins tracking. `get_traced_memory()` returns a pair of byte counts: what is allocated right now, and the highest point reached, which is the figure that decides whether a program fits in memory. A *snapshot* groups the allocations by source line, so `statistics("lineno")` shows which lines are responsible for the most memory. This is the tool that would have confirmed the point of Lesson 3.9 with real numbers: replace the list comprehension above with a generator expression, consume it, and the peak memory collapses to almost nothing, because only one string exists at any moment.

Common Causes of Slow Python, and Their Fixes

Profiles of real programs keep pointing at the same few patterns, and each has a standard remedy that you already know from earlier lessons.

```

1 import timeit
2
3 names = [f"user{i}" for i in range(10_000)]
4 name_set = set(names)
5
6 list_time = timeit.timeit("'user9999' in names", globals=globals(), number=2000)
7 set_time = timeit.timeit("'user9999' in name_set", globals=globals(), number=2000)
8 print(f"list membership: {list_time:.4f}s")
9 print(f"set membership: {set_time:.6f}s")

```

Checking membership in a list examines items one by one, so its time grows with the list's length. Checking a set or dictionary, using the hashing from Lesson 1.9, takes essentially the same time however large it is, and the measurement shows a difference of several hundred times. This is the most common quick win: if you test `in` repeatedly against a large collection, make it a set. A related pitfall is building a string by repeated `+=` inside a loop, where every step copies the growing string; collecting the pieces in a list and calling `"".join(pieces)` once is far faster, as your first Try It Yourself will show.

Other recurring fixes: move calculations that do not change out of a loop instead of repeating them on every pass; use `@lru_cache` from Lesson 3.10 when the same inputs recur; use a generator from Lesson 3.9 when memory is the problem; use threads or `asyncio` from Lessons 3.12 and 3.13 when the program is mostly waiting, and processes when it is computing. And when none of these is enough, the heavy numerical work is usually handed to libraries written in C, such as NumPy, which is a topic beyond this course. Every one of these is a *hypothesis* about what will help. The profiler tells you where to apply it, and a second measurement tells you whether it worked.

A Reusable Timing Decorator

Timing appears everywhere in performance work, and the decorator technique of Lesson 2.7 packages it so that you can measure any function by adding one line above it and remove the line when you are done.

```

1 import time
2 from functools import wraps
3
4 def timed(func):
5     @wraps(func)
6     def wrapper(*args, **kwargs):
7         start = time.perf_counter()
8         try:
9             return func(*args, **kwargs)
10        finally:
11            elapsed = time.perf_counter() - start
12            print(f"{func.__name__} took {elapsed * 1000:.2f} ms")
13        return wrapper
14
15 @timed
16 def build_squares(n):
17     return [i * i for i in range(n)]
18
19 build_squares(1_000_000)           # build_squares took 61.40 ms

```

The `try/finally` guarantees that the time is printed even when the function raises an exception, and the `@wraps(func)` keeps the function's name, which is used in the message. The same structure as the timing context manager in Lesson

3.11 works for a block of code inside a function, and you may reasonably prefer that form when you want to time only part of a function. What matters is the habit: whenever you believe something is slow, wrap it in a timer and look at the number before deciding anything.

Choosing the Right Data Structure

The most powerful speedups often come not from tuning code but from picking a structure whose operations are cheap for what you are doing. The membership test earlier in this lesson was one example; another is removing from the front of a sequence, where a list and a deque behave very differently.

```

1 import timeit
2 from collections import deque
3
4 setup = "from collections import deque\nitems = list(range(50_000))\nqueue = deque(items)"
5
6 list_time = timeit.timeit("items.pop(0)", setup=setup, number=20_000)
7 deque_time = timeit.timeit("queue.popleft()", setup=setup, number=20_000)
8
9 print(f"list.pop(0):      {list_time:.4f}s")
10 print(f"deque.popleft(): {deque_time:.6f}s")

```

Removing the first element of a list forces every remaining element to shift one place to the left, so the cost grows with the list's length. A deque, from the collections tools of Lesson 1.17, is built for adding and removing at both ends, and does so in the same tiny time no matter how long it is. The measurement shows a large difference, and it grows with the size. The pattern to remember is the one behind all such choices: each structure makes some operations cheap and others expensive. A list is best for indexing and appending at the end; a set or dictionary for membership and lookup by key; a deque for a queue; a sorted list with the bisect module for fast searching in ordered data. Before optimizing code, ask whether the data structure is the problem.

Doing Less Work

Some of the best optimizations remove work altogether. Three ideas recur. First, do not repeat a calculation whose result does not change: move it out of the loop, or cache it, as in Lesson 3.10. Second, stop as soon as you have your answer: any() and all() from Lesson 1.8 stop at the first decisive item, whereas building a list of all matches and then checking whether it is empty does all the work first. Third, do not fetch or store more than you need: reading a file line by line, as in Lesson 3.9, rather than loading it whole, and asking an API for only the fields you use, both cut time and memory before any clever code is written.

```

1 words = ["alpha", "beta", "gamma", "delta"] * 25_000
2
3 def has_long_word_slow(words):
4     long_words = [w for w in words if len(w) > 4]      # examines every word
5     return len(long_words) > 0
6
7 def has_long_word_fast(words):
8     return any(len(w) > 4 for w in words)              # stops at the first match
9
10 print(has_long_word_slow(words), has_long_word_fast(words))

```

Both functions return the same answer, but the first inspects a hundred thousand words and builds a list it immediately throws away, while the second stops after inspecting the very first word, since "alpha" already has more than four letters. On data where a match appears early, the second version is thousands of times faster, and it is also shorter and clearer. Comparing the two is a lesson in itself: the fastest code is frequently the simplest.

Try It Yourself 3.16c: Decorate two versions of a function that checks whether a list contains any negative number, one

that builds a full list of negatives and one that uses any. Run both on a list of 1,000,000 numbers whose first element is negative, and compare the printed timings.

Common Mistakes to Avoid

- **Optimizing without measuring.** Guessing usually targets the wrong code; profile first and fix only what the numbers point to.
- **Trusting a single timing.** One run is noisy; use `timeit` with many repetitions and compare minimums.
- **Confusing `tottime` with `cumtime`.** A function with a huge cumulative time may do no work itself; sort by `tottime` to find where the time is really spent.
- **Leaving the profiler on.** `cProfile` slows a program down noticeably, so use it to find where time goes, never to measure absolute speed.
- **Making code harder to read for a tiny gain.** An improvement that saves a fraction of a percent is not worth the complexity; keep the clear version unless the profile demands otherwise.

Lesson 3.16 Project: Profile and Fix

Write a program called `slow_report.py` that:

1. Contains a deliberately slow version of a small task, such as finding all duplicate words in a generated list of 50,000 words, using a list for membership tests and building the output string with `+=`.
2. Profiles it with `cProfile`, prints the top five lines sorted by `tottime`, and records in a comment which function the profile blames.
3. Writes an optimized version using a set or Counter and `"".join`, and confirms it produces exactly the same result with an `assert`.
4. Uses `timeit` to time both versions and prints the speedup as a ratio.
5. Uses `tracemalloc` to compare the peak memory of a list-based version and a generator-based version of one step, and prints both figures.

This project deliberately follows the full cycle of measure, change, and measure again, with correctness confirmed at each step.

Lesson Summary: Quick Reference

- Measure before optimizing: `time.perf_counter` for intervals, `timeit` for reliable comparisons of small snippets
- `cProfile` reports where a whole program spends time; `tottime` is a function's own time, `cumtime` includes its callees
- `tracemalloc` reports current and peak memory, and which lines allocated it
- Better algorithms and better data structures (sets for membership, `join` for strings) beat micro-tricks
- Always confirm an optimization with a second measurement, and keep the clearer code when the gain is small

Lesson 3.17: Design Patterns in Python

Why This Matters

Experienced programmers do not solve every problem from scratch. Certain design problems recur across nearly every large program, how to guarantee only one of something exists, how to create objects without hardcoding their exact class, how to notify several parts of a program when something happens, how to swap one algorithm for another, and over decades the community has named the standard solutions. These are *design patterns*. Knowing them gives you two things: a set of proven structures to reach for, and a shared vocabulary, so that saying “this is an observer” communicates a whole design in three words. This lesson builds four of the most useful, using the classes, abstract base classes, and decorators of Phase 2, and, just as importantly, shows how Python’s flexibility often makes a pattern simpler than it is in other languages.

Learning Objectives

By the end of this lesson, you will be able to:

- Explain what a design pattern is, and why it is a guide rather than a template to copy
- Implement a *Singleton*, and recognize when a module-level object is the simpler answer
- Implement a *Factory* that creates the right object from a name or setting
- Implement an *Observer* that notifies several listeners of an event
- Implement a *Strategy*, using either a class hierarchy or plain functions

Patterns Are Vocabulary, Not Recipes

A pattern is a description of a problem and the shape of a solution, not a block of code to paste. The same pattern looks different in different languages, and in Python many patterns nearly disappear, because functions are ordinary values (Lesson 1.11), classes are objects too, and modules already behave like single shared instances. The goal is to recognize the underlying problem when you meet it. A pattern that adds machinery your program does not need is worse than none, so each pattern below begins with the problem it solves, and ends with when *not* to use it.

Singleton: Exactly One Instance

The problem: some things should exist only once in a program: a configuration object, a connection pool, a logger. If different parts of the program created their own copies, they could disagree, and the point of sharing would be lost. A Singleton is a class that hands out the same single instance every time you ask for one.

```
1 class Config:
2     _instance = None
3
4     def __new__(cls):
5         if cls._instance is None:
6             print("Creating the one Config")
7             cls._instance = super().__new__(cls)
8             cls._instance.settings = {}
9             return cls._instance
10
11 a = Config()
12 b = Config()
13 a.settings["timeout"] = 10
14
15 print(a is b)           # True: the very same object
16 print(b.settings["timeout"]) # 10: a change through a is visible through b
```

Trace the mechanism. Calling `Config()` first runs the special method `__new__`, which *creates* the object, before

`__init__` initializes it (Lesson 2.1 covered `__init__`; `__new__` is the earlier step). The class attribute `_instance`, a class attribute in the sense of Lesson 2.1, starts as `None`. The first call finds it empty, creates a real instance through `super().__new__(cls)`, stores it on the class, and returns it. Every later call finds `_instance` already set, skips creation, and returns the stored object, so `a is b` is `True`. Note that "Creating the one Config" prints only once.

However, in Python there is usually a simpler way that needs no special class at all: a *module* is itself created only once, no matter how many files import it.

```
1 # settings.py
2 timeout = 10
3 retries = 3
```

```
1 # any_other_file.py
2 import settings
3
4 settings.timeout = 30      # every other file that imports settings sees 30
```

Python caches imported modules (Lesson 1.15), so `settings` is a single shared object across the entire program. For plain shared configuration, this is almost always the better choice, shorter and clearer than a Singleton class. Reserve the class version for cases where you need methods, initialization logic, or lazy creation. Be wary also of what a Singleton really is: global state under another name. It makes code harder to test, since every test shares the same instance, so use it sparingly.

Try It Yourself 3.17a: Write a Counter Singleton class with a `count` attribute and an `increment()` method. Create two variables that each call `Counter()`, `increment` through one, and print the count through the other to confirm they share state.

Factory: Creating the Right Object

The problem: code that needs an object often should not care exactly which class it gets. A notification system may need to send by email, by text message, or to a chat channel, chosen by a setting. If every caller writes `if kind == "email": EmailNotifier() elif ...`, that decision is duplicated everywhere, and adding a fourth kind means editing every copy. A Factory centralizes the decision in one place.

```
1 from abc import ABC, abstractmethod
2
3 class Notifier(ABC):
4     @abstractmethod
5     def send(self, message):
6         ...
7
8 class EmailNotifier(Notifier):
9     def send(self, message):
10         print(f"[email] {message}")
11
12 class SmsNotifier(Notifier):
13     def send(self, message):
14         print(f"[sms] {message}")
15
16 class ConsoleNotifier(Notifier):
17     def send(self, message):
18         print(f"[console] {message}")
19
20 NOTIFIERS = {
21     "email": EmailNotifier,
```

```

22     "sms": SmsNotifier,
23     "console": ConsoleNotifier,
24 }
25
26 def create_notifier(kind):
27     try:
28         return NOTIFIERS[kind]()
29     except KeyError:
30         raise ValueError(f"Unknown notifier: {kind!r}") from None
31
32 notifier = create_notifier("sms")
33 notifier.send("Site is down")

```

The abstract base class from Lesson 2.8 defines the contract every notifier must honor, so the code using a notifier can rely on `send` existing. The factory is the function `create_notifier`. Where other languages would need a long chain of conditions, Python uses a dictionary from Lesson 1.9 that maps each name to the *class itself*, since a class is a value like any other, and calling the looked-up class, `NOTIFIERS[kind]()`, builds an instance. Adding a fourth kind means writing one class and adding one dictionary entry; no calling code changes. The `from None` suppresses the confusing chained traceback, leaving one clear error message. A factory is especially natural when the kind comes from configuration, as in Lesson 3.5: the setting `notifier = "email"` in a config file becomes `create_notifier(settings.notifier)`. It is unnecessary when there is only ever one class, in which case calling the class directly is clearer.

Observer: Telling Everyone Who Cares

The problem: when something happens, several unrelated parts of a program may need to react, and the part that detects the event should not have to know about them all. A monitor that discovers a site is down might need to write a log entry, send an alert, and update a counter. If the monitor calls all three directly, it becomes tangled with each of them, and adding a fourth reaction means editing the monitor. In the Observer pattern, the source of events keeps a list of *listeners* and simply announces each event to whoever has subscribed.

```

1  class EventBus:
2      def __init__(self):
3          self._listeners = {}
4
5      def subscribe(self, event, callback):
6          self._listeners.setdefault(event, []).append(callback)
7
8      def publish(self, event, **data):
9          for callback in self._listeners.get(event, []):
10             try:
11                 callback(**data)
12             except Exception as error:
13                 print(f"Listener {callback.__name__} failed: {error}")
14
15  def log_failure(url, status):
16      print(f"LOG: {url} returned {status}")
17
18  def send_alert(url, status):
19      print(f"ALERT: {url} is down")
20
21  bus = EventBus()
22  bus.subscribe("site_down", log_failure)
23  bus.subscribe("site_down", send_alert)
24
25  bus.publish("site_down", url="https://example.com", status=503)

```

The bus stores, for each event name, a list of callback functions, using `setdefault` from Lesson 1.9 to create the list the first time an event is seen. `subscribe` adds a listener; `publish` calls every listener registered for that event, passing along the event's data as keyword arguments. The publisher knows nothing about who is listening: it only says “this happened”. Adding a new reaction is one `subscribe` call, with no change to the code that detects failures. Two design details matter. Listeners are plain functions, not objects of a special class, since functions are values; and each call is wrapped in `try/except` so that one failing listener cannot prevent the others from running, the same isolation principle from Lesson 3.3 and Lesson 3.14. The cost of the pattern is indirection: reading the monitor no longer tells you everything that happens when a site goes down, so use it when several independent reactions exist or are likely to, not for a single direct call.

Try It Yourself 3.17b: Extend the event bus with an `unsubscribe(event, callback)` method that removes a listener. Subscribe two listeners, publish once, unsubscribe one, publish again, and confirm only the remaining listener fires the second time.

Strategy: Swappable Algorithms

The problem: sometimes there are several ways to do one job, and the choice should be made at runtime or by configuration: several ways to compute a shipping cost, to sort results, or to decide whether a website counts as healthy. Hardcoding one, with an `if` chain to select among them, mixes the choosing with the doing. In the Strategy pattern, each algorithm is packaged separately behind the same interface, and the code that needs one simply takes it as a parameter.

```

1  class Site:
2      def __init__(self, url, status, seconds):
3          self.url = url
4          self.status = status
5          self.seconds = seconds
6
7  def strict(site):
8      return site.status == 200 and site.seconds < 1.0
9
10 def lenient(site):
11     return 200 <= site.status < 400
12
13 def report(sites, is_healthy):
14     for site in sites:
15         label = "OK" if is_healthy(site) else "PROBLEM"
16         print(f"{site.url}: {label}")
17
18 sites = [Site("a.example", 200, 0.4), Site("b.example", 301, 0.2), Site("c.example", 200,
19     2.5)]
20 print("strict:")
21 report(sites, strict)
22 print("lenient:")
23 report(sites, lenient)

```

Here the strategies are plain *functions*, and `report` receives the one it should use as an ordinary argument, `is_healthy`. That is the whole pattern, and in Python it is often this small, because a function can be passed anywhere a value can. The class-based form, familiar from other languages, is an abstract base class with one method and a subclass per algorithm, and it is the better choice when a strategy needs its own state or several related methods. For example, a `TimeoutStrategy` class might hold a configurable threshold. But do not build a class hierarchy for what is really a single function: choosing the lighter form is itself good design. The benefit is that `report` never changes when you add a new rule, and each rule can be tested on its own.

Choosing Wisely

The four patterns answer four different questions. Need exactly one shared instance? Singleton, but try a module first. Need to create objects without hardcoding their class? Factory. Need one event to trigger several independent reactions? Observer. Need to swap one algorithm for another? Strategy. Notice also how many earlier lessons were quietly patterns: the alternative constructors of Lesson 2.3 are factories, the decorators of Lesson 2.7 wrap behavior in a way related to strategy, the abstract base classes of Lesson 2.8 define the interfaces that make all of them possible, and the callbacks in schedule from Lesson 3.7 are the observer idea in miniature. Patterns emerge naturally from good design, and the real skill is to recognize the problem first; reaching for a pattern before the problem appears is how simple programs become complicated ones.

Dependency Injection: Passing Collaborators In

Every pattern in this lesson shares a quiet idea that deserves its own name. When a piece of code needs another object, a notifier, a clock, an HTTP session, it can either create that object itself, or *receive* it from outside, as an argument. Receiving it is called *dependency injection*, and although the name sounds grand, it is just passing an argument.

```

1  import time
2
3  class Checker:
4      def __init__(self, fetch, notify, clock=time.time):
5          self.fetch = fetch          # a function that returns a status code
6          self.notify = notify        # a function that takes a message
7          self.clock = clock
8
9      def run(self, url):
10         started = self.clock()
11         status = self.fetch(url)
12         if status != 200:
13             self.notify(f"{url} returned {status}")
14         return status, self.clock() - started
15
16 def fake_fetch(url):
17     return 503
18
19 sent = []
20 checker = Checker(fetch=fake_fetch, notify=sent.append, clock=lambda: 100.0)
21 print(checker.run("https://example.com"))    # (503, 0.0)
22 print(sent)                                  # ['https://example.com returned 503']

```

Checker never imports requests, never sends a real message, and never reads the real clock unless you let it. In production you would pass a function that makes a real request and a function that calls a real notifier. In a test, you pass a fake function that returns 503 instantly and a list's append method to capture whatever would have been sent, and the whole check can be verified in a fraction of a millisecond, with no network, no delay, and a completely predictable clock. That is the practical payoff: code that receives its collaborators is easy to test, easy to reconfigure, and free of hidden global dependencies. It also makes the other patterns fit together naturally: the factory produces the objects, the strategy and the notifier are the collaborators being injected, and the observer's callbacks are injected listeners.

Putting the Patterns Together

It is worth seeing how the four patterns cooperate in a small system, since real programs use them in combination. Imagine the monitor of the capstone. A factory turns the names in a configuration file into notifier objects. A strategy function decides whether a result counts as unhealthy. The checker receives both, by injection, and publishes a `site_down` event when the strategy says so. The event bus delivers that event to every subscribed listener: one writes a log line, one sends the notifier's message, and one updates a failure counter. A single shared `Settings` object, created once at startup and passed to where it is needed, plays the part a Singleton might in another language.

```

1 def build_monitor(settings):
2     notifiers = [create_notifier(name) for name in settings.notifiers]
3     bus = EventBus()
4     bus.subscribe("site_down", log_failure)
5     for notifier in notifiers:
6         bus.subscribe("site_down", lambda url, status, n=notifier: n.send(f"{url}: {status}"))
7     return Checker(fetch=settings.fetch, notify=lambda msg: bus.publish("site_down", url=msg, status=0))

```

The details do not matter here; the shape does. Each decision, which notifiers to use, how to judge health, who listens, is made in exactly one place, and changing one leaves the rest alone. Notice the small detail `n=notifier` in the `lambda`: it binds the current notifier as a default value at the moment the function is created, avoiding the late-binding trap described in Lesson 3.10, where every `lambda` in a loop would otherwise use the *last* notifier. The whole point of these patterns is to keep a program easy to change, and the best test of a design is how few files you must touch to add a new notifier, a new health rule, or a new listener; with these structures, the answer is one.

Try It Yourself 3.17c: Write a small Greeter class that receives a `get_name` function and a `print_line` function in its constructor and has a `greet()` method that uses them. Test it with fakes that return a fixed name and collect the printed lines in a list, then build one with input and print for real use.

Common Mistakes to Avoid

- **Applying a pattern because it sounds professional.** A pattern earns its place only by solving a problem you actually have; unnecessary structure makes code harder to read.
- **Writing a Singleton class where a module would do.** Python's cached modules already give one shared object, more simply and more testably.
- **Growing an if/elif chain in the factory.** Use a dictionary mapping names to classes, so new kinds need no change to the factory logic.
- **Letting one observer's failure stop the others.** Wrap each callback in `try/except`, so a broken listener cannot silence the rest.
- **Building a class hierarchy for a stateless strategy.** A plain function does the same job with far less code; use classes only when the strategy needs state.

Lesson 3.17 Project: Pluggable Alert System

Write a program called `alert_system.py` that:

1. Defines an abstract `Notifier` base class and at least three concrete notifiers, and a factory function that creates one from a name using a dictionary, raising a clear `ValueError` for an unknown name.
2. Defines an `EventBus` with `subscribe`, `unsubscribe`, and `publish`, where a failing listener is caught and reported without stopping the others.
3. Defines at least two strategy functions for deciding whether a check result is *critical* (for example, by status code, and by response time), and a function that takes the strategy as an argument.
4. Wires them together: publishes a `check_failed` event for each of several simulated results, with one listener that logs, one that uses a notifier chosen by the factory, and one that counts failures.
5. Demonstrates swapping the strategy and the notifier by changing only a single configuration dictionary at the top of the file, with no other code edited.

This project deliberately combines all four patterns, and shows that each one lets you change one decision without touching the rest.

Lesson Summary: Quick Reference

- A design pattern is a named, reusable solution to a recurring problem; in Python many are simpler than in other languages
- Singleton: one shared instance via `__new__`, though a module is often the simpler choice
- Factory: one place decides which class to build, ideally through a dictionary mapping names to classes
- Observer: a source publishes events to subscribed callbacks, so it need not know who is listening
- Strategy: pass the algorithm in as an argument, as a plain function or, when it needs state, a class

Phase 3 Capstone Project: Automated Site Monitor

Just as Phase 1 and Phase 2 each closed with a project pulling together everything in that phase, this capstone does the same for the whole course. It is deliberately the largest project you will build here, and every technique it needs was covered in a lesson you have already completed; nothing in it is new. Building it is the clearest proof that the seventeen lessons of this phase are not seventeen separate tricks, but the parts of one working system, and it also draws directly on the functions, files, and error handling of Phase 1 and the classes, properties, decorators, and dataclasses of Phase 2.

What You're Building

A command-line tool, `site_monitor.py`, that checks a list of websites, records how each one is doing, and alerts you when something is wrong. It reads its settings from a configuration file, checks all the sites at once instead of one at a time, logs everything it does, runs on a schedule without supervision, and can be started from a terminal like any professional tool. Run once, it prints a status table and exits; run in the background, it keeps watching.

A sensible layout is a single project folder containing `site_monitor.py` (the entry point), `config.toml` (the sites and settings), `.env` (a secret, such as an alert webhook address, never committed), `.gitignore`, and a `results/` folder for generated files. You may split the code into several modules if you prefer, using the packaging ideas from Lesson 1.15, but a single well-organized file is fully acceptable.

Capstone Requirements

1. **Command-line interface (Lesson 3.1).** Use `argparse` with three subcommands: `check` (run every check once and print a table), `watch` (run repeatedly on a schedule), and `report` (summarize the saved history). Add a `--config` option with a default, a `--verbose` flag, and a `--workers` option that takes an integer. The program must return 0 when every site is healthy and a non-zero exit code when any check failed, with the entry point structured as `sys.exit(main())`.
2. **Configuration and secrets (Lesson 3.5).** Load defaults from a `Settings` dataclass (Lesson 2.9), overlay a `config.toml` read with `tomllib`, overlay environment variables loaded from a `.env` file, and let command-line options override everything. Validate the final values and raise a clear `ValueError` for a nonsense setting, such as a negative timeout. The alert webhook address must come only from the environment and must never appear in the source.
3. **Health checks with requests (Lesson 3.4).** Write a function that checks one URL with a `requests.Session`, always with a timeout, using `raise_for_status`, and returns a result object. Add retries with exponential backoff, but only for connection errors, timeouts, 429, and 5xx. A check must never raise an exception to its caller: every outcome, good or bad, becomes a result.
4. **A data model (Lessons 2.5, 2.9, 2.6).** Define a frozen dataclass `CheckResult` holding the URL, the status code (or `None`), the response time, an `ok` flag, an error message, and a timestamp. Give it a read-only property such as `status_label`, and a `__str__` that formats one readable line.
5. **Alert channels with patterns (Lesson 3.17).** Define an abstract `Notifier` base class (Lesson 2.8) with at least a console notifier and a file notifier, and create them through a factory that reads the names from the configuration. Connect a check outcome to its notifiers with an event bus, so that the checking code publishes a `site_down` or `site_recovered` event and knows nothing about who is listening. Make the definition of “unhealthy” a swappable strategy function, for example one that only looks at the status code and one that also treats a slow response as a failure.
6. **Concurrency (Lessons 3.12 and 3.14).** Check all sites at the same time with a `ThreadPoolExecutor`, using `submit` and `as_completed` so that results appear as they arrive. Collect every outcome, success or failure, into one list, so a single bad site never hides the others, and report the total elapsed time. Include a comment explaining why threads are the right tool for this workload.
7. **Logging (Lesson 3.6).** Set up logging once, at the entry point, with a console handler whose level depends on `--verbose` and a `RotatingFileHandler` that always records everything. Use a named logger, pass

values as separate arguments, use `logger.exception` when you catch an unexpected error, and install a `sys.excepthook` that records any crash that nothing else caught.

8. **Scheduling (Lesson 3.7).** Implement `watch` with the `schedule` library, running the full check every configured interval and a daily summary at a configured time, with each job wrapped in a `safe_job` decorator (Lesson 2.7) so one failed run cannot stop the loop. Build every path from `Path(__file__).resolve().parent`, and include, in a comment or in your notes, the exact `crontab` line that would run `check` instead.
9. **Persistence and history (Lessons 1.13, 3.3).** Append each round of results to a JSON file in `results/`, writing the file safely: build the new content first, write it to a temporary file in the same folder, and then replace the original, so a crash in the middle never corrupts the history. Handle a missing or corrupted file by starting fresh, and record what happened in the log.
10. **Lazy reporting (Lessons 3.8 and 3.9).** Implement `report` using generators: one that reads the history records one at a time, one that filters by site or by date, and one that computes a running uptime percentage per site, so the history could grow very large without the report loading it all at once. Use `itertools.groupby` (after sorting) or `collections.Counter` to group the records by site.
11. **Performance (Lessons 3.10 and 3.16).** Use `functools.lru_cache` or `cached_property` somewhere it genuinely helps, for example caching the parsed configuration or a repeated calculation, and use `time.perf_counter` or a timing context manager to measure each check. Add a hidden `--profile` flag that runs the command under `cProfile` and prints the five most expensive functions by `tottime`.
12. **Context managers and system interaction (Lessons 3.11, 3.2).** Write at least one custom context manager with `@contextmanager`, for example a timing block or a block that guarantees the results file lock is released. Use `shutil.which` and `subprocess.run`, with a list of arguments and a timeout, in a small `doctor` check that verifies the tools your monitor relies on are available, or pings a host, reporting each as a pass or fail.
13. **Debugging (Lesson 3.15).** Deliberately introduce one bug into a copy of your code, such as a wrong comparison in the strategy function, locate it using `pdb` with a breakpoint, and describe in a short comment block exactly which commands you used and what you saw.

A Suggested Order of Work

A project of this size is easy to lose control of, so build it in layers, running the program after each one. First, the data model and a single-site check function, tested on its own. Second, the configuration loader. Third, concurrent checking with a printed table. Fourth, logging, then the command-line interface around what already works. Fifth, the notifiers, factory, event bus, and strategy. Sixth, persistence and the lazy report. Seventh, scheduling. Only then add the caching, profiling, and debugging exercises. At every stage the program should run; a project that only works when it is finished is a project that is hard to debug. Commit the project to version control as you go if you use it, keeping the `.env` file out of it.

Stretch Goals

If the core system feels solid, extend it. Rewrite the checking step with `asyncio` and a semaphore (Lesson 3.13) and compare the timing against the threaded version. Add a `--format json` option to `report` using the `choices` validation from Lesson 3.1. Add a `retry-with-cooldown` rule to the event bus so that an alert for a still-failing site is sent only once every thirty minutes. Or add a `--dry-run` flag, as in Lesson 3.3, that runs every check and prints what alerts *would* have been sent, without contacting any real notifier.